分类号 TP3　　密级

UDC　　编号

中国科学院研究生院

博士学位论文

软硬件协同设计的 Java 虚拟机系统实现与性能优化

蔡嵩松

指导教师　胡伟武　研究员

中国科学院计算技术研究所

申请学位级别　工学博士　学科专业名称　计算机系统结构

论文提交日期　2011 年 5 月　论文答辩日期　2011 年 11 月

培养单位　中国科学院计算技术研究所

学位授予单位　中国科学院研究生院

答辩委员会主席　韩承德

声　　明

我声明本论文是我本人在导师指导下进行的研究工作及取得的研究成果。尽我所知，除了文中特别加以标注和致谢的地方外，本论文中不包含其他人已经发表或撰写过的研究成果。与我一同工作的同志对本研究所做的任何贡献均已在论文中作了明确的说明并表示了谢意。

作者签名：　　日期：

论文版权使用授权书

本人授权中国科学院计算技术研究所可以保留并向国家有关部门或机构送交本论文的复印件和电子文档，允许本论文被查阅和借阅，可以将本论文的全部或部分内容编入有关数据库进行检索，可以采用影印、缩印或扫描等复制手段保存、汇编本论文。

（保密论文在解密后适用本授权书。）

作者签名：　　导师签名：　　日期：

摘要

随着操作系统和计算机体系结构的快速发展，计算机基础平台出现了多样化的趋势，不同平台间的应用程序不能无缝迁移成为了制约基础平台发展的主要问题，应用程序跨平台的支持变得越来越重要。Java 语言是平台无关的，是实现应用程序跨平台支持的重要途径，Java 程序不需要移植就可以运行在多样化平台上，因此 Java 虚拟机对于以龙芯为代表的国产高性能处理器丰富软件环境具有重要的现实意义。

Java 程序的运行需要 Java 虚拟机的支持，但目前多数的 Java 虚拟机还存在效率较低的情况，目前基于纯软件和软硬件协同设计方法具有效率和硬件设计、验证复杂度等多方面的弊端，严重制约着 Java 技术的发展。为了研究上述问题，提高 Java 虚拟机的性能，促进应用软件的跨平台融合，本文在现有平台的基础上，实现了一个简洁高效的 Java 虚拟机，并对该 Java 虚拟机的各组成部分进行了系统的优化。

本文的主要贡献如下：

1、以 Java HotSpot 虚拟机为基础，实现了基于龙芯国产高性能处理器的 Java 虚拟机系统。本文以 Java HotSpot 虚拟机为基础，修改 HotSpot 虚拟机中的类装载子系统以及运行时环境等，使这些环境能够符合龙芯处理器的结构设计，从而能够在龙芯硬件平台上稳定运行，并根据龙芯处理器的硬件结构，对 HotSpot 虚拟机做了初步优化。在龙芯处理器硬件平台上，以 HotSpot 为基础设计的 Java 虚拟机可以稳定运行, 并且性能达到了较高水平, 为本文提供了良好的研究平台。

2、通过在 CPU 中增加硬件 CAM 查找表，提出了一种软硬件协同设计的混合的虚方法调用机制，有效地解决了同一方法调用点频繁调用多种不同类型方法的问题。实验表明，采用本方法，virtual_test 测试程序虚方法调用点的类型比较命中率从 13.3%升高到了 76.4%，性能有了 16.2%的提升；SPECjvm98 性能平均提升 6.4% 。

3、通过在 CPU 中添加有界访存和 128 位访存指令，提出了一种软硬件结合的访存优化方法，有效地降低了数组越界检查的开销，提高了内存拷贝的效率，优化了 Java 程序的整体性能。使用有界访存指令，科学计算类测试 SciMark2.0 获得了 27%的性能提升，通用测试 SPECjvm2008 获得了 1.5%的性能提升；使用 128 位访存指令，SPECjvm2008 性能平均提升 5.9% 。

4、通过在 CPU 中增加硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。实验表明，生成代码执行时的 cache 失效次数降低了 6.9%，测试 SPECjvm2008 中程序最高获得了 17.9%的性能提升，平均性能提升 4.2%。

本文工作应用于我国首款四核通用处理器龙芯 3 号芯片中，有效地解决了应用软件的跨平台支持问题，对龙芯平台上产品化 Java 虚拟机的构建具有重要意义。

关键词： Java 虚拟机技术，类装载器，运行时环境，执行引擎，软硬件协同设计，龙芯

Abstract

With the rapid development of the operating system and computer architecture, diversification trend of computer platforms emerges. Under this circumstance, the migration of applications over different platforms has become the main issue of restricting the basic platform development, and cross-platform supporting of the applications has become increasingly important. Java language is platform independent, which is an important way to achieve cross-platform for applications, Java programs can run without porting among diverse platforms, thus Java Virtual Machine is important to enrich the software environments in the Loongson platform.

Java Virtual Machine is necessary for supporting the executions of Java programs, but the efficiencies of most Java Virtual Machines are still low. There are some defects in pure software and hardware/software co-design method, such as less efficient, higher verification complexity and so on. Because there are some problems in Java Virtual Machine, they seriously restrict the development of Java technology. In order to solve these problems and improve the performance of Java Virtual Machine, and promote cross-platform application integration, we implement a simple and efficient Java Virtual Machine, and optimize the components of Java Virtual Machine systematically.

The main contributions of this paper are as follows:

1、Based on the Java HotSpot Virtual Machine, we implement the Java Virtual Machine system on the platform of Loongson. We modify the HotSpot virtual machine class loader subsystem and runtime environment, so that these environments can meet the structural design of the Loongson processors, the Java Virtual Machine can run stably on the platform of Loongson. We have also made preliminary optimization based on the Loongson architecture. Finally, on the platform of Loongson, the Java Virtual Machine designed by us can stably run with high performance, it provides a good platform for this paper.

2、We implement the hardware CAM lookup table, and present a mechanism of virtual method call based on hardware and software co-design. It solves the problem that the types of objects change frequently at one call site, and the corresponding evaluation shows that it improves the cached hit rate from 13.3% to 76.4% and improves performance by 16.2% for the virtual test; it also improves the performance of SPECjvm98 by 6.4% on average.

3、We implement the memory accessing instructions with the functions of bound memory accessing and 128-bit memory accessing, and present a new method of memory accessing in Java Virtual Machine. The method decreases the overhead of array out of bound check,

improves the speed of memory copy, and optimizes the overall performance. With the boundary memory accessing intructions, the evaluation shows the improvement of performance on the scientific SciMark benchmark suite by 27% on average, the performance of SPECjvm2008 is also improved by 1.5%; with the 128-bit memory accessing instructions, the performance of SPECjvm2008 is improved by 5.9% on average.

4、We implement cache locking mechanism, and present a JIT compile method based on software and hardware co-design. The number of cache misses decreases by 6.9%, the highest improvement of performance attains 17.9%, and the program has 4.2% performance improvement on the SPECjvm2008 benchmark suite on average.

The works of this thesis are implemented on the Loongson processor, the problem of the application cross-platform supporting has been solved efficiently. In the future, it is important to build the product version of Java Virtual Machine on the platform of Loongson.

Keywords: Java Virtual Machine technology, class loader, run-time environment, execution engine, hardware and software co-design, Loongson

目 录

摘要 I

Abstract III

目 录 V

图目录 IX

表目录 XI

第一章 绪论 1

1.1 应用程序跨平台支持的需求 1
1.2 Java 体系结构的广泛应用 1
1.3 Java 体系结构基本技术 4
1.3.1 Java 程序的执行流程 4
1.3.2 Java 体系结构简介 4
1.3.3 Java 体系结构的特性 8
1.4 Java 虚拟机技术及其意义 10
1.4.1 Java 虚拟机技术面临的主要问题 10
1.4.2 Java 虚拟机技术对国产处理器的重要意义 10
1.4.3 高效 Java 虚拟机的设计方向 11
1.5 本文的主要贡献 11
1.6 论文的组织 12

第二章 Java 虚拟机相关工作介绍 15

2.1 基于软件设计的 Java 虚拟机 15
2.1.1 基于编译器设计的 Java 虚拟机系统 16
2.1.2 解释器编译器混合设计的 Java 虚拟机系统 21
2.1.3 软件 Java 虚拟机的关键技术 25
2.2 软硬件协同设计的 Java 虚拟机系统 26
2.2.1 直接仿照 JVM 的结构设计的 Java 专用处理器 26
2.2.2 以协处理器方式工作的 Java 字节码硬件解释器 29
2.2.3 以超长指令字 VLIW 技术设计的并行微处理器 30
2.3 本章小结 32

第三章 龙芯 Java 虚拟机系统硬件支持与研究平台 33

3.1 引言 33
3.2 龙芯高性能通用处理器硬件平台 33
3.2.1 GS464 处理器核内部结构 33
3.2.2 四核龙芯 3 号处理器结构 35
3.3 龙芯 Java 虚拟机系统的软件设计与实现 36
3.3.1 Openjdk Java 虚拟机系统的发展 36
3.3.2 龙芯 Java 虚拟机系统 37
3.3.3 本文采用的主要测试程序 43
3.4 本章小结 43

第四章 类装载器虚方法调用的分析和优化 47

4.1 引言 47
4.2 相关工作 48
4.3 Java 程序方法调用的主要类型和比例 50
4.4 HotSpot 虚方法调用单态内联高速缓存技术 52
4.5 虚方法调用全局索引表的硬件设计 54
4.6 软硬件协同设计的虚方法调用算法 55
4.7 实验结果和分析 58
4.8 本章小结 60

第五章 运行时环境的访存分析和优化 61

5.1 引言 61
5.2 相关工作 62
5.3 新型访存的硬件设计 64
5.3.1 越界检查指令的硬件设计 64
5.3.2 128 位访存指令的硬件设计 65
5.4 软硬件协同设计的 Java 虚拟机访存机制 66
5.4.1 数组越界检查软硬件优化 66
5.4.2 上下文切换的软硬件优化 67
5.5 实验结果和分析 68
5.6 本章小结 71

第六章 执行引擎编译器系统的设计和优化 75

6.1 引言 75
6.2 相关工作 76

6.2.1 静态 cache 锁机制 77
6.2.2 动态 cahce 锁机制 78
6.3 cache lock 机制原理简介 79
6.4 Java 虚拟机即时编译器访存行为分析 80
6.5 cache lock 机制硬件设计 83
6.6 软硬件协同设计 Java 虚拟机编译器系统 85
6.7 实验结果和分析 88
6.8 本章小结 89
第七章 结束语 91
7.1 引言 91
7.2 本文工作总结 91
7.3 未来研究方向 92
参考文献 95
致谢 i
作者简历 iii

图目录

图 1.1 Java 程序执行流程 4
图 1.2 Java 编程环境 5
图 1.3 Java API 实现 Java 程序平台无关性 7
图 1.4 Java 虚拟机的基本结构图 8
图 1.5 在 Java 平台上运行的 Java 程序 9
图 2.1 Jalapeno 动态编译器工作流程 17
图 2.2 Jalapeno 优化的编译器编译过程 18
图 2.3 MRL VM 编译系统结构 19
图 2.4 Self-93 编译过程 21
图 2.5 Java HotSpot 虚拟机结构 22
图 2.6 Java HotSpot 虚拟机客户端编译器结构 23
图 2.7 IBM JVM 动态编译系统结构 25
图 2.8 PicoJava 处理器核组成 27
图 2.9 PSC1000 微处理器结构 28
图 2.10 硬件解释器结构图 29
图 2.11 硬件解释单元内部结构 30
图 2.12 MAJC 处理器集群 31
图 3.1 龙芯 GS464 处理器微结构 34
图 3.2 四核龙芯 3 号处理器芯片结构 35
图 3.3 Openjdk 发展线路图 37
图 3.4 Java 虚拟机的内部体系结构 38
图 3.5 用户自定义类装载器虚拟机接口 39
图 3.6 线程调用 Java 方法和本地方法时的栈 42
图 4.1 使用单态内联高速缓存的虚方法调用 48
图 4.2 polymorphic 多态内联高速缓存 50
图 4.3 SPECjvm2008 虚方法调用比例 52
图 4.4 HotSpot 中对象内联结构 53
图 4.5 单态内联高速缓存状态转换图 53
图 4.6 CAM 结构 54
图 4.7 CAM 项数评估 55
图 4.8 完成写操作的 CAM 表项 56
图 4.9 虚方法调用机制流程图 57

图 4.10 三种动态解析技术比较 57
图 4.11 virtual_test 程序 58
图 4.12 改进前后 SPECjvm2008 测试结果 59
图 5.1 数组越界检查优化后的 HotSpot 执行流程 63
图 5.2 有界访存指令硬件实现 65
图 5.3 优化前 iaload 数组访问实现 66
图 5.4 优化后 iaload 数组访问实现 67
图 5.5 memcpy 改进前后的内部实现 68
图 5.6 数组越界检查优化前后 SciMark2.0 性能对比 69
图 5.7 数组越界检查优化前后 SPECjvm2008 性能对比 69
图 5.8 上下文切换优化前后 SPECjvm2008 性能对比 70
图 6.1 示例程序控制流图及内存 cache 分布 79
图 6.2 使用锁机制之后的 cache 分布 80
图 6.3 即时编译器工作过程 81
图 6.4 即时编译器生成代码尺寸统计 82
图 6.5 增加二级 cache 锁机制后处理器存储结构 84
图 6.6 mpegaudio 程序中单个方法生成代码尺寸 85
图 6.7 mpegaudio 中生成代码地址分布 86
图 6.8 使用 cache 锁机制的即时编译器执行流程 87
图 6.9 使用 cache 锁机制的处理器数据流 87
图 6.10 使用 cache 锁前后处理器直接读取内存次数比较 88
图 6.11 使用 cache 锁机制即时编译器性能测试 89

表目录

表 3.1 SPECjvm98 基准测试程序简介 44
表 3.2 SPECjvm2008 基准测试程序简介 45
表 4.1 CAM 表操作指令 55
表 4.2 改进前后 virtual test 测试结果 59
表 5.1 有界访存指令 72
表 5.2 地址比较指令 73
表 5.3 128 位访存指令 73
表 6.1 使用锁机制前后 cache 失效对比 80
表 6.2 龙芯 3 号 cache 参数 83

第一章　　绪论

应用程序跨平台的广泛需求，使我们需要寻找一种跨平台支持的有效方法，实现程序跨平台的无缝迁移。虚拟机［1］是实现应用程序跨平台支持的普遍技术，而 Java 语言的出现，使得虚拟机技术被进一步推广，Java 语言［2］与 Java 虚拟机天生就具有平台无关性、安全性和网络移动性，能够很好满足应用程序跨平台的要求。因此，Java 虚拟机对于以龙芯[3,4,5]为代表的国产高性能处理器丰富软件环境具有重要的现实意义。然而，目前基于纯软件和软硬件协同设计方法［6］具有效率和硬件设计、验证复杂度等多方面的弊端。本文采用软硬件结合的方法，在不增加通用处理器额外开销的情况下，适当改进处理器和虚拟机的结构，设计和实现了一个简洁高效的 Java 虚拟机。本章内容对上述问题做了简要介绍。

1.1 应用程序跨平台支持的需求

近年来，随着操作系统和计算机体系结构的快速发展，计算机基础平台出现了百家争鸣的局面。一方面，在操作系统基础软件领域，已经不是微软 windows 一家独大的局面，特别是在服务器领域，linux 操作系统所占市场份额已经相当可观，另外，macOS 等操作系统也在崛起，逐渐成为主流的操作系统；另一方面，计算机体系结构从发展之初就分为 CISC 和 RISC 两大派系，虽然属于 CISC 的 x86 架构［7］属于主流架构，但是 MIPS、PowerPC 等硬件结构发展迅速，也成为了不可忽视的一股力量。

丰富的系统平台虽然有利于计算机技术的快速发展，但是也给软件开发者带来了极大的挑战。由于不同平台所能够运行的二进制程序是不一样的，例如不同的操作系统之间能够运行程序的接口不同，不同的硬件平台上运行程序的指令集架构不同，所以针对多个平台，软件开发者需要对相同的软件开发多个不同的版本，这大大增加了软件开发者的工作量，直接限制了不同平台的软件丰富性。软件开发者往往因为繁重的工作不愿意移植程序到非主流的结构，导致了非主流结构上软件的进一步匮乏，间接地限制了计算机系统结构的发展。由于应用软件跨平台支持的广泛需求，需要寻找一种跨平台支持的有效方法，实现软件“一次编写，多次运行”的目的。

1.2 Java 体系结构的广泛应用

虚拟机［1］是实现应用程序跨平台支持的普遍技术，通过使用虚拟机，软件移植变得方便简洁——只需为不同的架构开发不同的虚拟机平台，而软件本身并不需要修改。Java 语言的出现，使得虚拟机技术被进一步推广。Java 语言是一种面向对象的程序设计语言，可以撰写跨平台的应用软件。它最初的目标定位于家用电器等小型系统的编程语

言，解决诸如电视机、电话、闹钟等家用电器的通讯和控制问题。但是由于这些智能化家电市场需求没有预期高，并且随着互联网的发展，以网络为核心的硬件环境逐渐增长，其包含的设备越来越广泛，成为了软件开发者需要面临的挑战。由于 Java 语言具有平台无关性、安全性和网络移动性等特点，因此，Java 语言也就逐渐成为了面向网络设计的主流语言。

一般网络都有很多不同的设备、不同的硬件体系、不同的操作系统，用于不同的用途。Java 通过创建与平台无关的程序来解决这个问题，一个 Java 程序可以不需要修改就在不同架构的计算机和设备上运行。(1) 与固定与特定平台的程序相比， Java 平台无关的程序会更容易编写、管理和维护，代价也更低。(2) 安全性是网络应用的一个重要需求，网络在带来巨大方便的同时，也为恶意程序打开通道，这些程序可能造成难以想象的后果，而 Java 语言可以提供一个受保护的环境，它对很多不可靠的 Java applet 活动做了限制，例如对本地硬盘的读写操作、创建新的进程、装载新的动态链接库等等，由于下载的代码不可能进行这些特定的操作，这就使 Java 安全模型可以使用户免受恶意代码的威胁。(3) 无所不在的网络的优势之一就是在线程序发布，Java 利用这个优点在网络上传送二进制代码。Java 程序是平台无关的，这使网络上移动代码和对象更加有效可行，同样的代码可以被送到网络中所有计算机和设备上，在不同硬件平台上运行的分布式系统中，各个不同组件也可以互相交换对象。通过这个能力，Java 程序的发布比传统的程序发布更加方便廉价。这也为版本控制带来了好处。因为最新版本的 Java 程序可以在用户需要的时候从网络上得到，所以不需要担心最终用户运行程序的版本。另外 Java 程序通过对象序列化 API 和 RMI（远程方法调用）实现了对象移动，在 Java 的底层结构之上，API 和 RMI 为系统中对象的共享提供了基础。对象在网络上的运动性为分布式系统编程创造了新模型，有效地把面向对象编程的优点带到了网络上。

平台无关性、安全性和网络移动性，使得 Java 和发展中的网络计算环境相得益彰。因为 Java 程序是平台无关的，可以在网络上移动的代码和对象就更加有效可行，同样的代码可以被送到网络连接的所有计算机和设备上。Java 的内置安全性框架也可以使网络上的软件传送更加可靠。

尽管 Java 目前在网络领域声名卓著，但它最初却是被期望用于嵌入式设备和消费电器领域的，而不是桌面计算机。这样的设计目标，部分原因是由于尽管微软和 Intel 公司在桌面市场占有统治地位，但是它们在嵌入式设备和消费电器市场并不具备这种优势。微处理器开始越来越多地出现在音频视频装置、蜂窝电话、打印机、传真机、复印机等设备中，而这些微处理器将可以连接到网络。因而，Java 最初的设计目标之一就是提供某种方式，让软件可以通过网络交付到任意种类的嵌入式设备中，不管它的微处理器和操作系统是什么。

为了达到这个目标，Java 运行时系统（Java 平台）不得不设计得尽量紧凑，以便它可以使用嵌入式系统中有限的资源。嵌入式微处理器通常有一些特殊的限制，比如容量小的内存、没有磁盘、没有图形化显示，甚至没有显示功能。这样的限制，意味着嵌入

式设备和消费品系统没有必要支持完整的 Java API。

针对嵌入式和消费性电器设备的特殊需求，Sun 创建了几个具体的 Java 平台，它们包含部分的 Java API：

- Java 个人版平台，用于消费性电器设备；
- Java 嵌入式平台，用于嵌入式系统；
- Java 卡平台，用于智能卡；

这些 Java 平台是由 Java 虚拟机和比标准 Java 平台更小的运行时库组成，个人版平台和标准版平台的区别就在于前者比后者提供的 Java API 运行时库内容更少，而嵌入式平台则比个人版平台还要少，最少的就是 Java 卡平台。虽然这些 Java 平台依次面向更小的执行环境，在资源利用上有更严格的限制，但是它们所提供的 API 之间并非是简单的子集关系。每一个平台提供的 API 子集都是面向一个特定的目标领域，因此也包含专门针对该目标领域的 Java API。Java 卡平台是专门面向智能卡的。除了提供的 API 集最少之外，也只使用了整个 Java 虚拟机指令集的一部分。因而只有特定的 Java 程序才能在智能卡上运行。

Sun 试图用这三个子集来匹配嵌入式设备和消费性电器市场的需求，或者说市场中对 API 的不同需求产生了这三种不同的 API 子集。因为嵌入式系统有特殊的限制，特别是内存很小，磁盘容量也有限，嵌入式系统的生产商不能容忍在设备中包含不需要的 API。就算 Sun 定义了这三个不同的子集，生产商仍然觉得需要定义自己的 API 子集。

最终 Sun 改变了他们定义的 API 标准。Sun 不再希望用一个 API 子集满足所有的需要，而是定义了一个非常小的 API 指令集，成为 Java 平台的微型版本（J2ME）。在 J2ME 之上，Sun 计划针对不同的行业定义他们自己的子集，以适合各自的市场需要。

因为 Java 平台很紧凑，它可以在很多嵌入式系统和消费性电器中实现。Java 平台的紧凑性并没有把实现限制在很小的范围。Java 平台可以在个人计算机、工作站和大型机上保持伸缩性。虽然在 Java 开始的几年里，Java 虚拟机在服务器端遇到过伸缩性的困难，但是虚拟机现在已经针对服务器做过优化，很多实现可以在服务器端得到非常好的性能。在这个方面，Sun 定义了一个 API 的超集：Java 企业版（J2EE）。除了标准的 Java API 之外，J2EE 包含了在企业服务环境中非常有用的一些技术，比如 servlet 和 Enterprise JavaBean。

最终，Sun 改变过的 API 定义方式得到了三个基础 API 集合，它们表现 Java 平台的不同伸缩性：

- 企业版（J2EE）
- 标准版（J2SE）
- 微型版（J2ME）

在高端，企业版的存在表明了 Java 平台在高端服务的可用性；在中端，标准版提供了在浏览器中启动传统 applet 的功能和桌面环境下的 Java 平台；在低端，微型版通过不同的行业子集，显示了 Java 平台可以向下伸缩，适应消费性电器市场和嵌入式系统需求

的能力。

在 Sun 制定基于 Java 平台的高中低三个版本之后，Java 软件变得越来越丰富：在企业应用方面，基于 Java 的 J2EE 在内部众多系统的整合、系统的延展性、安全性方面对比微软的.net 平台占有优势，符合 J2EE 规范的产品层出不穷，IBM 的 WebSphere 和 BEA 的 WebLogic 等基于 Java 的应用软件在目前服务器领域占了很大比例；在中间件市场，基于 Java 的实现也逐渐成为主流；在桌面应用方面，在办公软件方面有永中 office，在软件开发方面有 Eclipse，在个人通信平台方面有 Luma QQ 等，这些几乎构成了一整套的桌面环境；在嵌入式领域，基于 Java 的 J2ME 平台逐渐成为通信设备/消费类设备的基本功能，J2ME 应用程序的开发和部署的灵活性将有效地满足日益增长的无线领域的需求。

1.3 Java 体系结构基本技术

1.3.1 Java 程序的执行流程

Java 程序的执行是平台无关的。Java 高级语言源程序经过编译后，生成平台无关的中间代码 Java 字节码，Java 字节码再经过不同平台的 Java 虚拟机［8］编译后生成能够执行在相应平台上的二进制代码。具体的 Java 程序执行流程如图 1.1 所示。

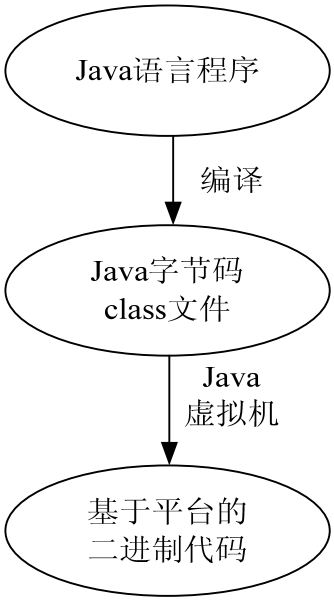

图 1.1 Java 程序执行流程

1.3.2 Java 体系结构简介

Java 体系结构由 Java 程序设计语言、Java class 文件、Java 应用编程接口和 Java 虚拟机等四个部分组成。当编写并运行一个 Java 程序时，就同时体验了这四种技术。用 Java 编程语言编写源代码，把它编译成 Java class 文件，然后在 Java 虚拟机中运行 class 文件。

当编写程序时，通过调用类（这些类实现了 Java API）中的方法来访问系统资源（比如 I/O）。当程序运行的时候，它通过调用 class 文件中实现了 Java API 的方法来满足程序的 Java API 调用。图 1.2 体现了这四者之间的联系。

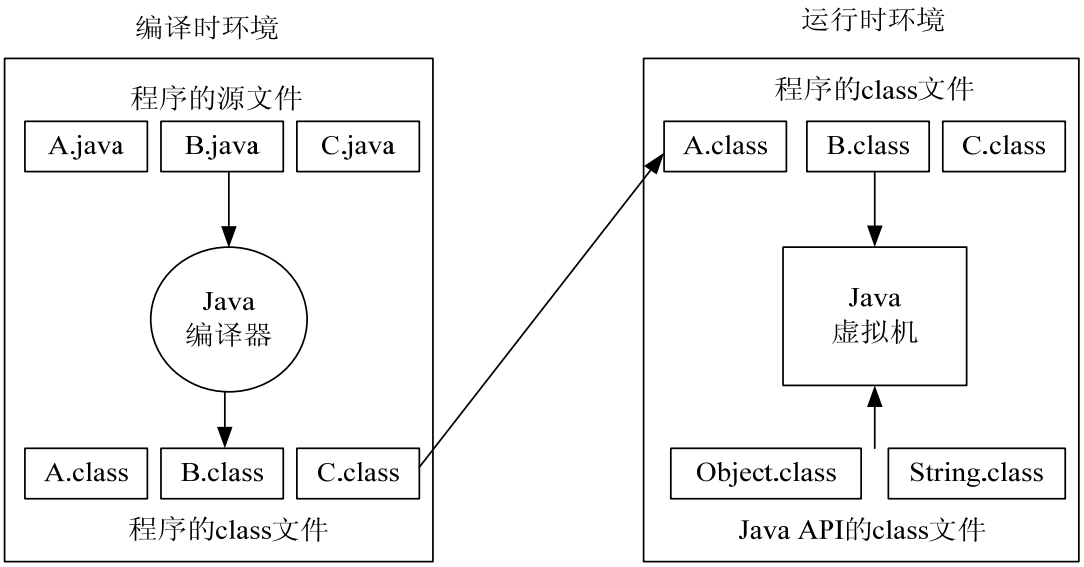

图 1.2　Java 编程环境

Java 虚拟机和 Java API 一起组成了一个平台，所有 Java 程序都在这上面编译。Java 程序可以在不同的计算机上运行，这是因为 Java 平台自己可以用软件实现。

1.3.2.1 Java 程序设计语言

Java 语言是一种面向对象的语言，它使用一种新的方法把已经在其他语言中得到验证的概念和技术组合到了一起。这些概念和技术使得 Java 程序设计语言成为强有力的工具。使用 Java 语言编写程序，能够充分利用诸如面向对象、多线程、结构化错误处理、垃圾收集、动态链接、动态扩展等许多软件技术。

作为一种程序语言，Java 和别的语言相比，有其自身的优缺点。优点是它能够提高开发者的效率；而它最主要的缺点在于执行速度较慢。首先，Java 中没有强制转换指针类型或者指针运算的方法；其次，Java 避免无意间破坏内存的方法是自动垃圾收集，在 Java 中只需要停止对一个对象的引用，一段时间后，垃圾收集器会自动回收这个对象所占据的内存；Java 在运行时保护内存完整性的第三个办法是数组边界检查，在 Java 中数组是发展完备的对象，在每次使用数组的时候，Java 都会检查数组的边界，但这是以牺牲性能为代价的；最后，Java 在每次使用对象引用的时候，都会对引用进行检查，确保这些引用不为空值。

使用 Java 语言会提高生产效率，效率提升就带来了开发周期的缩短和开发费用的降

低。如果利用到 Java 程序潜在的平台无关性，就会进一步提高效率。

1.3.2.2 Java Class 文件

Java class 文件主要实现了平台无关性。它为 Java 程序提供独立于底层主机平台的二进制形式的服务，这种途径打破了 C 或者 C++等语言所遵循的传统。使用这些传统语言写的程序通常首先被编译，然后被链接成单独的、专门支持特定硬件平台和操作系统的二进制文件。通常情况下，一个平台上的二进制可执行文件不能在其他平台上工作。而 Java class 文件是可以运行在任何支持 Java 虚拟机的硬件平台和操作系统上的二进制文件。

当编译和链接一个 C++程序的时候，所获得的可执行二进制文件只能在指定的硬件平台和操作系统上运行，因为这个二进制文件包含了目标处理器的机器语言。而 Java 编译器把 Java 源文件的指令编译成字节码，这种字节码就是 Java 虚拟机的“机器语言”。

除了特定处理器的机器语言之外，传统二进制可执行文件的另一个依赖于具体平台的属性是整数的字节顺序。比方说，在支持 X86 系列处理器的二进制可执行文件中，字节顺序是低位在前，而对于 MIPS 处理器，则是高位在前。在 Java class 文件中字节顺序是高位在前，这与使用何种平台产生和使用这个文件无关。

除了对于平台无关性的支持，Java class 文件还在支持网络移动性的 Java 体系结构中担当了至关重要的角色。首先，class 文件设计得紧凑，因此它们可以快速地在网络上传送。其次，由于 Java 程序是动态链接和动态扩展的，class 文件可以在需要的时候才下载。这个特点使得 Java 应用程序能够安排从网络上下载 class 文件的时间，从而可以最大限度地减少终端用户的等待时间。

1.3.2.3 Java 应用编程接口（API）

Java API 是运行库的集合，它提供一套访问主机系统资源的标准方法。编写 Java 程序时，可以假设在任何 Java 虚拟机上都能够获取 Java API class 文件。运行 Java 程序时，虚拟机装载程序所使用的 Java API class 文件。所有被装载的 class 文件（包括从应用程序中和从 Java API 中提取的）和所有已经装载的动态库（包含本地方法）共同组成了在 Java 虚拟机上运行的整个程序。

Java API 的重要作用之一就是它与平台相关，为访问主机上的本地资源，Java API 调用了本地方法，如图 1.3 所示，由于 Java API class 文件调用了本地方法，Java 程序就不需要再调用它们了，这就实现了 Java 程序的平台无关。通过这种方法，Java API class 文件为底层主机提供了具有平台无关的、标准接口的 Java 程序。对 Java 程序而言，无论平台内部如何，Java API 都会有同样的表现和可预测的行为。正是由于在每个特定的主机平台上都实现了 Java 虚拟机和 Java API，Java 程序自身就能够成为具有平台无关的程序。

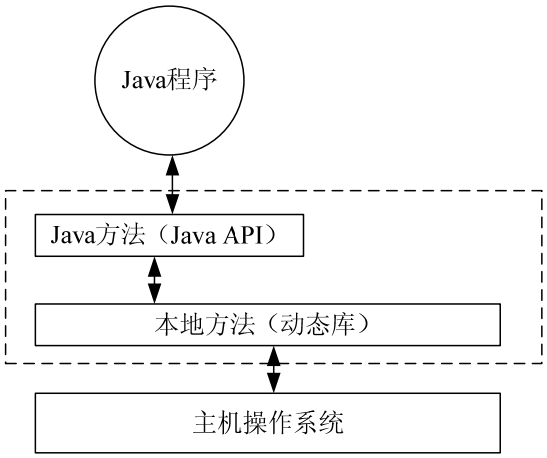

图 1.3 Java API 实现 Java 程序平台无关性

除了对平台无关性的促进之外，Java API 在 Java 安全性方面也做出了贡献。当 Java API 的方法进行任何有潜在危险的操作（比如进行本地磁盘写操作）之前，都会通过查询安全管理器来检验是否得到了授权。安全管理器是一个为应用程序提供自定义安全策略的特殊对象。例如，安全管理器能够限制对本地磁盘的访问。Java 1.2 版本之前，如果应用程序通过调用 Java API 的方法来请求对本地磁盘进行写操作，那个 API 方法会首先请求安全管理器进行验证。如果从安全管理器得知磁盘访问被禁止，Java API 会拒绝执行写操作。在 Java 1.2 版本中，安全管理器的工作被访问控制器所取代。访问控制器是一个类，该类用来执行栈检验，以决定是否准许某种操作。通过强制执行安全管理器建立的安全策略，Java API 促进了安全环境的建立，在这种安全环境中，可以运行具有潜在危险的代码。

1.3.2.4 Java 虚拟机

Java 虚拟机［8］是 Java 体系结构中的主要组成部分，它的主要任务是装载 class 文件并执行其中的字节码。Java 虚拟机包含一个类装载器，它可以从程序和 API 中装载 class 文件。只有程序执行时需要的类才会被装载。

装载后的字节码由执行引擎来执行。不同的 Java 虚拟机中，执行引擎可能实现得非常不同。第一种执行引擎就是一次性解释字节码，这种执行引擎启动开销最小，它将待执行的字节码单条解释并执行；第二种执行引擎是即时编译器（Just-in-time compiler）［9］，在这种执行引擎中，第一次被执行的字节码会被编译成本地机器代码。编译出的本地机器代码会被缓存，方法再次被调用的时候可以重用；第三种执行引擎是自适应优化器，在这种执行引擎中，虚拟机开始的时候解释字节码，但是会监视运行中程序的活动，并且记录下使用最频繁的代码段。程序运行的时候，虚拟机只把那些活动最频繁的代码编译成本地代码，其他的代码由于使用的并不频繁，继续保留为字节码——由虚拟

机继续解释它们。一个自适应优化器可以使得 Java 虚拟机在 80%-90%的时间里执行被优化过的本地代码；最后一种虚拟机由硬件芯片构成，它用本地方法执行 Java 字节码，这种执行引擎实际上是内嵌在芯片里的。Java 虚拟机的基本结构如图 1.4 所示。

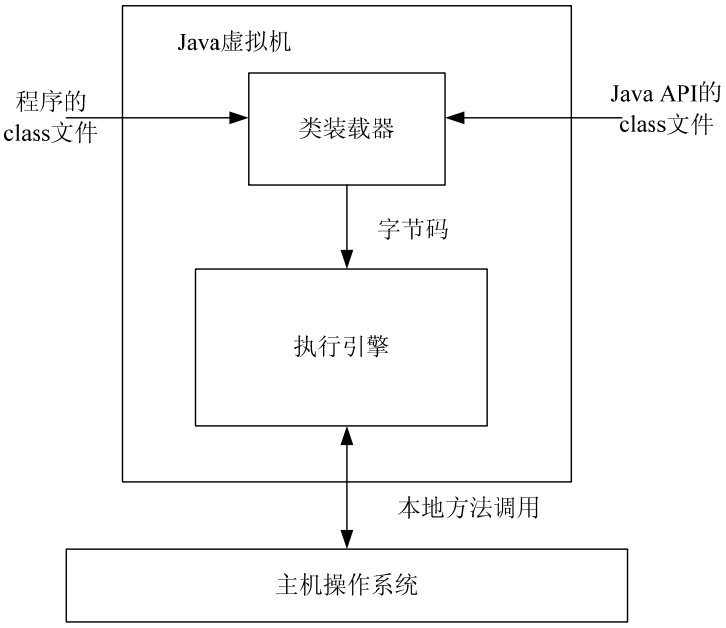

图 1.4　Java 虚拟机的基本结构图

Java 中有两种方法：Java 方法和本地方法。Java 方法由 Java 语言编译成字节码，存储在 class 文件中。本地方法是由其他语言（比如 C，C++，或者汇编语言）编写的，编译成和处理器相关的机器代码。本地方法保存在动态链接库中，格式是各个平台专有的，Java 方法是平台无关的，本地方法是平台相关的。运行中的 Java 程序调用本地方法时，虚拟机装载包含这个本地方法的动态库，并调用这个方法。通过本地方法，Java 程序可以直接访问底层操作系统的资源。

1.3.3 Java 体系结构的特性

平台无关性、安全性和网络移动性是 Java 体系结构的三大特性。其中，最重要也是最显著的特性就是平台无关性。正是由于 Java 体系结构的平台无关性，才使它成为解决应用程序跨平台的有效方法。对平台无关性的支持，是分布在整个 Java 体系结构中的，它的所有组成部分都扮演着重要角色，而这其中最重要的就是 Java 虚拟机。Java 虚拟机在 Java 程序与底层的硬件和操作系统交互时，起到了缓冲的作用。无论 Java 程序被部署到何处，它只需要与 Java 虚拟机交互，而不需要担心底层的硬件和操作系统。因此它就能够运行于任何拥有 Java 虚拟机的计算机。图 1.5 就是在不同平台上运行的 Java 程序。

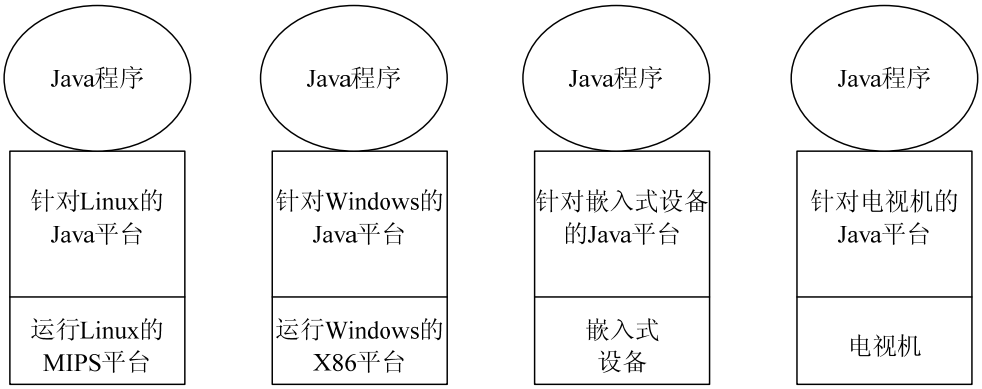

图 1.5 在 Java 平台上运行的 Java 程序

为了实现安全性，Java 虚拟机将类装载器作为名字空间的一部分。不同的类装载器装载的类属于不同的名字空间，这样，虚拟机可以将代码归入不同的类型，并确定某类型的代码可以进行的操作。作为一种类型安全的语言，Java 语言还具有许多内置的安全特性：类型引用的安全转换、结构化的内存访问（不能使用指针来访问任意的内存地址）、自动内存管理（垃圾收集）、数组边界检查、空引用检查等等。这些安全特性使得程序员更容易避免编程时引入的错误，从而使得用 Java 语言进行软件开发变得非常方便和高效，特别是对于大规模的服务器端应用来说更是如此。

Java 语言是网络应用中广泛流行的语言，网络移动性是它的重要特性。Java 体系结构对网络移动性的支持是和它对平台无关性和安全性的支持密不可分的。平台无关性使得在网络上传送程序更加容易，因为不需要为每个不同的主机平台都准备一个单独的软件版本，一个版本就可以运行于所有平台。Java 的安全性也促进了网络移动性的推广，因为用户就算从不信任的来源下载 class 文件，也不必担心造成严重的后果。除了平台无关性和安全性之外，Java 体系结构对网络移动性的支持主要集中在 Java 语言动态链接、动态扩展和紧凑的 class 文件格式。在网络传送时，Java 体系结构把单一的二进制可执行文件切割成 Java class 文件来传输，由于 Java 程序是动态链接、动态扩展的，只有在 Java 程序运行使用到某个 class 文件才动态装载，因此用户不需要等待所有的程序 class 文件都下载完毕，就可以开始运行程序，这样就大大解决了网络传输时间慢的问题。另外，class 文件包含的字节码设计得非常紧凑，同样大小的文件包含更多的指令码，进一步降低了传输时间。

1.4 Java 虚拟机技术及其意义

1.4.1 Java 虚拟机技术面临的主要问题

Java 体系结构的核心是 Java 虚拟机，它用于将 Java 字节码解释成为能够在本地执行的二进制代码，是 Java 程序跨平台的关键所在。由于 Java 语言是一种通过翻译执行的语言，跟编译后直接可以本地执行的 C、C++等语言相比，它在具有上述的大量优点的同时也带来了程序运行效率低的缺点，这主要是由于字节码的翻译执行速度较慢、自动内存管理带来的时空开销、动态运行方式（虚方法、接口和类载入）的运行时开销及大量同步开销等，性能问题成为制约 Java 发展的一个重要因素。Java 运行环境是一个十分庞大的系统，其中最为关键的部分是 Java 虚拟机，它是执行 Java 程序的核心部件，Java 虚拟机性能的高低也就跟程序的性能紧密地联系在一起。

目前，虽然业界的 Java 虚拟机种类繁多［10, 11, 12, 13, 14, 15, 16, 17, 18, 19］，但是却没有一款性能较高、开销较小的虚拟机。纯软件实现的 Java 虚拟机由于受限于程序的剖析等功能，在提高性能上往往得不偿失、顾此失彼，不能达到令人满意的效果；软硬件协同设计的虚拟机有些开销过大，另外还有些做成了专用的处理器，没有通用的特性。

综上所述，目前性能普遍较低的 Java 虚拟机制约着 Java 的发展，在 Java 广泛需求的形势下，为了提高 Java 程序的性能，在现有平台的基础上，设计和实现一个简洁高效的 Java 虚拟机具有十分重要的意义。

1.4.2 Java 虚拟机技术对国产处理器的重要意义

随着综合国力的提高，我国的处理器设计水平也开始跻身于世界前列。近年来涌现出了以龙芯 [3,4,5] 为代表的国产高性能处理器设计。龙芯处理器采用简洁高效的 MIPS 兼容指令系统［20, 21］。选择 MIPS 兼容的指令集主要有软件兼容性较好、处理器设计高效等等诸多优点。然而，目前桌面和服务器等领域的主流指令集架构采用 X86 及其衍生架构。商用高性能处理器市场长期被 Intel、AMD 等厂商垄断。导致大量优秀的商用软件无法在龙芯处理器上运行。随着龙芯和其它国产高性能处理器市场化步伐的加快，这些软件越来越明显的成为限制其推广的瓶颈。因此，构建丰富的软件库是龙芯能够成功推广的关键因素之一。

Java 程序天生具有跨平台的特点，无论是在何种操作系统，何种体系结构上开发的 Java 软件，都可以无差别地运行在标准的 Java 虚拟机的平台上。由于 Java 程序这种跨平台的特性，只要龙芯上有一个稳定、高性能、标准的 Java 平台，将使得现有的大量 Java 应用程序可以直接在基于龙芯的产品上运行，从而让龙芯的软件库成倍地增长，极大地促进龙芯处理器的推广进程。

1.4.3 高效 Java 虚拟机的设计方向

鉴于 Java 技术对应用程序跨平台和国产处理器的重要意义，设计一款高效简洁的 Java 虚拟机十分必要。Java 虚拟机概括来讲，由类装载子系统、运行时数据区和执行引擎三部分组成。类装载子系统根据给定的全限定名来装入类型（类或接口）；执行引擎负责执行那些包含在被装载类的方法中的指令；运行时数据区存放 Java 虚拟机运行时需要的数据内容。这三部分的性能共同决定了 Java 虚拟机的效率，本文从这三方面出发，分别设计高性能的类装载子系统、运行时数据区和执行引擎，以达到提高 Java 虚拟机整体性能的目的。

1.5 本文的主要贡献

本文系统而深入地研究了 Java 虚拟机技术，其主要贡献和创新如下：

第一，以 Openjdk6 Java 编译环境为基础，实现了基于龙芯国产高性能处理器的 Java 虚拟机系统。Openjdk6 是一套完整的 Java 编译环境，它包括命令工具代码、本地方法库代码、虚拟机代码等，其中的 HotSpot 是最核心的 Java 虚拟机代码。本文以 Java HotSpot 虚拟机为基础，修改 HotSpot 虚拟机中的类装载子系统以及运行时环境等，使这些环境能够符合龙芯处理器的结构设计，从而能够在龙芯硬件平台上稳定运行。在此基础上，根据龙芯处理器的硬件结构，对 HotSpot 虚拟机做了初步优化。使用 SPECjvm2008 测试套件测试，在龙芯处理器硬件平台上，以 Openjdk6 为基础设计的 Java 虚拟机可以稳定运行并且性能达到了较高水平。该高性能的 Java 虚拟机系统是本文 Java 虚拟机研究和性能评估的基础平台。

第二，通过软硬件协同的方式，设计并实现了一种混合的虚方法调用机制。Java 作为一种面向对象的语言，为了方便实现多态的特性，有大量的虚方法调用。由于虚方法只有在运行时才能得知该方法的对象是什么，所以对一个虚方法的解析就比较复杂，只能用运行时动态解析的方式来确定目标方法。这种解析机制只能在运行到调用点时才能获得调用的类型信息，从而用该类型信息检索特定类的方法并且调用该方法。这种动态解析一般被编译成间接跳转，而这种间接跳转很容易引起流水线的停顿，限制了 Java 虚拟机的性能。本文设计并实现了硬件 CAM 查找表，提出了一种软硬件协同设计的混合的虚方法调用机制。实验表明，采用本方法，virtual_test 测试程序虚方法调用点的类型比较命中率从 13.3%升高到了 76.4%，性能有了 16.2%的提升；SPECjvm98 性能平均提升 6.4% 。

第三，通过硬件设计，优化了 Java 虚拟机中的数组越界检查和上下文切换两部分访存机制。Java 程序中数组的长度是动态决定的，数组的分配之初，不能够确定数组的真正长度，为了保证 Java 程序运行的正确性，必须在运行时检查数组访问是否越界，这种附加运行时开销很大；另外，Java 虚拟机在内存中移动生成代码和 Java 栈保存恢复时需要用到大量的访存操作，而传统的按字访问存储器效率较低，不能满足 Java 虚拟机上下

文切换的需求。本文设计并实现了有界访存和 128 位访存指令，提出了一种软硬件结合的访存优化方法，有效地降低了数组越界检查的开销，提高了内存拷贝的效率，优化了 Java 程序的整体性能。使用有界访存指令，科学计算类测试 SciMark2.0 获得了 27%的性能提升，通用测试 SPECjvm2008 获得了 1.5%的性能提升；使用 128 位访存指令，SPECjvm2008 性能平均提升 5.9% 。

第四，设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。通过对 Java 虚拟机即时编译器的编译行为的分析，认为编译生成的大量可执行代码在 cache 中的频繁替换是即时编译器的性能瓶颈，因为当生成代码被替换出 cache 后，处理器取指执行时必须访问速度较慢的主存储器。从这一点出发，本文设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。通过该方法，生成代码执行时的 cache 失效次数降低了 6.9%，测试 SPECjvm2008 中测试程序最高获得了 17.9%的性能提升，平均性能提升 4.2% 。

1.6 论文的组织

本文共分七章，组织如下：

第一章，主要介绍了由于跨平台应用的广泛需求，虚拟机技术成为了支持跨平台的重要技术，鉴于 Java 体系结构的重要特性，Java 技术得到了广泛应用。并且简要介绍了 Java 体系结构的基本技术。在此基础上，该章还介绍了虚拟机技术对以龙芯为代表的国产处理器的应用和推广的重要作用。

第二章，介绍了本文研究所涉及的 Java 虚拟机技术的研究现状。Java 虚拟机主要分为基于软件设计和软硬件协同设计两大类。其中基于软件设计的 Java 虚拟机又根据执行引擎的实现方式进行分类，软硬件协同设计的 Java 虚拟机包括直接仿照 JVM 的结构设计的 Java 专用处理器、以协处理器方式工作的 Java 字节码硬件解释器和以超长指令字 VLIW 技术设计的并行微处理器。本文的工作也从上述研究工作中得到了启发。

第三章，介绍了本文研究工作的软硬件平台。本文以龙芯三号处理器平台作为硬件平台，首先对该平台结构进行了介绍，在此平台上，以 Java HotSpot 虚拟机作为基础实现了相对高效的 Java 虚拟机系统。本文的工作在该软硬件平台上展开。

第四章，提出了一种软硬件协同设计的混合的虚方法调用机制。该章节首先分析了 Java 虚拟机中虚方法调用机制，并对 Java 程序中方法类型进行了统计。通过上述分析，本文设计并实现了硬件 CAM 查找表，提出了一种软硬件协同设计的混合的虚方法调用机制，并进行了性能评估。

第五章，主要介绍了 Java 虚拟机运行时环境的访存分析和优化。首先，从上下文切换和数组越界检查两方面对 Java 虚拟机运行时环境进行了分析，找出了优化的方向。从上述分析出发，实现了有界访存和 128 位访存指令，并描述了软硬件协同设计的访存优化方法。

第六章，介绍了软硬件协同设计的 Java 虚拟机即时编译器设计。分析了 Java 虚拟机即时编译器的工作流程，找出了性能瓶颈，设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。

第七章，对本文进行了总结，并介绍了未来研究方向。

第二章　　Java 虚拟机相关工作介绍

本章对 Java 虚拟机技术的研究背景和相关工作进行介绍及分析。随着 Java 语言越来越普及，Java 虚拟机的研究也成为业界普遍关注的热点问题，而这其中对 Java 虚拟机性能的提升成为了研究的重点。Java 虚拟机中最主要的部分是执行引擎，因为执行引擎直接关系着字节码到本地机器代码的转换方式，执行引擎的性能高低直接影响着 Java 虚拟机的性能。近年来，业界出现了很多的符合 Java 规范的执行引擎的设计，伴随着出现了大量的 Java 虚拟机的实现。而所有的这些设计基本可以分为两大类：基于软件设计的 Java 虚拟机和软硬件协同设计的 Java 虚拟机。

这些 Java 虚拟机技术有它们各自的优缺点，本文的研究重点是利用上述虚拟机技术的优点来优化龙芯 Java 虚拟机系统的设计。本章对相关涉及的技术进行了较为全面细致的介绍。

2.1 基于软件设计的 Java 虚拟机

基于软件设计的 Java 虚拟机是只依靠软件实现符合 Java 规范的 Java 虚拟机，它不依赖于任何硬件的辅助，只依靠软件实现类的装载、本地方法的调用、Java 方法的解释编译等功能。这种虚拟机的优化纯靠软件实现，借鉴了二进制翻译器和二进制优化器中的很多优化方法，取得了很好的效果。

如前文所述，Java 虚拟机中解释引擎主要有解释器和即时编译器两种实现。解释器是按照逐条指令取指、分析、执行相应的操作一直循环下去，按单条指令解释执行，这种方法虽然解释效率较低，但是启动开销最小；而即时编译器则试图每次以方法为单位将需要执行的代码转换为本地代码并缓存，下次执行到相同代码的时候就不需要再次编译，增加了代码的重用性。为了进一步提高编译后代码的质量，还可以把保存下来的二进制代码做进一步优化[22,23,24,25,26,27,28,29,30,31,32]。这种方法可以利用代码的局部性，达到一次编译多次执行的效果，并且配合对编译后代码的优化，达到较高的性能，缺点就启动开销较大。

对于初次执行的代码，解释执行具有较好的初始执行速度（也称热身速度），然而对于编译执行来讲，一旦重复执行已经编译过的代码，因为不需要重新编译代码，这时编译执行就会有较大的优势。在已经编译代码的基础上，Java 虚拟机系统还可以做更加复杂的二进制优化提高编译后代码的执行速度。

为了达到较好冷代码和热点代码执行速度的平衡，Java 虚拟机系统往往综合结合上述技术。根据执行引擎实现的不同，目前的 Java 虚拟机系统主要分为两大类：基于编译器设计的 Java 虚拟机系统和解释器与编译器混合设计的 Java 虚拟机系统。下面介绍相关的系统和设计。

2.1.1 基于编译器设计的 Java 虚拟机系统

这类 Java 虚拟机的执行引擎只有编译器，没有解释器。但是编译器的实现分成了好多层次，在这种实现中比较重要的机制就是对程序进行剖析，根据程序行为的不同和函数的执行频率，对程序的不同部分进行不同程度的编译优化。编译器的不同层次在优化开销和性能提升中进行权衡，根据剖析的结果，对于执行频度比较高的方法，也就是热点方法，可以针对方法本身的特点甚至是体系结构的相关特点进行特殊的优化，虽然优化的开销比较大，但由于方法的执行频度比较高，性能的提升也就比较明显；对于执行频度比较低的普通方法，采取普通的编译方式，编译的开销较小。这种针对程序行为的多层次的编译方式对 Java 程序的性能提升有显著的效果。代表性的结构有以下几种：

2.1.1.1 Jalapeno

Jalapeno[10,33,34,35,36]是 IBM 公司开发的 Java 虚拟机，最初由 IBM T.J.Watson 研究中心开发，并且一直发展至今，是最早的仅使用编译器系统动态编译的 Java 虚拟机。它最显著的特点是对程序的编译只使用编译器。与其他虚拟机解释器和编译器混合的方式不同，Jalapeno 是执行前编译代码的方式。对于这种编译方式，Jalapeno 采用了三层编译器：用于密集型计算的优化编译器、使用低级别优化的快速编译器[37]和基本的编译器。相比于同时使用编译器和解释器的 Java 虚拟机系统，Jalapeno 只使用编译器能够使程序在优化和未优化代码间切换更加灵活。早期的高性能 Java 虚拟机为了获得性能上的优势，使用静态优化的方法，并且不允许类的动态载入，而 Jalapeno 克服了这一缺点，它严格遵守所有的 Java 规范，并且它其中的优化编译器是一个动态的编译器。

Jalapeno 由以下几部分组成：动态类装载器、动态链接器、垃圾收集器、线程调度器、对象分配器以及三个不同级别的编译器，并且支持一些动态的特征，比如动态异常管理、类型检查等等。在优化的编译器启动之前，基本的编译器工作，它起到对其他编译器校验的作用；类装载器支持动态链接的功能；在 Jalapeno 的内存管理系统中，有垃圾收集器，支持类型精确的垃圾收集。

Jalapeno 另外一个显著的特征是整个虚拟机都是用 Java 语言实现的。虽然是用 Java 实现的，但是它是自引导的，不需要其它的 Java 虚拟机来支持。使用 Java 语言实现的一个优点是它可以方便地实现代码的自修改和优化。

Jalapeno 的优化编译器是动态自适应的，当代码运行时它会根据代码的运行情况自行启动或者关闭，优化编译器的目标就是在有限的开销下实现最优化的编译代码；另外，在异常处理、垃圾收集以及多线程的各种优化下，代码达到了更好的性能；最后，Jalapeno 由于是 Java 实现的，因此它可以方便地移植于各个平台上。

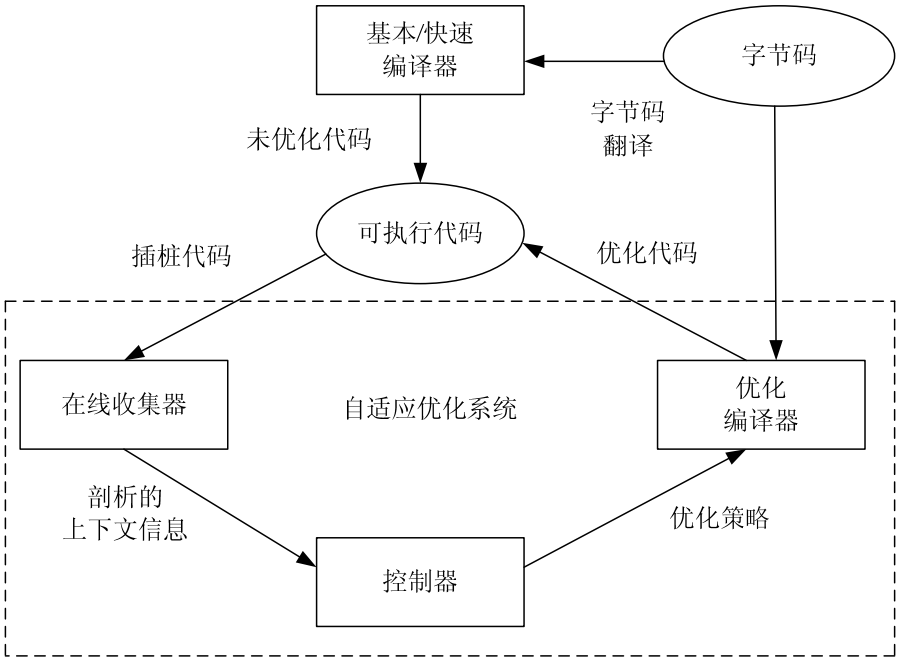

图 2.1 Jalapeno 动态编译器工作流程

Jalapeno 动态编译器工作流程如图 2.1 所示，动态编译器是 Jalapeno 自适应优化系统的主要部分，另外还包含一个在线信息收集器和一个全局控制器。字节码装载之后，首先调用基本的编译器和快速编译器，编译出基本的可执行代码，该部分代码是未优化的。编译器在可执行代码执行过程中，向代码中动态插入插桩代码，这部分代码结合硬件的相关计数器，用于软件采样以及动态收集剖析信息，全局控制器利用采样信息以及剖析获得的程序运行时方法信息，选取合适的优化策略，启动优化的编译器对代码进行优化，产生优化的二进制代码，提高程序的性能。

另外，这个优化的编译器还可以作为静态编译器来使用，此时将编译好的优化代码存储在某个文件中，再次执行该代码时从文件中取得。

Jalapeno 优化的编译器是整套动态优化系统的核心，在编译一个方法时，各个组成部分的微结构如图 2.2 所示，从顶层来看，该结构分为前端优化器和后端优化器两部分。

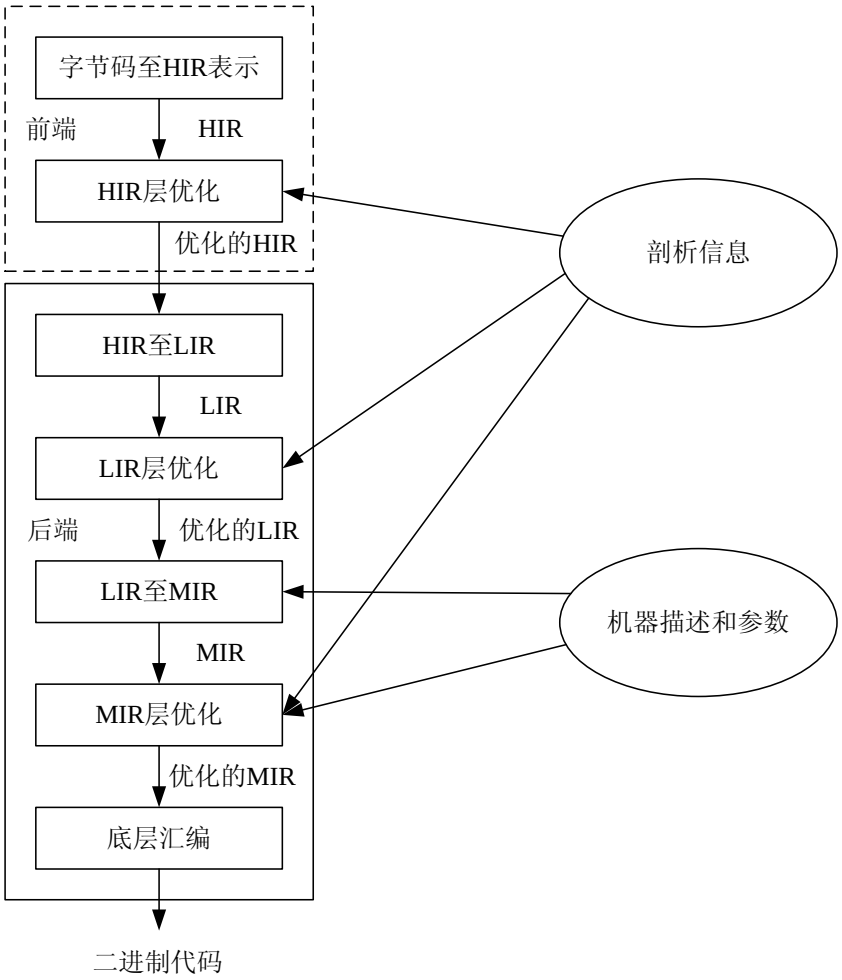

图 2.2　Jalapeno 优化的编译器编译过程

Jalapeno 在将字节码编译成底层汇编的过程中，需要经过三成中间表示，分别是：高层表示（HIR）、中间层表示（LIR）和带有机器信息的中间表示（MIR）。前端优化器主要完成两部分工作：第一，字节码首先通过 BC2IR 算法编译成 HIR 高层表示，并且在编译的过程中进行简单优化；第二，在 BC2IR 算法之后对产生的 HIR 表示作优化。前端优化完成后，进入后端优化，后端优化器首先将 HIR 表示转换成带有 Java 虚拟机信息的中间表示 LIR，在 LIR 表示上做优化，然后由 LIR 转换成带有机器信息的 MIR，根据机器信息优化 MIR，最终转换成可执行的机器代码。在整个过程中根据剖析所得的运行时信息确定优化策略。除了执行引擎的优化过程之外，Jalapeno 还引入了寄存器分配、垃圾收集、冗余代码消除等优化机制。

在使用所有的优化技术之后，使用 Java 语言实现的 Jalapeno 达到了用 C 语言实现的 JDK1.1.6 编译器的性能水平。

2.1.1.2 JUDO

JUDO[11]是在 MRL VM Java 虚拟机基础上增加静态和动态优化的优化编译器。

MRL VM 内部实现了一个动态编译系统，它的结构跟 Jalapeno 很相似，由三个主要部分组成，分别是：基本的编译器、优化的编译器和信息剖析器，如图 2.3 所示。使用分阶段编译[23,28]的思想来提高虚拟机的效率。

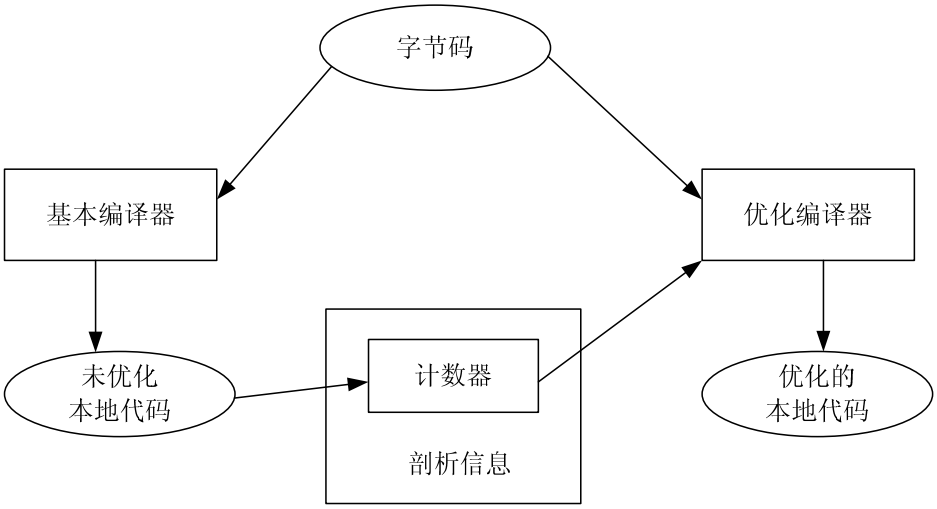

图 2.3 MRL VM 编译系统结构

首先，所有的字节码都由基本编译器编译成未优化的本地代码，在这些本地代码中插入剖析代码统计程序运行时的信息，代码执行时，剖析代码相应地更新代码的统计信息。然后，根据剖析信息，有些方法被标记为热点方法，转入优化的编译器，重新编译这些热点方法，在编译的过程中加入了很多优化，从而得到优化的本地代码。编译系统记录下这些方法，当下次在字节码中碰到该代码时，直接使用优化的编译器编译，而不用基本的编译器。

基本的编译器扫描两遍字节码，快速地产生高效的本地代码。第一遍扫描出字节码的块边界信息；第二遍产生高效的代码并进行简单的优化（例如越界检查、公共表达式消除等等）。另外，基本的编译器负责插入剖析代码。剖析代码在两类位置插入：方法的入口点和方法的回路。前者判断方法是否被密集调用，后者判断方法是不是处于核心循环中。利用这两部分剖析信息，编译系统就可以判断热点方法并且进行重新优化编译。MRL VM 优化的编译器中也采用了大量的优化技术，与 Jalapeno 基本类似。

在 MRL VM 编译器框架的基础上，JUDO 进行了优化，这些优化分为静态优化和动态优化两类。

静态优化主要有类初始化、边界检查以及方法内联的优化。Java 规范规定，在类使用之前需要确定这个类是否已经激活，因此在某个类使用前需要进行初始化检查，而在 MRL VM 编译字节码 getstatic 和 putstatic 的过程中加入了检查的方法，从而避免了运行时的检查；Java 语言规定运行时在数组访问之前需要检查数组访问是否越界，如果访问数组的索引值超过了数组长度，那么引发异常，在数组访问密集型程序里，如果每次访

问之前都需要进行越界检查，那么开销将是非常大的，如果能够在越界检查之前证明在程序运行过程中某个数组访问不会发生越界，那么就可以将此处的越界检查消除，正是借用了这种思想，在原来实现的基础上使用复制循环的方法，消除了冗余的数组越界检查；在 Java 虚拟机中，虚方法调用的处理比较复杂，而 JUDO 使用内联的方法优化虚方法的调用。

动态优化的方法在运行时使用，JUDO 使用的动态优化方法有：动态内联补丁、减小垃圾收集映射的大小、缓存栈帧以及避免创建可能发生异常的对象。在虚方法调用时，Java 程序往往具有局部性的特点，下次调用的虚方法与上次调用的目标方法可能相同。利用这个特点，JUDO 解析成功之后，将调用方法的入口缓存起来，下次调用时直接执行，这样就节省了方法解析的时间；垃圾收集是在编译时得到垃圾收集的映射信息，这些信息比较多，但又不是每个信息都需要使用，因此，在基本编译器中，只保留剖析信息指针和垃圾收集内存映射指针，当垃圾收集发生时，如果需要使用某个方法的其他信息，那么在运行时动态编译，重新生成这些信息；对于栈遍历的优化使用缓存相同栈帧的方式，以消除重复性的工作；在 Java 中有一类对象类型专门使用异常改变程序流，但是这些异常对象并经常使用，可以将不经常使用的类对象消除。

2.1.1.3 Self-93

Self-93[39,40]是从它的第一代产品 Self-91[41]发展而来的。它引入了 online feedback 的技术，也就是在方法执行的内部进行实时的统计，如果发现该方法的执行次数达到了某一阈值，那么该方法就确定为热点方法，立即就对该方法进行优化的编译执行，而不是在下次进入该方法时再优化。

Self-93 最显著的特点是同时在程序的编译时间和运行时间上取得了优化。传统的 Smalltalk-80 环境采用了解释器和普通编译的技术，虽然编译时间较短，但是程序的性能比 C++语言的程序慢了很多；而其他采用了优化编译器的系统，虽然程序性能上有所提高，但是编译带来的停顿时间却很长。而 Self-93 自适应优化系统在编译时间和程序性能之间取得了平衡，两者都达到了相对较好的效果。

当遇到一个新的方法时，方法第一次编译是通过基本的编译器来进行的，通过基本的编译器，将新方法编译成未优化的本地代码，当遇到另外一个新方法时，编译好的旧方法被摒弃，实现非常灵活。传统的代码编译执行流程是“编辑——编译——链接——执行”，而现在只需要“编辑——继续编译”即可，从而程序某个方法改变时不需要重新编译代码，提高了程序的灵活性，节省了编译时间。

同其他的自适应优化编译系统一样，Self-93 也有优化的编译系统，使用自适应的方式处理基本编译和优化编译的切换，当通过运行时的剖析发现某方法是热点方法时，转入优化的编译，另外，基本的编译器对优化的编译器还有验证的功能。方法编译流程如图 2.4 所示。

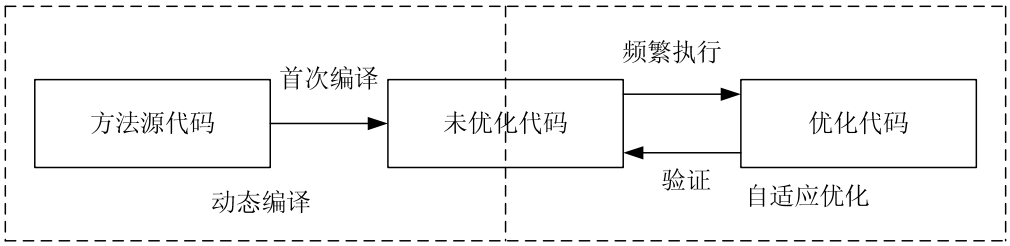

图 2.4 Self-93 编译过程

经过评测，在编译时间基本相同的情况下，Self-93 编译的程序性能是 Smalltall-80[42]环境的 3.3 倍。

2.1.2 解释器编译器混合设计的 Java 虚拟机系统

这类 Java 虚拟机中既有解释器也有编译器，因此是一个比较复杂的编译系统。因为这类 Java 虚拟机中的执行引擎既包含解释器也包含编译器，所以在这类系统中的优化机制也就更多，可以采取更加丰富的手段来提高 Java 程序的性能。Java 字节码在执行的时候首先解释执行，当方法的执行频度达到了一定阈值，则转为编译执行，编译执行的过程中同时进行剖析，成为热点方法后，对热点方法再进行优化编译。代表性的结构有以下几种：

2.1.2.1 HotSpot

JDK 系列 Java 虚拟机是 Sun 公司开发的 Java 编译环境，2008 年已经发展到 JDK6 版本，HotSpot[16,43]是 JDK[44]中的 Java 虚拟机核心，JDK6 中的 HotSpot 使用了近年来学术界和工业界的先进技术。

HotSpot 内部结构如图 2.5 所示。当编译一个方法时，首先执行的是解释器，解释器对于方法中的字节码单条解释并执行，当某个方法执行频繁，标记为热点方法时，调用即时编译器编译执行。由于在解释的时候大部分类已经动态加载，因此在即时编译的时候可以得到非常多的内联方法信息并且产生高质量的代码。

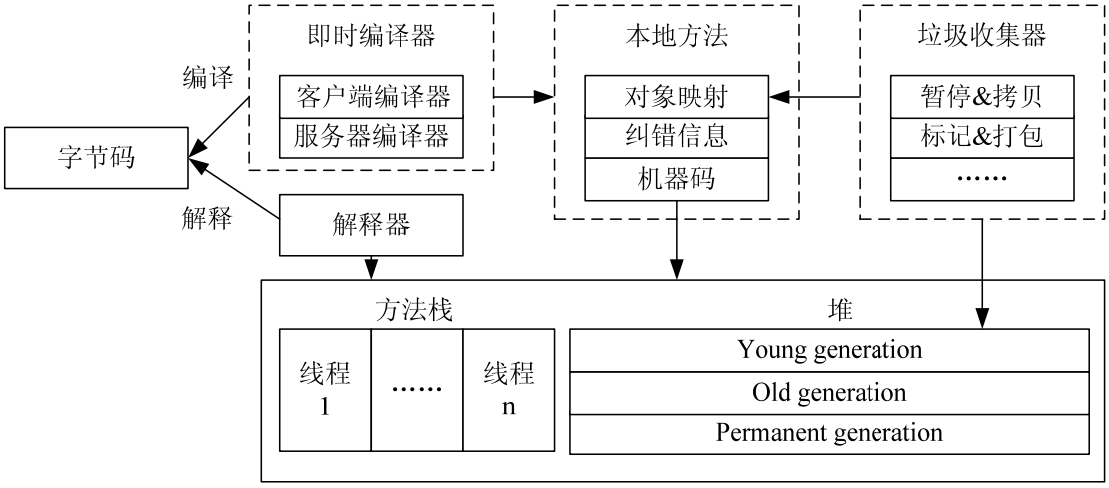

图 2.5 Java HotSpot 虚拟机结构

如果一个方法包含长时间运行的循环，那么不管该方法的执行频率如何，都对该方法即时编译。虚拟机对分支的回路进行计数，如果达到了一定阈值，那么暂停解释器，对方法进行编译，这时一个本地方法栈的栈帧建立，并且初始化为对应于解释器的栈帧，对于这个方法的执行也就调用本地方法栈的栈帧。这种在方法执行期间动态转换解释和编译的方法是在 HotSpot 中引入的新技术，称为 OSR（on-stack-replacement）[45,46]。

Java HotSpot 虚拟机有两个即时编译器：服务器端编译器[16]和客户端编译器。服务器端的即时编译器采用了最大力度的优化措施，产生高质量的代码，但是编译时间较长。对于长时间重复运行的服务器程序来说，较长的编译时间是可以容忍的，并且在多核服务器中，编译往往可以在某个核作为后台程序运行。对于具有图形化界面的客户端程序来说，反应时间比程序的峰值性能更重要，基于这个考虑，客户端的编译器在代码优化和编译时间之间做了很好的平衡[47]。

现代的 Java 虚拟机在处理同步时都使用 thin lock 机制[48,49]，Sun 的 JDK6 扩展了这一机制，使用 biased lock 机制[50]。之前的机制只对那些在同一时间多个线程同时访问的对象加锁，在 biased lock 机制中，当某个对象第一次被上锁时，这个对象头保存指向访问它的线程的指针，这时称作这个对象倾向于该线程，因此下次这个线程对该对象上锁或者解锁时，操作就不必是原子性的。

Java 虚拟机使用垃圾收集技术[51]自动回收和分配内存。在 HotSpot 中，使用精确的垃圾收集技术，在垃圾收集时，对象和指向对象的指针必须是精确的，这对于垃圾收集的压缩算法尤其重要。在这种机制下，内存被分为了三部分：一部分是新分配对象的内存，一部分是已经有较长生存期的对象所在的内存，还有一部分是内部数据结构作占用的内存，这部分内存是永远不会被回收的。新的对象分配在第一部分内存区域，每个线程都有各自的线程局部分配缓存（TLAB），因此分配操作不需要锁机制，并且是多线程安全的。当一个新的对象被分配之后，stop-and-copy 垃圾收集同时被初始化。当该对象的生存期达到一定的周期，垃圾收集器把它用 mark-and-compact 算法[52,53]放入老对

象内存区。Java HotSpot 虚拟机除了基本的垃圾收集之外，还提供和很多其他的垃圾收集机制，例如基于服务器的垃圾收集机制，由于多核和大的物理内存，使用多线程的垃圾收集，这就降低了垃圾收集的开销，提高了程序的吞吐率；另外并发的 mark-and-sweep 垃圾收集算法允许死对象在垃圾收集之后用户程序仍然能够继续运行。

精确的垃圾收集需要指向堆对象的精确信息；对于机器代码而言，这些信息存放在即时编译器产生的对象映射中；另外，编译器还产生纠错代码用来产生编译代码状态和解释代码状态之间的映射。这些映射信息使编译器可以进行更激进的优化，因为如果优化的前提失效，那么仍然可以使用映射信息反优化[54]到基本的代码，保证程序代码的正确性。在虚拟机中，机器代码、对象映射和纠错信息都保存在本地方法对象中。垃圾收集和反优化不是在任何程序的运行期间都允许执行的，而是只允许在一些称作 safepoint 的时间点可以执行，像分支回路、方法调用、返回指令以及可能引发异常的操作。除了上述技术外，Java HotSpot 虚拟机还允许对象的打包操作，以减少诸如不同数据类型间大小不同、运行时类的重定义以及完整 debug 带来的空间浪费。

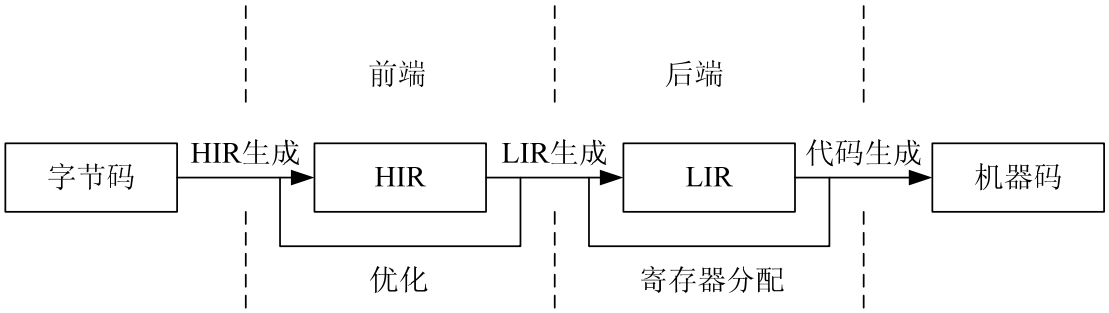

图 2.6 Java HotSpot 虚拟机客户端编译器结构

Java HotSpot 虚拟机客户端编译器是基于使用较少的启动时间和内存占用设计的，结构如图 2.6 所示。对于字节码的编译分为了三个阶段，这样比一遍编译字节码增加了优化空间，所有的字节码信息都保存在了编译过程的中间表示中。首先，字节码被编译成 HIR 高层中间表示，HIR 表示中包含由程序执行基本块链接成的控制流图（CFG），很多的优化都可以在 HIR 这一层进行，比如常量展开、方法内联、空指针检查消除等等。

虚拟机的后端将 HIR 进一步转换成 LIR 底层中间表示，LIR 表示近似于机器代码，但是仍然是平台无关的。相比于 HIR 表示，LIR 表示指向虚拟的寄存器，而不是指向前一条指令，LIR 层可以进一步进行更底层的优化，并且进行线性寄存器分配，将虚拟寄存器映射到真实的物理寄存器。

在寄存器分配完成之后，虚拟机就可以将 LIR 转换为底层机器代码了，并且将这些机器代码放入代码缓存中。这个过程同时产生对象的映射和纠错信息。

由于 JDK6 的 Java HotSpot 虚拟机采用了如上所述的诸多先进技术，并且具有清晰的代码层次和广泛的使用度，因此我们选用 HotSpot 作为我们研究的软件平台。

2.1.2.3 IBM 研究型 JVM 框架

这款研究型的 JVM 框架基于 IBM Developer Kit[17]开发，是一款基于运行时剖析信息的自适应优化 Java 虚拟机系统。该虚拟机执行引擎是一个解释器和三层编译器的混合实现，这个混合系统权衡了快速编译、完全优化和专项优化之间的开销，使程序的编译时间和性能达到了一个很好的平衡。在这套系统中，各个编译层次之间的切换使用动态剖析技术，在程序运行的整个生命周期，一个轻量级的采样剖析器统计程序的基本信息，用于基本的优化，当需要时，系统采用动态插桩的方式，插入详细的剖析代码，统计程序的全局变量值等信息，依靠这些信息，编译器进行完全或者专项的优化。

IBM JVM (Java Virtual Machine) 动态编译系统的结构如图 2.8 所示。该动态编译系统主要由一个混合的解释器和一个三层的编译器组成。混合模式的解释器由解释器和编译器组成，最初所有的方法都由解释器解释，每个方法都有一个计数器统计调用的频率和循环的次数，当计数器达到一定阈值时，启动即时编译器，因此即时编译器有两个入口点，分别是方法的调用点和分支的回路。另外，解释器在碰到条件分支的时候，记录下条件分支成功与否，给即时编译器动态编译提供信息。

动态编译器有三个优化层次：第一层是快速优化，在这个层次中仅仅使用有限的优化策略，包括复写传播、数组越界检查消除、空指针检查消除、公共表达式消除和死代码消除，没有引入那些能够导致编译时间增长或者代码尺寸增大的优化；第二个层次是完全优化，在这个层次中使用全部的优化策略，例如方法内联、代码生成和调度的扫描、基于 DAG 图的循环优化等等；第三个层次是专项优化，在这个层次中，根据程序运行时的反馈信息，进行更深入的特别优化。采用这三个优化层次之后，Java 虚拟机的运行更加适合程序的性能需求，在编译时间和程序性能间达到了平衡。

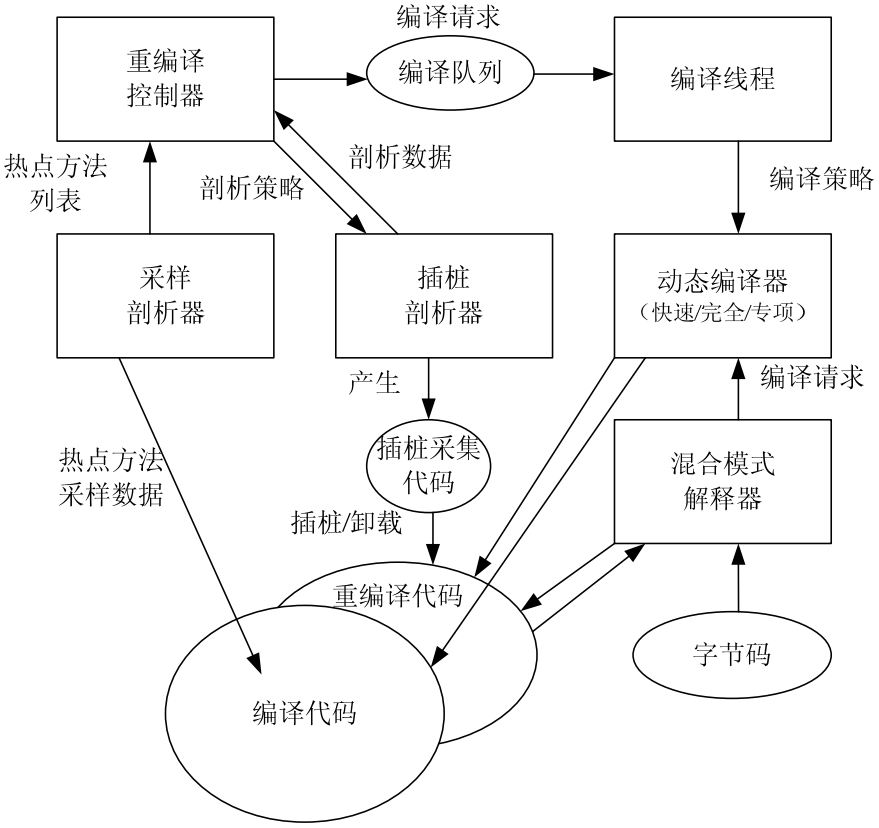

图 2.7 IBM JVM 动态编译系统结构

动态编译系统有两套主要的运行时剖析器。动态编译系统运行期间，有一个一直跟随程序运行的采样剖析器实时采样程序的运行时信息，它主要用来统计方法的 CPU 占用时间，从编译代码中找到热点方法，建立一个热点方法列表，为重新编译和下一步优化提供信息；另外，在需要重新编译的代码中，可以插入插桩代码，统计更加详细的信息，例如调用者-被调用者关系等。

程序执行时，首先由解释器进行解释，同时计数器开始计数，当达到一定阈值时转入即时编译器进行编译，此时系统的采样剖析器开始工作，统计方法的执行频率和 CPU 时间，找出热点方法，并形成热点方法列表，准备重新对这些热点方法使用更大力度的优化策略进行编译，对这些热点方法，重编译控制器进行选择，一部分方法直接重新编译，另外一部分方法，插入插桩代码进行更细粒度的信息统计，根据这些统计信息，进行方法的完全优化或者是专项优化。

通过多层次的编译结构，动态编译器针对不同的应用都能够获得很好的程序性能值和相对平衡的编译时间。

2.1.3 软件 Java 虚拟机的关键技术

综合上述多款 Java 虚拟机，我们发现基于纯软件实现的 Java 虚拟机的核心是它的

执行引擎，而它们在执行引擎的实现上又包含有很多共同点。

首先，无论是只有编译器的实现，还是解释器和编译器的混合实现，执行引擎的框架设计都是多层结构的。第一层结构是一个最基本的字节码编译系统，对于只有编译器的虚拟机来说，这一层是一个基本的编译器；对于解释器和编译器混合实现的虚拟机来说，这一层是解释器，这一层在于降低程序的启动时间，以最快的速度编译代码，让代码能够在最短的时间内执行，提高程序的响应度；第二层结构是一个较优化的编译系统，对于只有编译器的系统来说，该层是一个有限优化的编译器，对于混合实现的虚拟机来说，该层是简单的编译器，这一层对代码进行了一定的优化，对代码质量有了较高的要求，但是并没有将优化达到最大力度，是在编译时间和执行时间之间的一个权衡，适合于普通用户程序的运行；第三层是最大优化的编译系统，这一层的优化达到了最大力度，并且结合了很多底层硬件结构相关的优化，对于长时间运行的循环来说，一次编译之后的高质量代码能够很大程度上带来程序性能的提高。对于有些虚拟机，可能还有更多层的结构。通过执行引擎多层次的设计，满足了不同应用的需求，形成了一个自适应的优化系统，这也是当前 Java 虚拟机普遍采用的结构，并且未来 Java 虚拟机也会向这一方向发展。

其次，要实现上述的多层次执行引擎设计，在多个层次的编译系统中进行灵活切换，对程序的剖析是必不可少的。最初的剖析信息来自于静态的代码，但是这种静态信息却只能用于进行有限的优化；近年来，剖析信息以插桩代码的方式插入了程序的代码段中，可以在程序运行时进行信息的统计，从而能够最大限度地提供优化所需要的运行时信息。但是在插桩代码的同时，必然会引起程序的下降，这就需要在权衡利弊之后做更加合理的设计。

2.2 软硬件协同设计的 Java 虚拟机系统

软硬件协同设计的 Java 虚拟机是在处理器或者硬件结构上对 Java 字节码的解释或编译做特有的支持，从而达到软件 Java 虚拟机不能达到的性能优势。这种结构也可以成为专有结构，硬件专门为 Java 虚拟机而定制，它的特点是性能很高，但是通用性和可移植性比较差。

这种结构的 Java 虚拟机也被称为 Java 处理器，目前，Java 处理器的研究主要集中在以下三方面：

2.2.1 直接仿照 JVM 的结构设计的 Java 专用处理器

这类设计以 Java 字节码作为处理器的指令集合，或者设计类似 JVM 的堆栈型处理器，将 Java 虚拟机的功能用硬件处理器来实现，尽量减少编译、优化字节码的开销。是专门用于执行 Java 字节码的专用处理器。

PicoJava[57,58]是 Sun 公司开发的基于堆栈结构的 Java 处理器，处理器结构如图 2.9 所示。内部采用 6 级 RISC 流水线，处理器由 6 个基本单元模块组成：整数单元、浮点单元、指令 cache、数据 cache、堆栈管理单元和总线接口单元。PicoJava 支持 Java 虚拟机的运行环境，具有多线程同步、多种垃圾回收方法、方法调用、本地变量隐含调用等特点。

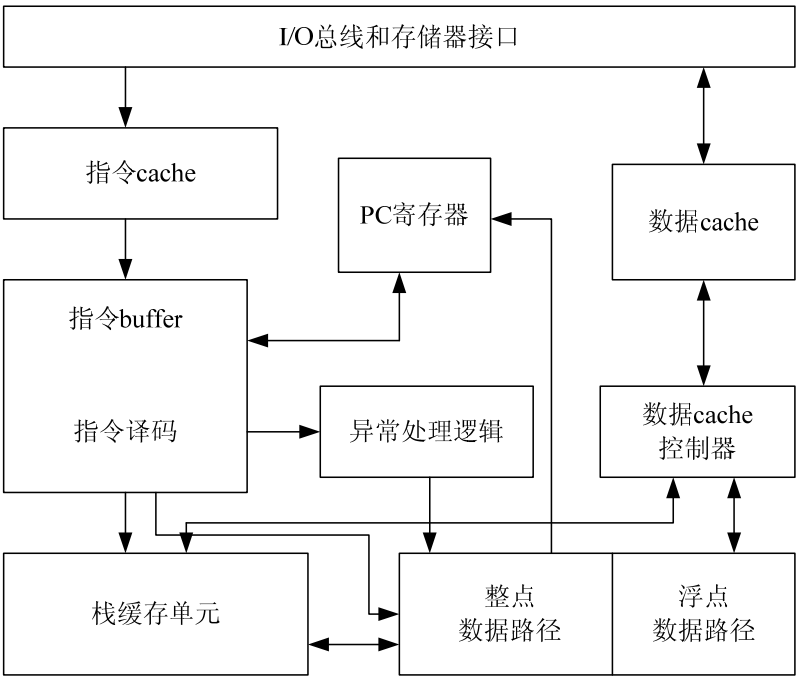

图 2.8 PicoJava 处理器核组成

PicoJava 既可以执行 JVM 中定义的 Java 字节码，也可以执行 C 代码。它可以执行大部分常用的 Java 字节码，对于较复杂的字节码以微代码的方式执行，而最复杂的字节码则引发异常，由软件解释执行。除标准字节码外，PicoJava 还扩展了一系列字节码，用于直接地址读写、cache 管理和访问内部寄存器，但这些字节码只能由操作系统使用。它内部的堆栈管理单元由堆栈 cache、堆栈控制器和 dribble 控制器组成，堆栈控制器既为整数单元提供检索操作数的控制信号，也负责控制数据在数据 cache 和堆栈 cache 之间传送，dribble 控制器负责在堆栈发生上溢或者下溢时控制堆栈和存储器之间传送数据。PicoJava 为支持 Java 而采用了堆栈结构，而堆栈本身就是 Java 程序运行的瓶颈，为此 PicoJava 采用了复杂的流水线技术和堆栈自动管理技术，片上 cache 及其控制逻辑也占用了较大的硅片面积，从而导致成本和功耗增加，虽然它能够执行 Java 字节码，省去了软件解释字节码的开销，却是以处理器的性能下降为代价的。

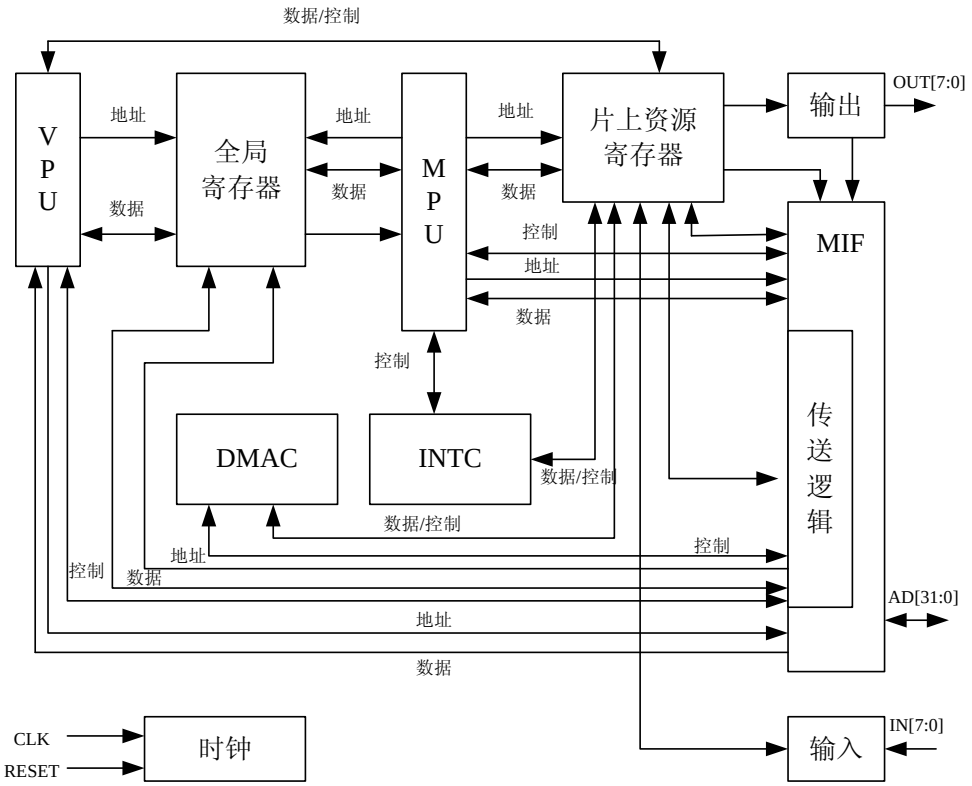

图 2.9　PSC1000 微处理器结构

Petriot Scientific 公司的 PSC1000[59]也采用堆栈结构，它最初的设计是面向其他语言，由于其结构与 JVM 相似，大部分 Java 字节码和 PSC1000 的指令有对应关系，配合 PSC1000 使用的即时程序只需要 20K 字节，而通用处理器的即时编译器需要 200K 字节。PSC1000 主要用于嵌入式环境，设计时注重低成本和低功耗，因此 PSC1000 既没有采用流水线和超标量技术，也没有采用指令和数据 cache。

PSC1000 微处理器的内部结构如图 2.10 所示，共包含 10 个主要部件，分别是 32 位微处理器单元 MPU、虚拟外设单元 VPU、全局寄存器、直接存储器访问控制器 DMAC、中断控制器 INTC、片上资源寄存器、位输入、位输出、可编程存储器接口 MIF 和时钟。

在结构上，指令采用硬联控制，速度较快。flow-through 方式允许在上一条指令执行完之前启动执行下一条指令，因此大多数指令可在单指令周期内完成。复杂指令多周期完成，由于工作时钟频率较高，复杂运算也可以较快完成。移位操作采用智能移位的方式，先按字节移动，再按位移动，减少了移位操作的周期数。采用零操作数的结构，单个指令为一字节，可以同时读出 32 位指令数据，其中最多可以包含 4 条指令，与采用 32 为指令格式的 RISC 处理器相比，相当于提高了存储器的带宽。为了节省面积，充分发挥上述特点，PSC1000 实现了资源共享：MIF 由 MPU、VPU、DMAC 和位输入、位输出共享，全局寄存器由 MPU、VPU 和 MIF 中的传送逻辑共享。PSC1000 以简单的结构实现较短的时钟周期，因此具有较高的性价比，适合低端嵌入式应用。

2.2.2 以协处理器方式工作的 Java 字节码硬件解释器

普通 Java 虚拟机在执行 Java 程序时，需要完成三部分的工作：第一部分是将 Java 程序的字节码载入并分析字节码；第二部分是将字节码解释或编译成本地可执行的二进制代码，这部分工作由解释器或者即时编译器完成；第三部分是处理器执行编译好的可执行代码。以协处理器方式工作的 Java 字节码硬件解释器[60]用硬件编译的方式代替了软件 Java 虚拟机中的第二部分工作，也就是用硬件的方式将原有 Java 虚拟机的编译过程完成，提高了 Java 虚拟机的编译效率。

这种结构将字节码对应的机器码固化在存储器中，执行引擎不需要编译，协处理器只需要读入字节码，从存储器中得到对应的机器码，将机器码送到处理器执行。这种方法在通用处理器加入硬件部件，处理 Java 字节码编译的过程，新增硬件部件的功能跟软件的解释器相似。解释好的代码存储在解释代码缓存中，由通用处理器进行取指令执行。

图 2.11 展示了硬件解释器的内部原理。硬件解释器首先从指令 cache 中取指并对指令进行预译码。字节码对应的可执行代码存储在硬件 ROM 中，因为 ROM 的空间有限，不可能存储 Java 所有的字节码，因此一些复杂的字节码并不存储在 ROM 中，而是还由软件编译，实验表明，频繁使用的字节码只占字节码总数的很少比例。译码之后如果该字节码在 ROM 中存有对应的可执行代码，那么就转入硬件解释；如果发现字节码不在 ROM 中，那么就转入软件编译。硬件解释器中有一个 B-PC 寄存器，用于对字节码地址进行存储，这样就可以按顺序解释字节码。解释好的字节码对应的可执行代码从 ROM 中取至编译代码缓存中，通用处理器的取指部件从编译代码缓存中取指并执行。另外，缓存中的代码还可以利用即时编译器的技术进行优化，从而达到更好的性能。

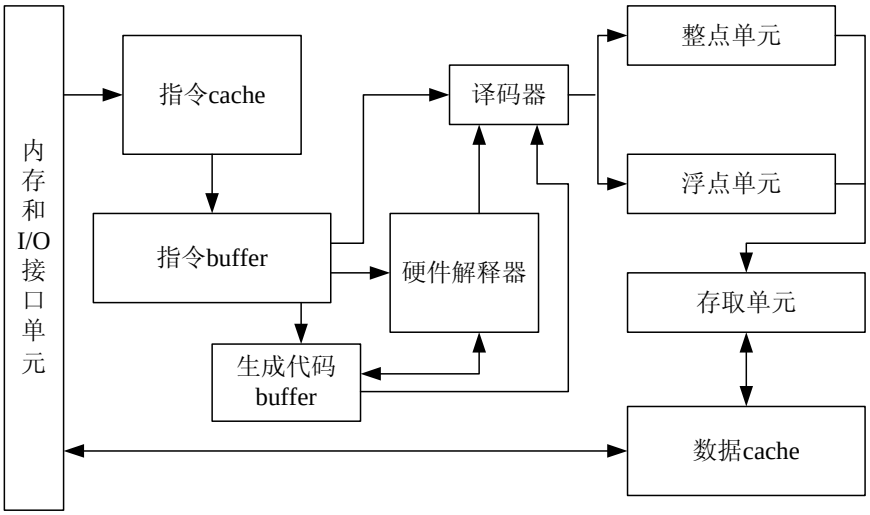

图 2.10 硬件解释器结构图

该结构比较类似于指令路径协处理器。传统的协处理器用于协助主处理器处理浮点运算、图形处理器和内存管理等操作，这些处理器并不进行取指和译码操作。另外还有

一些协处理器，他们在主处理器的指令执行路径上，并且负责从指令 cache 中取指译码并执行，这类协处理器称为指令路径协处理器，这类处理器主要帮助主处理器并行处理复杂的程序。上述结构就属于执行编译操作的指令路径协处理器，它与主处理器并行工作，协处理器负责编译字节码，主处理器只负责执行指令。

硬件解释器的性能很高，经过评测，使用 SPECjvm98 测试套件，性能可以达到普通软件解释器的 2.8-7.7 倍，是普通即时编译器的 2.7-5.0 倍。

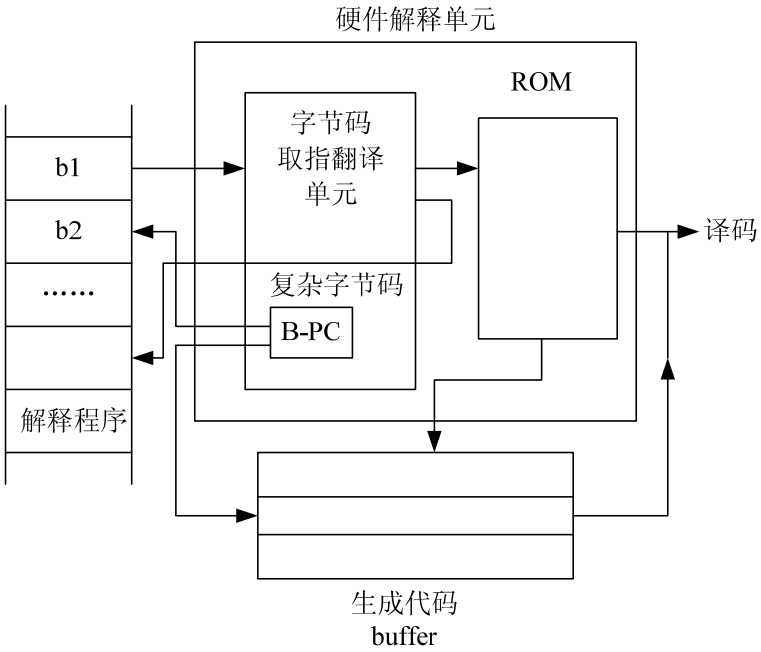

图 2.11　硬件解释单元内部结构

Jedi Technologies 公司的 JSTAR 就是一个硬件解释器，如图 2.12 所示。它可以加入 8 到 64 位的 RISC 或者 CISC 处理器环境中，协处理器把字节码对应的处理器机器码固化在存储器中，并与主处理器及 cache/memory 交互，JSTAR 读取字节码，检索对应的机器指令，送入处理器中执行，对于处理器而言，直接执行机器码。它不需要修改本地处理器核，指令集和处理器通路都不需要改变；具有较强的可扩展性，可以和任何 RISC 和 CISC 处理器配合使用。JSTAR 不需要存储编译好的大量代码，只是在需要时编译并送给主处理器，消除了即时编译器带来的存储器开销。在功耗方面，处于空闲模式时，JSTAR 功耗很低，而工作时功耗也只有少量增加，JSTAR 的这些优势使它很适合嵌入式环境。

2.2.3 以超长指令字 VLIW 技术设计的并行微处理器

VLIW 体系结构是一种让多条指令进入流水线执行的微处理器体系结构，借助编译器将能同时执行的操作安排在一条很长的超长指令字中，是 RISC 结构的一个分支。在

支持 Java 的处理器中采用 VLIW 技术是 1999 年出现的一种新方案，这种结构支持 Java 程序的线程级并行，处理器内部的功能模块采用 RISC 结构，以求实现处理器的高性能。

Sun 公司采用这个方案推出了 MAJC（Microprocessor Architecture for Java Computing）结构的处理器[61]，它是一种针对 Java Computing 的多处理器系统结构。Java Computing 是一种基于客户/服务器模型的计算方式，目的是将计算技术的复杂性从以桌面系统为中心转向以服务器为中心，由服务器进行集中的、专业的管理。Java 语言的平台独立性，适合于以网络为中心的计算方式，因此 Sun 公司称这种计算方式为 Java Computing。

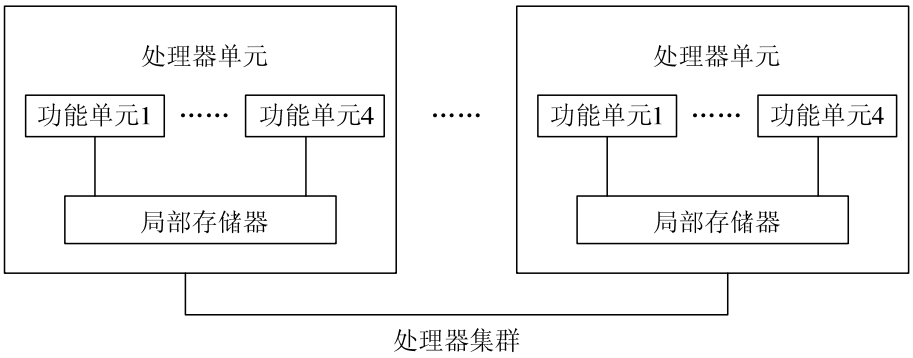

图 2.12 MAJC 处理器集群

MAJC 的结构如图 2.13 所示。MAJC 由处理器群、处理器、功能模块等组成，一个芯片上的处理器构成处理器群，每个处理器包括 1 到 4 个功能模块，每个功能模块相当于一个带有 DSP 功能的 RISC 处理器，每一个功能模块拥有自己的私有寄存器堆、局部指令译码逻辑、状态信息、局部布线等，只能由该功能模块使用。处理器单元中的功能模块可以共享一个全局寄存器堆，寄存器堆的大小根据需要确定，范围从 32 到 512，该全局寄存器堆不能由同一处理器群中的其他处理器单元访问，每条指令都是发送到某一个功能模块上执行的，一个处理器单元单周期内可以处理的指令数取决于当时能够使用的功能模块的数目。因此，MAJC 采用片上多处理器集成技术，使得它能够实现线程级并行，可以应用于实时处理声音、图像等大规模数据的情况。

MAJC 的优点主要体现多级并行上。在数据级，支持 SIMD 工作方式；在指令级，每个处理器有多个功能单元；在线程级，支持多线程的软件执行；在系统级，支持传统的片上多处理器。

除 Sun 公司外，IBM 公司于 2000 年研制支持 Java 的 VLIW 结构的微处理器，与 MAJC 相似，内部采用指令级并行的 RISC 结构，通过快速编译的方法将 Java 字节码转换为该处理器的机器码。

2.3 本章小结

本章主要对涉及本文研究内容的 Java 虚拟机技术进行了介绍。Java 虚拟机的设计总体来说可以分为两大类：基于纯软件设计的 Java 虚拟机和软硬件协同设计的 Java 虚拟机。本章对这两大类 Java 虚拟机的实现技术进行了介绍，并分析了各自的一些代表性结构。在此基础上，总结了 Java 虚拟机实现的一些关键技术，目的是借鉴这些系统和实现的优点，帮助我们实现高效的 Java 虚拟机系统设计。

第三章　　龙芯 Java 虚拟机系统硬件支持与研究平台

3.1 引言

龙芯处理器作为国产高性能处理器的代表，由于指令系统兼容性的原因，大量优秀软件无法在龙芯处理器上运行，随着龙芯和其它国产高性能处理器市场化步伐的加快，软件的匮乏越来越明显地成为限制推广的瓶颈。Java 是解决应用程序跨平台的有效方法。Java 体系结构的核心是 Java 虚拟机，它用于将 Java 字节码解释成为能够在本地执行的二进制代码，是 Java 程序跨平台的关键所在。因此，在龙芯上实现一个稳定、高性能、标准的 Java 平台，是促进龙芯处理器推广的必然要求。本文的研究在龙芯处理器平台上展开，在龙芯处理器平台上设计并开发一个简洁高效的 Java 虚拟机系统。

本章第二部分首先介绍了龙芯 3 号处理器平台。龙芯 3 号处理器是 4 核高性能通用处理器，具有低成本、低功耗、高性能等特点。在龙芯 3 号硬件平台的基础上，我们实现了 Java 虚拟机系统。第三部分介绍了该 Java 虚拟机系统的实现。本文针对 Java 虚拟机系统的研究在上述平台上展开。

3.2 龙芯高性能通用处理器硬件平台

3.2.1 GS464 处理器核内部结构

GS464 是龙芯 2 号和 3 号系列高性能处理器采用的处理器核。如图 3.1 所示，GS464 是一款实现 64 位 MIPS64 指令集的通用 RISC 处理器 IP。GS464 的指令流水线每个时钟周期取四条指令进行译码，并且动态地发射到五个全流水的功能部件中。虽然指令在保证依赖关系的前提下进行乱序执行，但指令的提交还是按照程序原来的顺序以保证精确例外和访存顺序执行。

四发射的超标量结构使得指令流水线中指令和数据相关问题十分突出，GS464 采用乱序执行技术和激进的存储系统设计来提高流水线的效率。

乱序执行技术包括寄存器重命名技术、动态调度技术和转移预测技术。寄存器重命名解决 WAR（读后写）和 WAW（写后写）相关，并用于例外和错误转移预测引起的精确现场恢复，GS464 分别通过 64 项的物理寄存器堆进行定点和浮点寄存器的重命名。动态调度根据指令操作数准备好的次序而不是指令在程序中出现的次序来执行指令，减少了 RAW（写后读）相关引起的阻塞，GS464 有一个 16 项的定点保留站和一个 16 项的浮点保留站用于乱序发射，并通过一个 64 项的 Reorder 队列（简称 ROQ）实现乱序执行的指令按照程序的次序提交。转移预测通过预测转移指令是否成功跳转来减少由于控制相关引起的阻塞，GS464 使用 16 项的转移目标地址缓冲器（Branch Target Buffer，

简称 BTB），2K 项的转移历史表（Branch History Table，简称 BHT），9 位的全局历史寄存器（Global Histiry Registor，简称 GHR），和 4 项的返回地址栈（Return Address Stack，简称 RAS）进行转移预测。

GS464 先进的存储系统设计可以有效地提高流水线的效率。GS464 的一级 Cache 由 64KB 的指令 Cache 和 64KB 的数据 Cache 组成，均采用四路组相联的结构。GS464 的 TLB 有 64 项，采用全相联结构，每项可以映射一个奇页和一个偶页，页大小在 4KB 到 16MB 之间可变。GS464 通过 24 项的访存队列以及 8 项的访存失效队列（Miss Queue）来动态地解决地址依赖，实现访存操作的乱序执行、非阻塞 Cache、取数指令猜测执行（Load Speculation）等访存优化技术。GS464 支持 128 位的访存操作，其虚地址和物理地址均为 48 位。

GS464 有两个定点功能部件和两个浮点功能部件。每个浮点部件都可以全流水地执行 64 位双精度的浮点乘加操作，并通过浮点指令的 fmt 域的扩展执行 32 位和 64 位的定点指令，以及 8 位和 16 位的用于媒体加速的 SIMD 指令。

GS464 支持 MIPS 公司的 EJTAG 调试规范，采用标准的 AXI 接口，其指令 Cache 实现了奇偶校验，数据 Cache 实现了 ECC 校验。

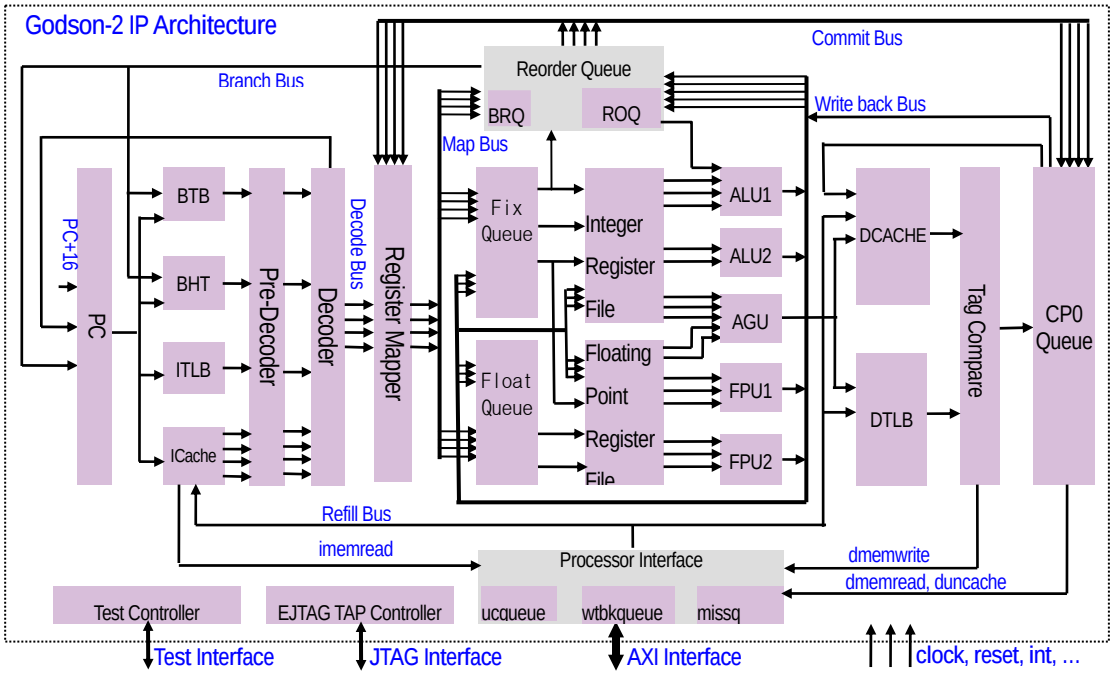

图 3.1　龙芯 GS464 处理器微结构

龙芯 GS464 处理器核兼容 MIPS64 R2 体系结构，提供了其所定义的全套必需指令，包括访存指令、运算指令、转移指令、协处理器指令和其它指令。在此基础上还提供部分媒体与 X86 虚拟机扩展指令。上述特点可以增加龙芯处理器核的适用范围。

3.2.2 四核龙芯 3 号处理器结构

四核龙芯 3 号是龙芯 3 号多核处理器系列的第一款产品，是一个配置为单节点 4 核的处理器，采用 65nm 工艺制造，最高工作主频为 1GHz，主要技术特征如下：

- 片内集成 4 个 64 位的四发射超标量 GS464 高性能处理器核；
- 片内集成 4 MB 的分体共享二级 Cache(由 4 个体模块组成，每个体模块容量为 1MB) ；
- 通过目录协议维护多核及 I/O DMA 访问的 Cache 一致性；
- 片内集成 2 个 64 位 400MHz 的 DDR2/3 控制器；
- 片内集成 2 个 16 位 800MHz 的 HyperTransport 控制器；
- 每个 16 位的 HT 端口拆分成两个 8 路的 HT 端口使用。
- 片内集成 32 位 100MHz PCIX/66MHz PCI；
- 片内集成 1 个 LPC、2 个 UART、1 个 SPI、16 路 GPIO 接口；

四核龙芯 3 号芯片整体架构基于两级互连实现，结构如图 3.2 所示。

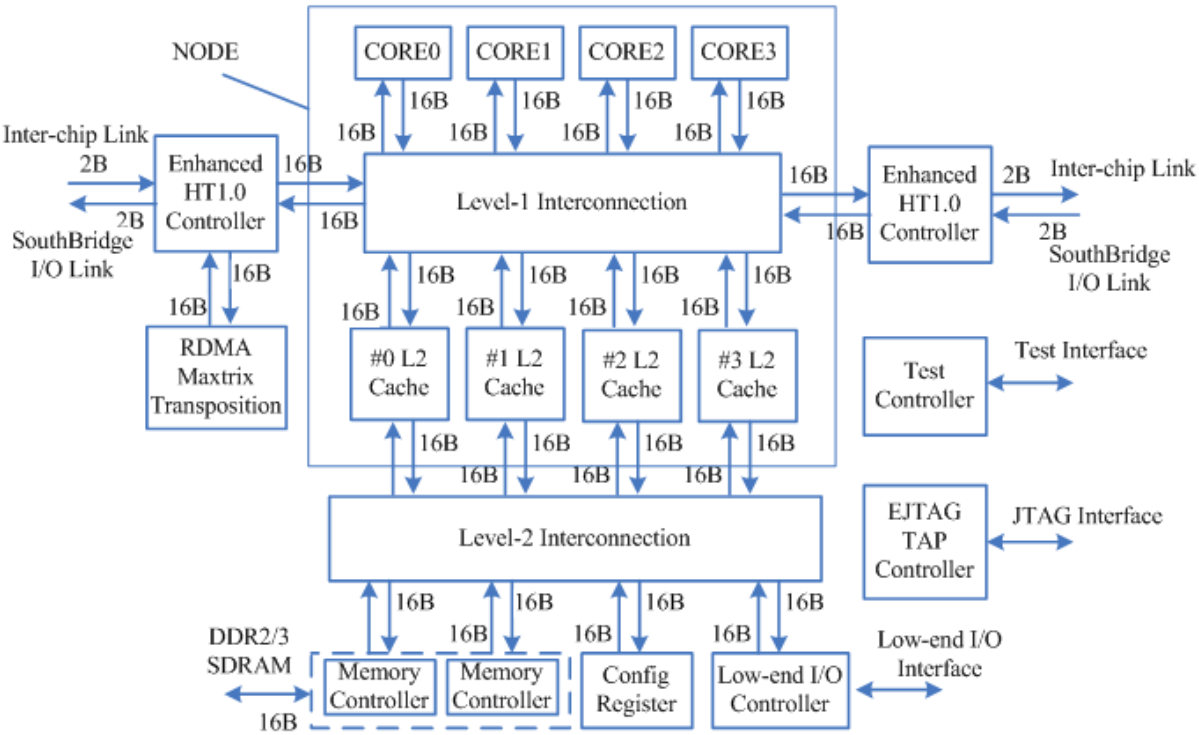

图 3.2　四核龙芯 3 号处理器芯片结构

第一层互连采用 6x6 的交叉开关，用于连接四个 CPU（作为主设备）、四个二级 Cache 模块（作为从设备）、以及两个 IO 端口（每个端口使用一个 Master 和一个 Slave）。一级互连开关连接的每个 IO 端口连接一个 16 位的 HT 控制器，每个 16 位的 HT 端口还可以作为两个 8 位的 HT 端口使用。HT 控制器通过一个 DMA 控制器和一级互联开关相连，

DMA 控制器负责 IO 的 DMA 控制并负责片间一致性的维护。龙芯 3 号的 DMA 控制器还可以通过配置实现预取和矩阵转置。

第二级互连采用 5x4 的交叉开关，连接 4 个 2 级 Cache 模块（作为主设备），两个 DDR2 内存控制器、低速高速 I/O（包括 PCI、LPC、SPI 等）以及芯片内部的控制寄存器模块。

上述两级互连开关都采用读写分离的数据通道，数据通道宽度为 128bit，工作在与处理器核相同的频率，用以提供高速的片上数据传输。

基于龙芯 3 号可扩展互联架构，4 片四核龙芯 3 号处理器可以通过 HT 端口连接构成 4 芯片 16 核的 SMP 结构。

3.3 龙芯 Java 虚拟机系统的软件设计与实现

在上述龙芯处理器硬件平台基础上，本文实现了龙芯 Java 虚拟机软件系统。该系统是本文研究的基础平台。尽管运行和模拟的平台不同，Java 虚拟机系统的工作原理和组成是相同的。为了减小开发工作量，保证实现的平台的完整性和稳定性，本文在 Openjdk6 HotSpot[16,43]虚拟机基础上实现了龙芯平台的 Java 虚拟机系统。该系统以 Openjdk6 HotSpot 实现的系统架构为基础，对内部运行时环境进行了改动，以满足龙芯处理器硬件平台的高性能要求。本小节对 Openjdk6 HotSpot 的发展历史和龙芯 Java 虚拟机系统的设计和实现进行了介绍。

3.3.1 Openjdk Java 虚拟机系统的发展

Openjdk 的前身是 Sun 公司的 JDK 虚拟机，最初 Java 标准是由 Sun 公司制定的，他们也开发出了严格遵循 Java 标准的 Java 虚拟机 JDK，在当时 JDK 源代码不是开放的，只是由 Sun 公司内部的开发者进行维护。当时比较流行的版本是 JDK6 和 JDK7，他们在代码结构和风格上有较大的不同。随着开源成为软件发展的一个必然趋势，在广大开发者和爱好者的呼吁下， 2007 年 5 月 9 日, Sun 公司在 GPL V2 协议下，开放了 JDK7 的大部分代码，成立了 Openjdk 项目，该项目由 Openjdk 社区来维护，Openjdk 社区从理论上来说是独立于 Sun 公司的。这样就形成了 Openjdk 的第一个版本——Openjdk7[44]。

虽然 Sun 公司开放了大部分的源代码，但是还有一部分代码并没有开放，Sun 只是以二进制插件的形式对开发者提供，因此 Openjdk 还有一部分代码是闭源的。这样，在 Openjdk 的基础上，逐渐成立了另外一个项目——IcedTea，IcedTea 项目开放了全部的二进制插件，是全部开源的 Java 虚拟机版本。

Sun 公司在成立 Openjdk 项目之后，原来的 JDK6 版本还继续维护，这样 Openjdk7 和 JDK6 并行发展。Openjdk7 由 JDK7 代码开源而来，继承了 JDK7 的代码结构和风格，由 Openjdk 社区维护；JDK6 由 Sun 公司内部维护。Sun 公司 JDK6 的很多技术来自 Openjdk

社区，由于社区的 Openjdk7 与 JDK6 代码结构有差异，因此技术的迁移产生了很多障碍；JDK6 属于非常流行的 Java 虚拟机版本，在重重因素的驱使下，Openjdk 社区在 Openjdk7 发展到第 22 个发行版的时候，从该发行版建立了另外一个分支，也就是我们现在普遍使用的 Openjdk6。

Openjdk6 使用了与 Sun JDK6 统一的代码结构，因此 Sun 公司的内部维护与社区的技术也就实现了很方便的融合。这样，随着时间的推移，Openjdk6、Openjdk7 和 JDK6 三个版本各自独立地发展着，也各自逐步完善起来，Openjdk6 与 Openjdk7 开源，由 Openjdk 社区开发者维护，JDK6 由 Sun 公司内部维护。其中 Openjdk6 继承了 Openjdk7 强大的功能、较高的效率以及 JDK6 优秀的代码风格，渐渐成为开源 Java 虚拟机的主流版本。由于它的高效、稳定、开源易开发等优点，我们选择 Openjdk6 作为我们的 Java 虚拟机研究平台。

Openjdk 发展的线路图如图 3.3 所示：

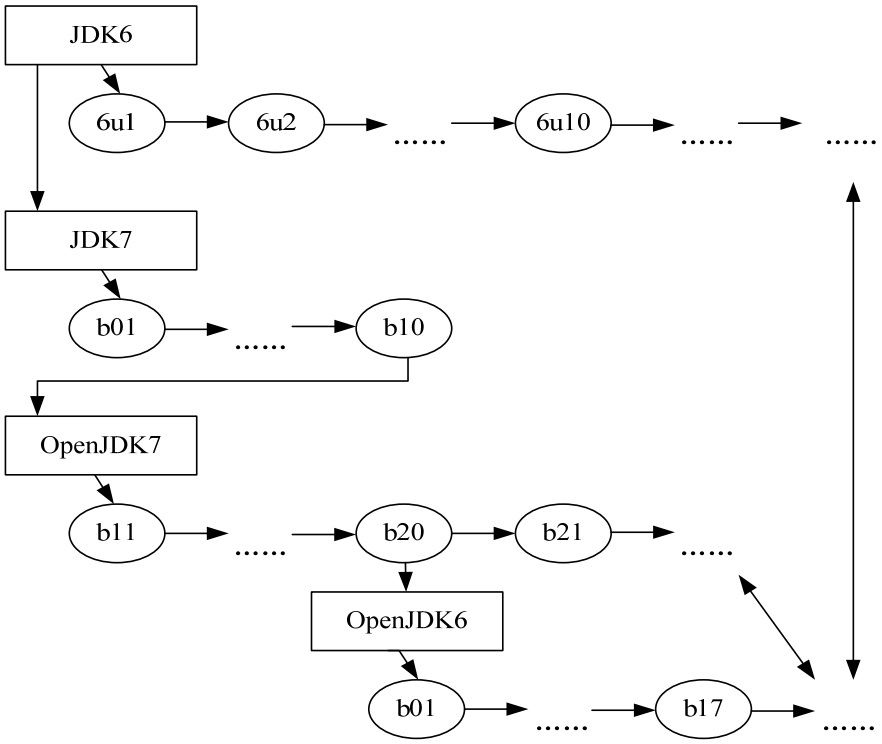

图 3.3　Openjdk 发展线路图

3.3.2 龙芯 Java 虚拟机系统

由于 Openjdk6 在业界广泛使用，并且具有稳定高效的特点。本文在龙芯处理器上使用 Openjdk6 作为基础进行 Java 虚拟机设计。Openjdk6 是一套完整的 Java 编译环境，它包括命令工具代码、本地方法库代码、虚拟机代码等，其中的 HotSpot 就是最核心的 Java 虚拟机代码。Java 虚拟机系统按结构划分，由类装载子系统、运行时数据区和执行引擎三部分组成，如图 3.4 所示。

类装载子系统根据给定的全限定名来装入类型（类或接口），执行引擎负责执行包含在被装载类的方法中的指令。当 Java 虚拟机运行一个程序时，它需要内存来存储许多东西，例如字节码、从已装载的 class 文件中得到的其他信息、程序创建的对象、传递给方法的参数、返回值、局部变量、以及运算的中间结果等等。Java 虚拟机把这些内容都组织到运行时数据区中，以便于管理。

运行时数据区包含方法区、堆、Java 栈、PC 寄存器、本地方法栈。某些程序运行时数据区是由程序中所有线程共享的，还有一些则只能由一个线程拥有。每个 Java 虚拟机实例都有一个方法区以及一个堆，它们是由该虚拟机实例中所有线程共享的。当虚拟机装载一个 class 文件时，它会从这个 class 文件包含的二进制数据中解析类型信息。然后这些信息被放入方法区中。当程序运行时，虚拟机会把所有该程序在运行时创建的对象都放到堆中。当每一个新线程被创建时，它都将得到它自己的 PC 寄存器（程序计数器）以及一个 Java 栈。如果线程正在执行的是一个 Java 方法（非本地方法），那么 PC 寄存器的值将指示下一条将被执行的指令，而它的 Java 栈则总是存储该线程中 Java 方法调用的状态——包括它的局部变量、被调用时传进来的参数、返回值、以及运算的中间结果等等。而本地方法调用的状态，则依赖于具体实现的方式，存储在本地方法栈中，也可能是在寄存器或者内存中。

Java 栈是由许多栈帧组成的，一个栈帧包含一个 Java 调用的状态。当线程调用一个 Java 方法时，虚拟机压入一个新的栈帧到该线程的 Java 栈中；当该方法返回时，这个栈帧从 Java 栈中弹出被抛弃。Java 虚拟机没有寄存器，其指令集使用 Java 栈来存储中间数据。这样设计的原因是为了保持 Java 虚拟机的指令集尽量紧凑，同时也便于 Java 虚拟机在那些只有很少通用寄存器的平台上运行。另外，这种基于栈的体系结构，也有助于运行时动态编译器和即时编译器的代码优化。

本小节对组成 Java 虚拟机的这几部分做简要介绍。

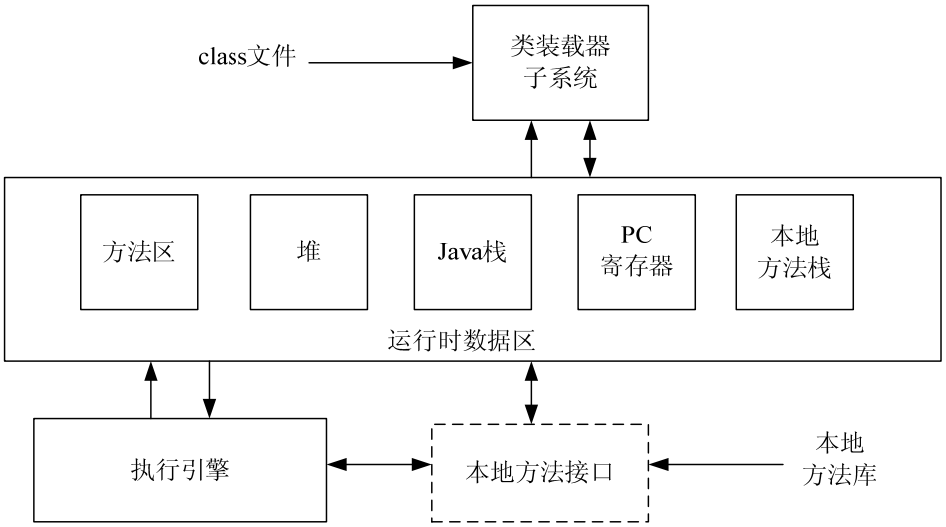

图 3.4　Java 虚拟机的内部体系结构

3.3.2.1 类装载子系统

在 Java 虚拟机中，负责查找并装载类型的部分被称为类装载子系统。Java 虚拟机有两种类装载器：启动类装载器和用户自定义类装载器。由不同的类装载器装载的类将被放在虚拟机内部的不同命名空间中。

类装载器子系统涉及 Java 虚拟机的其它几个组成部分，以及几个来自 java.lang 库的类。对于每一个被装载的类型，Java 虚拟机都会为它创建一个 java.lang.Class 类的实例。和所有其他对象一样，用户自定义的类装载器以及 Class 类的实例都放在内存中的堆区，而装载的类型信息则都位于方法区。

类装载器子系统除了要定位和导入二进制 class 文件外，还必须负责验证被导入类的正确性、为类变量分配并初始化内存、以及帮助解析符号引用。这些动作必须严格按以下顺序进行：

装载——查找并装载类型的二进制数据；

链接——执行验证、准备，以及解析；

验证——确保被导入类型的正确性；

准备——为类变量分配内存，并将其初始化为默认值；

解析——把类型中的符号引用转换为正确初始值。

只要是符合 Java class 文件格式的二进制文件，Java 虚拟机都必须能够辨别并装载其中的类和接口。只要给定某个类型的全限定名，启动类装载器就必须能够以某种方式得到定义该类型的数据。虽然用户自定义类装载器是 Java 程序的一部分，但类 ClassLoader 中的四个方法是通往 Java 虚拟机的通道，这四个方法如图 3.5 所示：

```
//Four of the methods declared in class java.lang.ClassLoader:
protected final Class defineClass (String name,  byte data[],
            int offset, int length);
protected final Class defineClass (String name, byte data[],
            int offset, int length, protectionDomain protectionDomain);
protected final Class findSystemClass (String name);
protected final void resolveClass (Class c);
```

图 3.5　用户自定义类装载器虚拟机接口

每个类装载器都有自己的命名空间，其中维护着由它装载的类型，所以一个 Java 程序可以多次装载具有同一个全限定名的多个类型。这样一个类型的全限定名就不足以确定在一个 Java 虚拟机中的唯一性。因此，当多个类装载器都装载了同名的类型时，为了唯一地标识该类型，还要在类型名称前加上装载该类型的类装载器的标识。Java 虚拟机

中的命名空间，其实是解析过程的结果，对于每一个被装载的类型，Java 虚拟机都会记录装载它的类装载器。当虚拟机解析一个类到另一个类的符号引用时，它需要被引用类的类装载器。

3.3.2.2 运行时数据区

如上文所述，运行时数据区存放 Java 虚拟机运行时需要的数据内容，分为方法区、堆、Java 栈、PC 寄存器和本地方法栈五部分。

方法区

在 Java 虚拟机中，被装载类型的信息存储在一个逻辑上被称为方法区的内存中。当虚拟机装载某个类型时，它使用类装载器定位相应的 class 文件，然后读入这个 class 文件，将它传输到虚拟机中。紧接着虚拟机提取其中的类型信息，并将这些信息存储到方法区。该类型中的类（静态）变量同样也存储在方法区中。

方法区的大小不是固定的，虚拟机可以根据应用的需要动态调整；同样方法区也不是连续的，方法区可以在一个堆中自由分配；另外，虚拟机允许用户或者程序员制定方法区的初始大小以及最小和最大尺寸等。方法区可以被垃圾收集，因为虚拟机允许通过用户定义的类装载器来动态扩展 Java 程序，因此一些类也会成为程序不再引用的类。当某个类变为不再被引用的类时，Java 虚拟机卸载这个类（垃圾收集），从而使方法区占据的内存保持最小。

对每个装载的类型，虚拟机都会在方法区中存储以下类型信息：这个类型的全限定名、这个类型直接超类的全限定名、这个类型是类类型还是接口类型、这个类型的访问修饰符、任何直接超接口的全限定名的有序列表。

除了上面列出的基本类型信息外，虚拟机还得为每个被装载的类型存储以下信息：该类型的常量池、字段信息、方法信息、除了常量以外的所有类（静态）变量、一个到类 ClassLoader 的引用以及一个到 Class 类的引用。

堆

Java 程序在运行时创建的所有类实例或数组都放在同一个堆中。一个 Java 虚拟机实例中只存在一个堆空间，所有线程共享这个堆。因为一个 Java 程序独占一个 Java 虚拟机实例，所以每个 Java 程序都有自己的堆空间，他们不会彼此干扰。但是同一个 Java 程序的多个线程却共享一个堆空间，多线程访问对象（堆数据）需要同步。

Java 虚拟机有一条在堆中分配新对象的指令，没有释放内存的指令。虚拟机自己负责决定何时释放不再被程序引用的对象，程序本身不用考虑何时需回收对象所占用的内存，完成这个任务的部分是垃圾收集器。垃圾收集器的主要工作就是主动回收不再被程

序引用的对象。它也可能去移动还在使用的对象，以此减少堆碎片。

另外，在 Java 中，数组是真正的对象。和其他对象一样，数组总是存储在堆中。数组也拥有一个与它们的类相关联的 Class 实例，不管数组的长度是多少，所有具有相同维度和类型的数组都是同一个类的实例。由于这点设计，Java 虚拟机中引入了数组越界检查的机制，但是效率较低，是我们需要优化的方向。

Java 栈

每当启动一个新线程时，Java 虚拟机都会为它分配一个 Java 栈。Java 栈以帧为单位保存线程的运行状态。虚拟机只会直接对 Java 栈执行两种操作：以帧为单位的压栈或者出栈。

每当线程调用一个 Java 方法时，虚拟机都会在该线程的 Java 栈中压入一个新帧。而这个新帧自然就成为了当前帧。在执行这个方法时，它使用这个帧来存储参数、局部变量、中间运算结果等等数据。Java 方法可以以两种方式完成：一种通过 return 正常返回；另一种通过抛出异常而中止。不管以哪一种方式返回，虚拟机都会将当前帧弹出 Java 栈，这样上一个方法的帧就变为当前帧。

栈帧由三部分组成：局部变量区、操作数栈和帧数据区。Java 栈帧的局部变量区被组织为一个以字长为单位、从零开始计数的数组，字节码指令通过从零开始的索引来使用其中的数据。局部变量区包含对应方法的参数和局部变量。和局部变量一样，操作数栈也是被组织成一个以字长为单位的数组，但和前者不同的是，它不是通过索引来访问，而是通过标准的栈操作来访问的，虚拟机把操作数栈作为它的工作区——大多数指令都从这里弹出数据，执行运算，然后把结果压回操作数栈。除了局部变量区和操作数栈外，Java 栈帧还需要一些数据来支持常量池解析、正常方法返回以及异常派发机制，这些信息都保存在 Java 栈帧的帧数据区中。

PC 寄存器

对于一个运行中的 Java 程序而言，其中的每一个线程都有它自己的 PC 寄存器，它是在该线程启动时创建的。当线程执行某个 Java 方法时，PC 寄存器的内容总是下一条将被执行指令的地址。如果该线程正在执行一个本地方法，那么此时 PC 寄存器的值是未定义的。

本地方法栈

对于一个运行中的 Java 程序而言，可能会用到一些跟本地方法相关的数据区。本地方法也可以通过本地方法接口来访问运行时数据区。因此本地方法和 Java 虚拟机之间是可以相互调用的。但是调用本地方法时不是操作 Java 栈，而是操作本地方法栈。如果某个 Java 虚拟机需要的本地方法是 C 语言方法时，那么它的本地方法栈是 C 栈。图 3.6 就是这种情况，一个线程调用一个本地方法，本地方法又回调虚拟机中的 Java 方法。

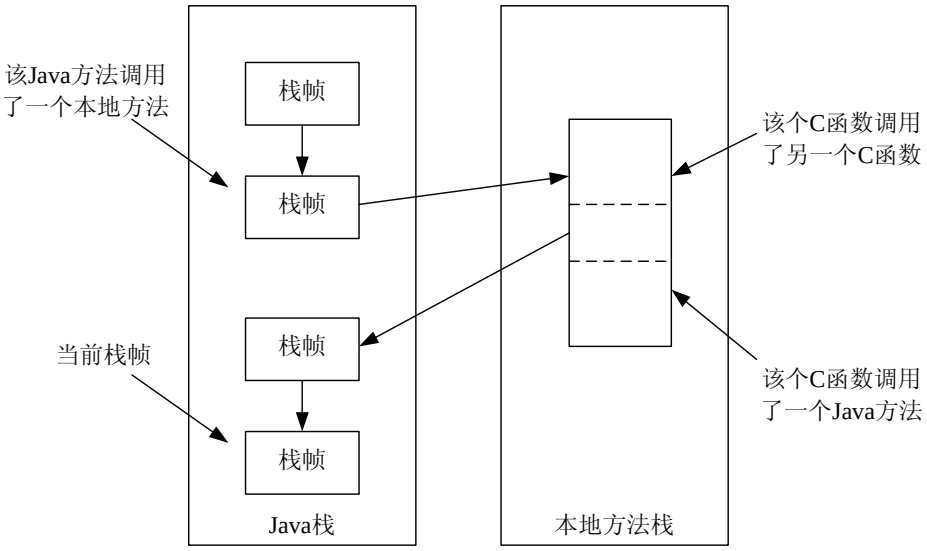

图 3.6　线程调用 Java 方法和本地方法时的栈

如图 3.6 所示，该线程首先调用了两个 Java 方法，而第二个 Java 方法又调用了一个本地方法，因此虚拟机使用了一个本地方法栈。假设这是一个 C 语言栈，其间有两个 C 函数，第一个 C 函数被第二个 Java 方法当做本地方法调用，而这个 C 函数又调用了第二个 C 函数，之后第二个 C 函数又通过本地方法接口回调了一个 Java 方法，最终这个 Java 方法又调用了一个 Java 方法，也就是当前方法。

3.3.2.3 执行引擎

执行引擎是 Java 虚拟机实现的核心，被装载的字节码由执行引擎编译成本地代码并执行。传统执行引擎的实现是解释器和即时编译器（Just-in-time compiler），其中解释器一次性解释字节码，这种执行引擎启动开销最小，它将待执行的字节码单条解释并执行；对于即时编译器而言，第一次被执行的字节码会被编译成本地机器代码，编译出的本地机器代码会被缓存，该方法下次被调用的时候可以重用，这种方法可以利用代码的局部性，达到一次编译多次执行的效果，并且配合对编译后代码的优化，达到较高的性能，但是缺点就是启动开销较大，对于小规模单次执行的代码不适用。

而龙芯平台的 HotSpot 虚拟机执行引擎实现了这两者的融合，称为自适应优化技术。自适应优化器搜集那些运行时才有效的信息，试图以某种方式把字节码解释和编译结合起来，利用解释器开销小，即时编译器效率高的优点，以达到最优化的性能。自适应优化器开始的时候对所有的代码都是解释执行，同时监视代码的执行情况，大多数程序花费 80%~90%的时间执行 10%~20%的代码。因为可以监视程序的执行情况，所以虚拟机可以识别哪些方法是程序的“热区”。当自适应优化器判断出某个特定的方法是热点方法后，它启动一个后台线程，把字节码编译成本地代码，并且花大力度优化这些本地代码。

自适应优化技术使程序最终把原来占 80%~90%运行时间的代码变为极度优化的、静态链接的本地代码，大大提高了程序的性能。当热点方法不再是热点后，取消那些编译过的代码，重新解释执行那些字节码。

3.3.3 本文采用的主要测试程序

在实验中我们使用 SPEC 基准测试程序，SPEC 基准测试程序 [84] 是由标准性能评估公司 (SPEC，Standard Per-formance Evaluation Corporation) 提供的测试程序，主要用于测试计算机系统的性能。我们选取 SPECjvm98 和 SPECjvm2008 两个主要的测试套件。

SPECjvm98[38]和 SPECjvm2008[64]用于对 Java 虚拟机平台客户端进行评估。包括硬件平台和软件平台两个方面。从软件角度出发，它模拟 Java 真实应用中的整数，浮点，库函数调用，I/O 等；从硬件平台出发，它评测处理器整点和浮点计算能力、访存能力。每个测试套件都包含了若干个程序，涉及不同领域，可以对 Java 虚拟机进行全面评测，是国际上公认的 Java 虚拟机性能测试程序。这两个测试套件程序的简介见表 3.1 和表 3.2。

3.4 本章小结

本章介绍了本文研究工作的软硬件平台。首先介绍了本文研究工作的硬件平台，总体概括了龙芯国产高性能通用处理器的内部结构；在该硬件平台上，本文以 Openjdk6 HotSpot Java 虚拟机为基础，实现了高效的 Java 处理器软件平台，本章对软件平台的内部结构原理进行了介绍。该软硬件平台也是本文的工作基础。本文将在此基础上，对 Java 虚拟机的总体性能优化进行研究，并在此平台上进行性能评测与分析。

表 3.1 SPECjvm98 基准测试程序简介

测试名称	功能描述
mtrt	光线跟踪算法，主要是浮点操作
jess	交互式脚本系统，利用一个数据集合和规则来解迷，主要是整数操作
compress	使用 LZW 算法压缩，解压缩数据，主要是整数和字符操作，访存操作
db	数据库访问操作的集合，主要是的操作时排序算法，同时有大量的同步操作
mpegaudio	主要是解码 mp3 格式文件，这个解码的过程包含大量的浮点操作
jack	从一个规范文件产生 Java 字节码，这个过程的实现使用了很多处理异常操作
javac	是一个 Java 编译器，主要是大量的字符串操作

表 3.2　SPECjvm2008 基准测试程序简介

测试名称	功能描述
startup.helloworld	测试 helloworld 程序从运行开始到结束所需的时间
startup.compiler.compiler	普通 java 编译所需要的时间
startup.compiler.sunflow	编译 sunflow 图像渲染引擎所需要的时间
startup.compress	测试压缩程序，单次压缩所需的时间
startup.crypto.aes	测试 AES/DES 加密算法，单次加解密所需的时间 输入数据长度为 100 bytes , 713KB
startup.crypto.rsa	测试 RSA 加密算法，单次加解密需要的时间 输入数据长度为 100 bytes, 16KB
startup.crypto.signverify	测试单次用 MD5withRSA, SHA1withRSA, SHA1withDSA, SHA256withRSA 来签名，识别所需要的时间。 输入数据长度为 1KB, 65KB, 1MB
startup.mpegaudio	单次 mpeg 音频解码所需的时间
startup.scimark.fft	单次快速傅立叶变换所需的时间
startup.scimark.lu	单次LU 分解所需的时间
startup.scimark.monte_carlo	单次运行蒙特卡罗算法所需的时间
startup.scimark.sor	单次运行 jacobi逐次超松弛迭代法所需的时间
startup.scimark.sparse	单次稀疏矩阵乘积所需的时间
startup.serial	单次通过 socket 传输 java 序列化对象到对端反序列化完成所需的时间（基于 jboss serialization benchmark）
startup.sunflow	单次图片渲染处理所需的时间
startup.xml.transform	单次 xml 转换所需的时间，转换包括 dom,sax,stream 方式
startup.xml.validation	单次 xml schema 校验所需的时间
compiler.compiler	在规定时间内，多线程迭代测试普通 java 编译，得出 ops/m
compiler.sunflow	在规定时间内，多线程迭代测试 sunflow 图像渲染，得出 ops/m
compress	在规定时间内，多线程迭代测试压缩，得出 ops/m
crypto.aes	在规定时间内，多线程迭代测试 AES/DES 加解密算法，得出 ops/m
crypto.rsa	在规定时间内，多线程迭代测试 RSA 加解密算法，得出 ops/m

crypto.signverify	在规定时间内，多线程迭代测试使用 MD5withRSA, SHA1withRSA, SHA1withDSA, SHA256withRSA 来签名，识别，得出 ops/m
derby	在规定时间内，迭代测试数据库相关逻辑，包括数据库锁，BigDecimal 计算等，最后得出 ops/m
mpegaudio	在规定时间内，多线程迭代 mpeg 音频解码，得出 ops/m
scimark.fft.large	在规定时间内，多线程迭代测试快速傅立叶变换，使用 32M 大数据集，最后得出 ops/m
scimark.lu.large	在规定时间内，多线程迭代测试LU 分解，使用 32M 大数据集，最后得出 ops/m
scimark.sor.large	在规定时间内，多线程迭代测试 jacobi逐次超松弛迭代法，使用 32M 大数据集，最后得出 ops/m
scimark.sparse.large	在规定时间内，多线程迭代测试稀疏矩阵乘积，使用 32M 大数据集，最后得出 ops/m
scimark.fft.small	在规定时间内，多线程迭代测试快速傅立叶变换，使用 512K 小数据集，最后得出 ops/m
scimark.lu.small	在规定时间内，多线程迭代测试LU 分解，使用 512KB 小数据集，最后得出 ops/m
scimark.sor.small	在规定时间内，多线程迭代测试 jacobi逐次超松弛迭代法，使用 512KB 小数据集，最后得出 ops/m
scimark.sparse.small	在规定时间内，多线程迭代测试稀疏矩阵乘积，使用 512KB 小数据集，最后得出 ops/m
scimark.monte_carlo	在规定时间内，多线程迭代测试蒙特卡罗算法，得出 ops/m
serial	在规定时间内，多线程迭代测试通过 socket 传输 java 序列化对象到对端反序列化（基于 jboss serialization benchmark），得出 ops/m
sunflow	在规定时间内，利用 sunflow 多线程迭代测试图片渲染，得出 ops/m
xml.transform	在规定时间内，多线程迭代测试 xml 转换，得出 ops/m
xml.validation	在规定时间内，多线程迭代测试 xml schema 验证，得出 ops/m

第四章　　类装载器虚方法调用的分析和优化

4.1 引言

Java 语言提供了两种基本的方法：实例方法和类（或者静态）方法[1][2]。这两种方法的区别在于：实例方法在被调用之前，需要一个实例，而类方法不需要；实例方法使用动态绑定，而类方法使用静态绑定。当 Java 虚拟机调用一个类方法时，它会基于对象引用的类型（通常在编译时可知）来选择所调用的方法。相反，当虚拟机调用一个实例方法时，它会基于对象实际的类（只能在运行时得知）来选择所调用的方法。

实例方法，又称虚方法。虚方法只有在运行时才能得知该方法的对象是什么，所以对一个虚方法的解析就比较复杂，只能用运行时动态解析的方式来确定目标方法。这种解析只能在运行到调用点时才能获得调用的类型信息，从而用该类型信息检索特定类的方法，然后调用该方法。这种动态解析一般被翻译成间接跳转，而这种间接跳转很容易引起流水线的停顿[65]。实例方法调用所占比例很大，比如在 SPECjvm98 测试程序中，Java 语言每隔 12 到 40 个字节码就会发生一次虚方法调用[66]，非常制约 Java 虚拟机的性能。

无论是虚方法还是静态方法调用，在调用之前都需要类对象的查找和装载，这部分工作由虚拟机中类装载器完成。在 Java 程序运行时，类装载器负责查找并装载类型，是 Java 虚拟机中非常重要的一部分。而类装载器的效率也很大程度上决定了虚拟机的整体性能。由于 Java 方法中的虚方法调用比较复杂，类装载器的实现效率比较低。因此需要优化类装载器中虚方法调用机制，提高 Java 方法调用的性能。

一般来说，在一个调用点上的调用类型不会频繁地变化[67]。有研究表明，在同一个调用点，大约有 95%的虚方法调用类型都是不变化的[42][68][69]。利用这种局部性，可以在调用点处进行缓存和方法查找。由于这些方法查找结果都内联在每个方法调用点，因此这种技术称为内联高速缓存[70]。虽然内联高速缓存能够较为有效地减少虚方法调用的开销，但是如果程序处理在同一调用点频繁调用多种不同类型的方法，该技术的处理就较为复杂，性能开销较大。

在内联高速缓存技术基础上，我们设计了硬件 CAM 查找表，使用 CAM 表，提出了一种基于硬件 CAM 查找表的软硬件协同设计的混合虚方法调用机制，在不引入额外开销的前提下，有效地解决了同一方法调用点频繁调用多种不同类型方法的问题。

本章第二部分对虚方法调用的相关技术进行介绍；第三部分统计并分析了在 Java 程序中方法调用的类型和比例，发现虚方法调用所占比例很大，优化这部分很有必要；第四部分介绍了我们虚拟机的软件平台 HotSpot 中虚方法调用机制；第五部分描述了硬件 CAM 查找表的设计和实现；第六部分介绍了软硬件协同设计虚方法调用的算法；接下

来进行了实验评测和分析；最后总结本章。

4.2 相关工作

虚方法调用时，目标方法依赖于接收者的类型信息，这些类型信息只有在实际运行的时候才能够得到，因此方法调用的开销很大。为降低方法调用的开销，解决方法调用的性能问题，近年来内联高速缓存成为了优化虚方法调用的主要技术。

内联高速缓存技术是许多运行时语言使用的一种优化技术，首先被 Smalltalk 语言所使用。内联高速缓存技术的目标是加速运行时方法调用的类型绑定，它记录上次方法调用时的类型，绑定在本次方法调用点处使用。这种技术在动态类型语言中使用得尤其广泛。

在这种技术中，调用点处分为几种状态。最初的状态是“未初始化”，如果运行时出现了未初始化的状态，那么就开始动态查找，然后在这个调用点存储类型结果并把状态改为单态，下次再执行到这个方法调用点的时候，如果比较成功，那就直接跳转调用该方法而不用再次查找。运行时语言在程序运行时加入了统计代码来统计代码执行时不能命中的次数，用于记录调用点处的命中情况。如果在单态状态的调用点遇到了目标代码与存储的方法不一致，那么就重新动态解析目标方法并且把状态改回未初始化。

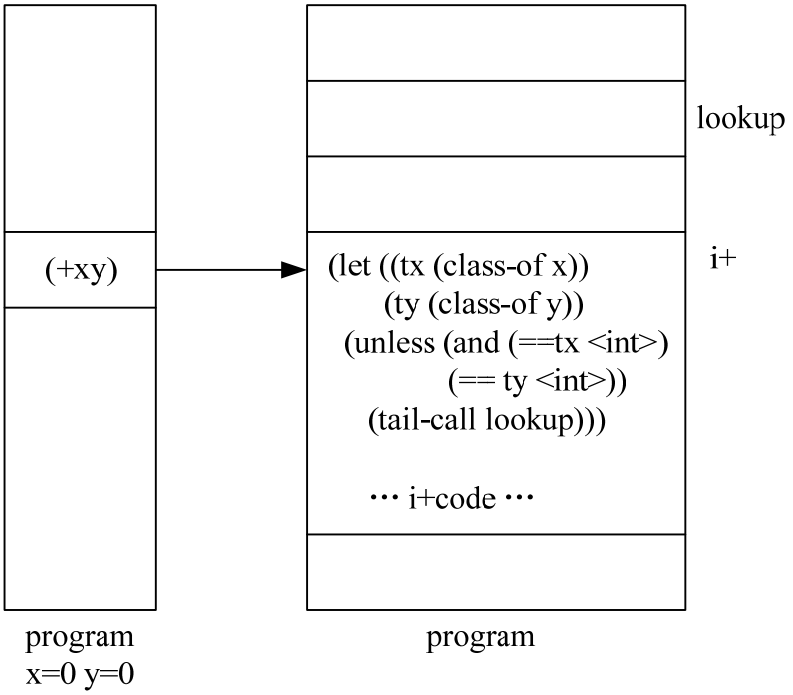

图 4.1　使用单态内联高速缓存的虚方法调用

例如图 4.1 所示，使用单态内联高速缓存[71,72,73]技术，程序中虚方法调用的执行流程。在这个程序中，虚方法调用 +xy，因为 x 和 y 的类型不能确定，所以+xy 操作需要在运行时才能确定。图中所示在第一次解析（lookup）之后，整型操作的代码内联缓

存在该调用点，下次如果 x 和 y 的类型变为浮点型，那么需要重新解析+xy，并且在调用点内联新解析的目标方法，替换目前的方法。

这种技术的一个典型实现是用一个寄存器存放类型值，在每次方法调用的时候，进行动态查找函数，查找到类型值之后，就把该类型保存到寄存器中，下次再调用的时候，就可以直接跳转到目标方法了。这样就将调用点和目标方法联系了起来。方法调用时，用当前的方法类型与寄存器中的类型相比较，如果相同，那么就直接跳转的目标方法，否则就跳转到动态查找函数，查找新的类型方法。这样只需要一次类型比较就可以解析目标方法，而不是原来的类型比较和调用点比较的两次操作，当前技术编译后是直接跳转，传统技术是查找虚方法调用表之后的间接跳转。

在一般情况下，这种技术能够解决的虚方法调用问题，但是如果同一个调用点总是调用不同类型的目标方法，那么性能损失将会非常大。在这种情况下，最初的那种内联技术会在未初始化和单态状态中间不停地切换，这样必然会带来性能损失。为了避免这种性能损失的发生，许多单态内联高速缓存的实现都进行了优化，引入了第三种状态——megamorphic 多态。在一定数量的失效之后，调用点的状态停留在 megamorphic 多态，而不再变为单态状态。在 megamorphic 多态状态，代码不再内联，只是执行最基本的动态解析。这样再遇到切换时，只需要执行动态查找即可。

虽然加入了 megamorphic 多态状态，使解析状态不再频繁切换，但是在 megamorphic 状态只是进行基本的动态解析，开销仍然很大。为了解决调用方法类型频繁切换的情况，出现了 polymorphic 多态内联高速缓存技术[74,75]。在这种技术中，如果处在单态的调用遇到了一个不同类型的方法，不是转换到未初始化状态，而是转换到 polymorphic 多态状态。在这种状态中，同一个调用点处会记录多个不同类型的目标方法。不同类型的目标方法数目是有上限的，一旦达到一定的上限，就转换到 megamorphic 多态状态，执行普通的动态方法类型查找。例如图 4.2 所示，使用 polymorphic 多态内联高速缓存技术，虚方法调用的执行流程。

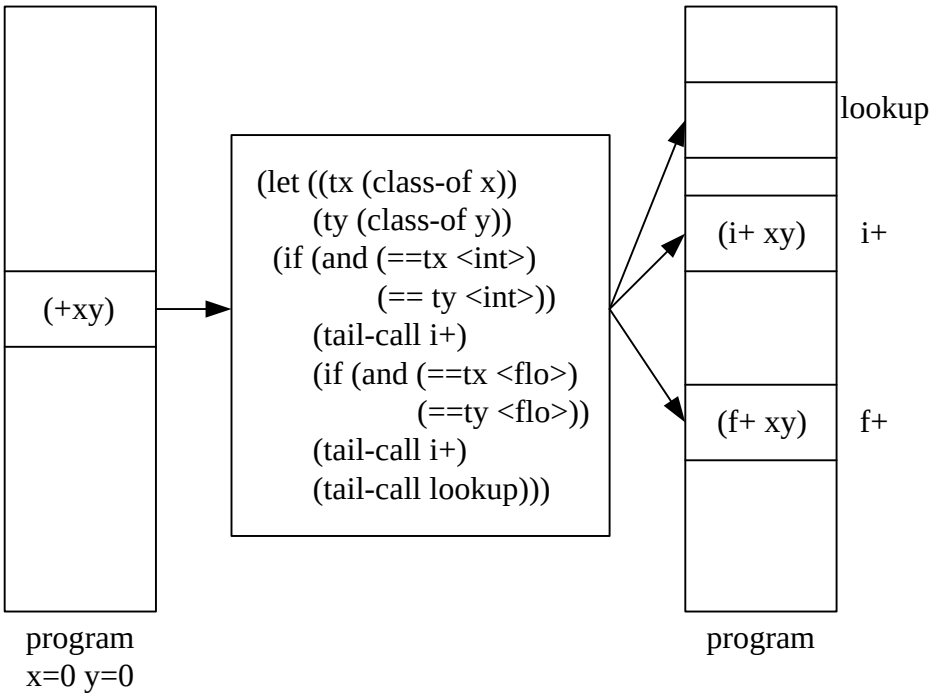

图 4.2　polymorphic 多态内联高速缓存

在这种技术中，第二次虚方法调用+xy 时，解析得到的浮点方法内联缓存，同时上一次内联缓存的整点方法不需要删除，两个方法内联在同一个调用点。使用跳转指令，首先跳转到处理多个调用 stub 的入口处，这段 stub 就是处理类型比较的，比较成功之后进行一个条件跳转，跳转到目标方法的入口。跳转表有固定的大小，并且可以增长，一旦方法的类型数达到了跳转表的上限，那么就转到 megamorphic 多态状态执行普通动态解析。

在 polymorphic 多态内联缓存和单态内联缓存技术的方法调用中，性能开销比较大的就是 polymorphic 多态状态查找不命中时动态方法查找。为了优化这种动态查找，有一种技术是 megamorphic 多态内联高速缓存。在这种技术中，一旦 polymorphic 状态不能命中转入 megamorphic 状态后，不是进行普通的动态方法解析，而是创建一段类型查找的代码来索引目标方法类型。这段代码比普通的动态方法解析更加高效。因为这时的调用方法是一个常数，减少了寄存器压力，并且查找和调用的代码也不是运行时查找，这样就能够借助于分支预测来提升性能。

单态内联高速缓存由于实现简单，使用较为普遍，但是它却不能处理在同一调用点频繁调用多种不同类型方法；多态内联高速缓存[8][15]虽然能够解决这一问题，但是由于实现较为复杂，在使用时会给程序带来额外的开销。本章基于硬件 CAM 查找表的虚方法调用机制，能够有效地解决这一问题，并且不引入额外的开销。

4.3 Java 程序方法调用的主要类型和比例

如上节所述，Java 语言方法分为实例方法（虚方法）和类方法（静态方法）两种。

在 Java class 文件中，有四条字节码实现了方法调用的功能，也分别处理不同形式的方法调用。这四条字节码分别是：invokevirtual、invokestatic、invokespecial 和 invokeinterface。其中，invokestatic 用于处理静态方法的调用，另外的三条 invokevirtual、invokespecial 和 invokeinterface 用于处理虚方法调用，它们的处理过程如下。

invokestatic

在执行 invokestatic 之前，Java 虚拟机使用操作数 indexbyte1 和 indexbyte2 产生一个指向当前类常量池的无符号 16 位索引。这个常量池入口包含一个纸箱将要调用的方法的符号引用，然后把需要的参数从栈中弹出，调用常量池索引指向的类方法。

invokevirtual

与 invokestatic 相同，在调用之前也需要使用操作数产生纸箱当前类常量池的索引，另外虚方法必须在一个对象中被调用，对每一次调用，需要把对象引用和参数从栈中弹出，调用常量池索引指向的方法。对象和参数必须在调用指令直向前，被其他指令压入所调用方法的操作数栈。

invokespecial

当根据引用的类型来调用实例方法，而不是根据对象的类来调用的时候，通常使用 invokespecial 指令。这又分为三种情况：实例初始化方法、私有方法和使用 super 关键字所调用的方法。invokespecial 和 invokevirtual 的主要区别在于它是静态绑定而不是动态绑定。

invokeinterface

操作码 invokeinterface 和 invokevirtual 的功能相同，它调用实例方法并使用动态绑定。这两条指令的区别在于：当引用的类型为类的时候，使用 invokevirtual；当引用类型为接口时，使用 invokeinterface。

针对上述方法调用类型，在龙芯 3 号硬件平台以及 Java 虚拟机系统中，使用 SPECjvm2008 测试程序，本文统计了 Java 程序执行过程中虚方法调用所占比例，如图 4.3 所示。

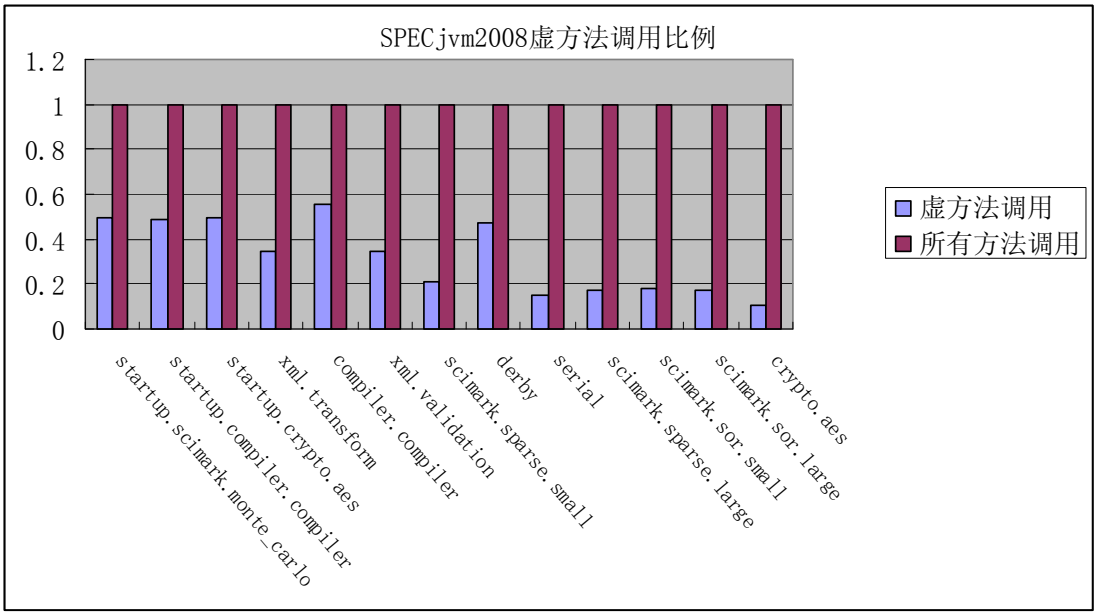

图 4.3　SPECjvm2008 虚方法调用比例

结果表明，在 SPECjvm2008 测试的大多数程序中，虚方法调用的比例都很高，因此我们需要在 Java 虚拟机中降低虚方法调用的开销，提高类装载器的性能。

4.4 HotSpot 虚方法调用单态内联高速缓存技术

HotSpot 是开源项目 Openjdk6 的 Java 虚拟机核心，是 Java 的标准实现。由于 HotSpot 的标准性和稳定性，我们选取 HotSpot 作为 Java 虚拟机研究平台。在 HotSpot 中，虚方法调用采用了优化的单态内联高速缓存技术。HotSpot 中对象内联的结构如图 4.4 所示。

如上一小节所述，Java 程序方法调用主要就是静态方法和虚方法两种。静态方法的目标方法类型时唯一的，可以在编译时获得，因此不需要在运行时解析；虚方法的调用目标方法类型不确定，在编译时无法获得目标方法的值，只能在运行时动态解析。

在 HotSpot 中，对这两类方法的调用都做了优化的处理。

对静态方法调用而言，虽然不需要运行时动态解析，但是编译时也需要解析目标方法，HotSpot 采用了简单内联的方法，将本次解析获得的方法地址内联在调用点，如果下次运行时目标方法地址一致，那么直接跳转，不需要编译时再解析，只有内联不命中的情况再解析。

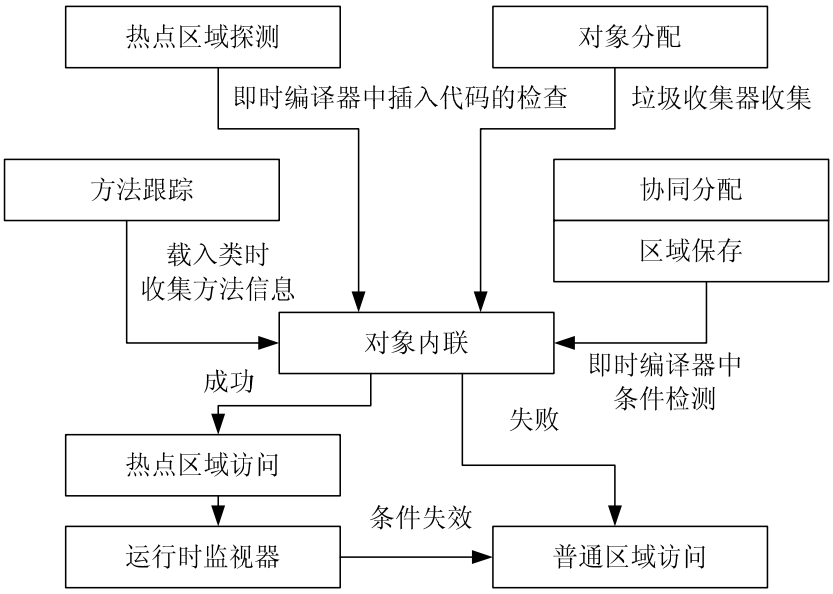

图 4.4 HotSpot 中对象内联结构

对虚方法调用而言，HotSpot 采用了优化的单态内联高速缓存技术[5][7]，这种技术比最初技术的改进是引入了第三种状态——megamorphic 多态。在调用点处加入了统计执行时不能命中的次数的功能，如果达到了一定次数不能命中，那么该调用点的状态就转为 megamorphic 多态，程序再执行到该调用点时，不与此前存储的类型值比较，只执行最基本的动态方法解析，而且该调用点的状态一直保持在 megamorphic 多态。直至一定次数的方法调用之后，该调用点的状态再转为未初始化。这样就避免了多次不命中状态改变而带来的性能开销。三种状态转换图如图 4.5 所示：

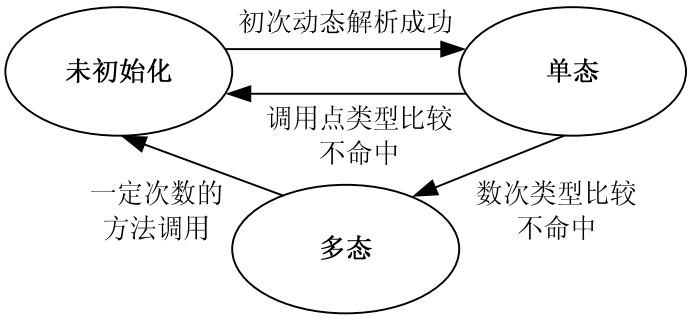

图 4.5 单态内联高速缓存状态转换图

这种技术在一般情况下能够带来性能的大幅度提升，但是如果同一个方法调用点总是调用不同类型的目标方法，那么性能损失也将会非常大。下面的例子就是最极端的不能命中的情况：

```
Var value = {1,”a”,2,”b”,3,”c”,4,”d”};
For (var I in values){
    Document.write(values[i].toString());
}
```

在这个程序段中，toString 是虚方法调用，而且每次循环调用的方法类型都在变化，在这种情况下，使用 HotSpot 中的单态内联缓存技术就不会带来性能的提升。因为每次方法调用的类型都在变化，每次类型比较都失败，目标方法的跳转要进行最基本的动态方法解析，而调用点的状态也在不停改变，必然会带来性能的损失。

虽然这种极端的例子并不多见，但是虚方法调用类型变化的情况却很普遍，由虚方法调用开销导致的程序性能下降就比较严重。

4.5 虚方法调用全局索引表的硬件设计

为了优化 Java 虚拟机中虚方法调用引起的性能开销，我们实现了 CAM（content associated memory）的方法处理虚拟目标方法索引和查找。CAM 通过硬件实现，并且可以通过软件进行管理，结构如图 4.6 所示。所有的 CAM 项可以通过指令 CAMWI 写入，RAM value 部分可以通过指令 CAMRI 读出。指令 CAMPI 和 CAMPV 用来查询 CAM，当 CAM 命中时上述两条指令分别产生索引和值。通过上述硬件，当 Java 虚拟机遇到一条方法调用指令时，该方法调用指令的地址和虚方法的类型异或后被送到 CAM 进行全相联查找。如果命中，方法调用就转到目标方法的地址执行；如果没有命中，则转入方法调用的软件解析代码执行。

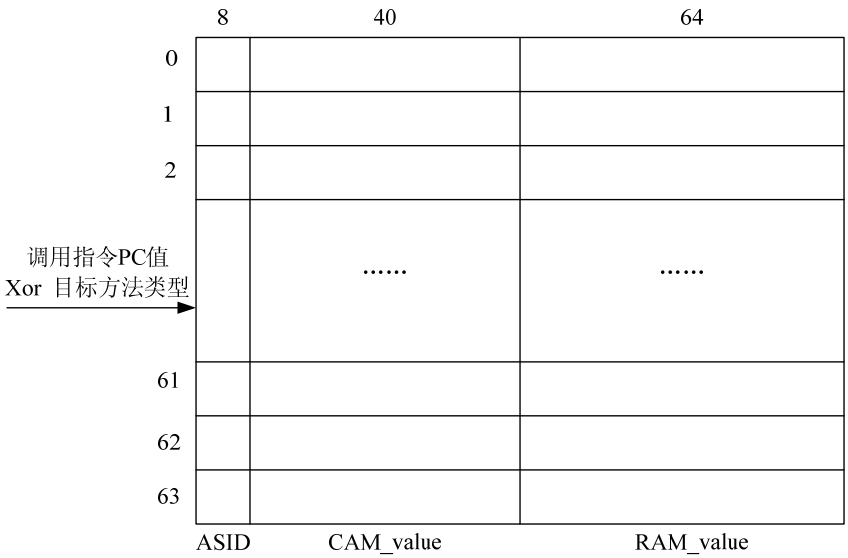

图 4.6 CAM 结构

硬件 CAM 表共 64 项，每项拥有 8 位 ASID、40 位 CAM VALUE 和 64 位的 RAM VALUE。此外，处理器还提供了四条指令用于 CAM 的访问，如表 4.1 所示。Java 虚拟机管理代码访问该 CAM 表时，如果命中，仅仅需要 2 条汇编指令就可以得到正确的目标方法地址，并且不需要访问内存，极大地提高了效率。

表 4.1 CAM 表操作指令

指令	作用
CAMPI	根据索引查找 CAM 表
CAMPV	根据值查找 CAM 表
CAMWI	根据索引写 CAM 表
CAMRI	根据索引读 CAM 表

为了得到合适的 CAM 项数目，在处理器设计阶段，不同的 CAM 项数目也进行了评估。实验在 Java 虚拟机上模拟硬件的 CAM 表，使用该 CAM 表来实现虚方法调用。实验结果如图 4.7 所示，纵轴为各程序虚方法调用的命中率，横轴为 SPECjvm2008 测试的不同程序。最终龙芯 3 号实现了 64 项的 CAM。因为这个项数的 CAM 能够达到较好的命中率，同时物理实现较为容易。该结果是性能和物理设计复杂度权衡的结果，在尽量不占用太大面积的情况下，达到相对较优的效果。

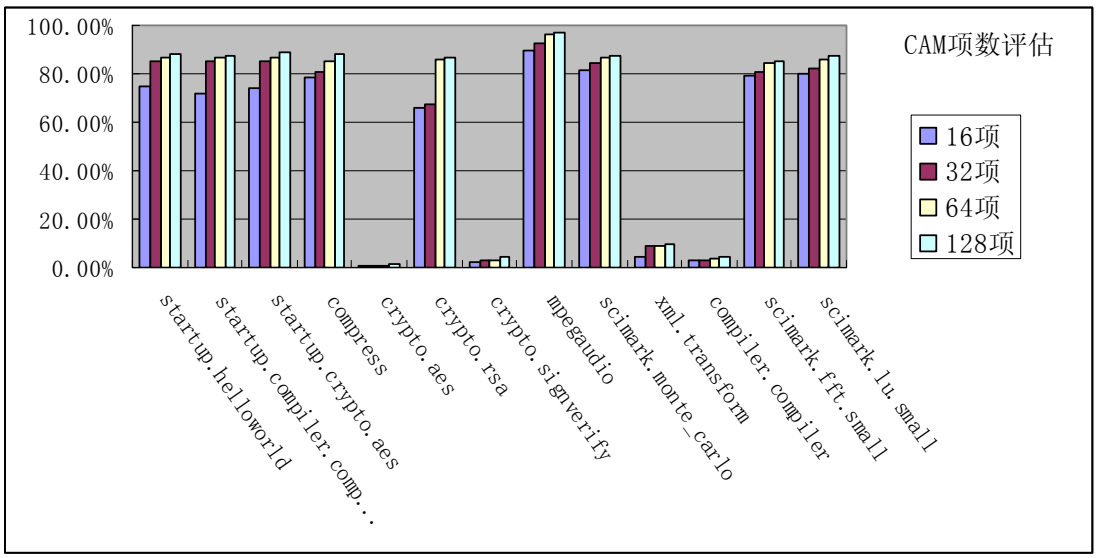

图 4.7 CAM 项数评估

4.6 软硬件协同设计的虚方法调用算法

为了提高 Java 虚拟机中虚方法调用的效率，更加完备地处理 HotSpot 虚拟机中单态

内联高速缓存技术所不能处理的情况，在 HotSpot 虚拟机中，我们在原有虚方法调用机制的基础上，增加使用 CAM 硬件查找表解析虚方法调用。HotSpot 在某个方法调用点首次进行虚方法调用时，使用最基本的动态方法解析，解析成功得到目标方法后直接跳转至该目标方法，同时在该调用点处存储本次方法调用的类型信息以及目标方法的地址。程序再次运行到该调用点时，使用本次调用的类型与存储在该调用点的类型信息比较，如果比较成功，那么直接跳转到存储的目标方法，并且在此时开始填写 CAM 表的过程。因为类型比较成功，该类型及目标方法被视为热点方法，此时将调用指令的地址与类型异或（采用异或作为哈希函数），用 CAMWI 指令写入 CAM 中作为索引键值，同时 CAMWI 指令也将目标方法地址写入 CAM 中，这样 CAM 中的一个表项就完成了写操作。完成写操作的 CAM 表项如图 4.8 所示。

当虚拟机再次遇到虚方法调用时，首先将当前方法调用指令的 PC 值与调用方法的类型做异或操作，然后用异或的结果在 CAM 表中查找（使用 CAMPV 指令）。如果有 CAM 表项命中，说明此次方法调用点跟存储的调用点相同，是同一个方法调用，并且调用目标方法的类型也相同，那么就跳转到相应表项存储的目标方法地址执行；如果没有 CAM 表项命中，说明该调用点没有写入 CAM 表，或者是已经写入了 CAM 表但是目标方法的类型不同，此时就需要重新调用 HotSpot 中原有的单态内联高速缓存来查找目标方法，如果这条路径成功，那么再次将类型值和目标方法写入 CAM 表中，下次虚方法调用的时候可以继续查找 CAM 表。我们软硬件协同设计的虚方法调用机制流程如图 4.9 所示：

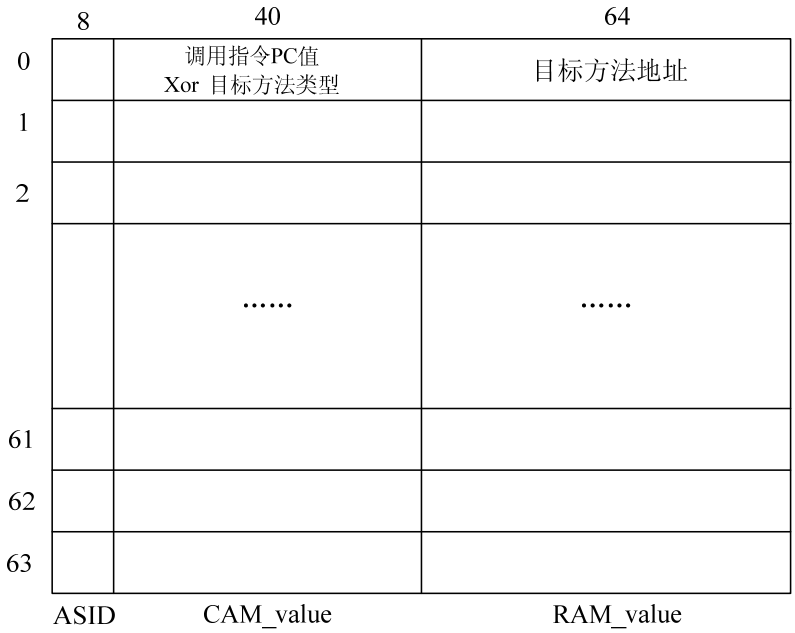

图 4.8 完成写操作的 CAM 表项

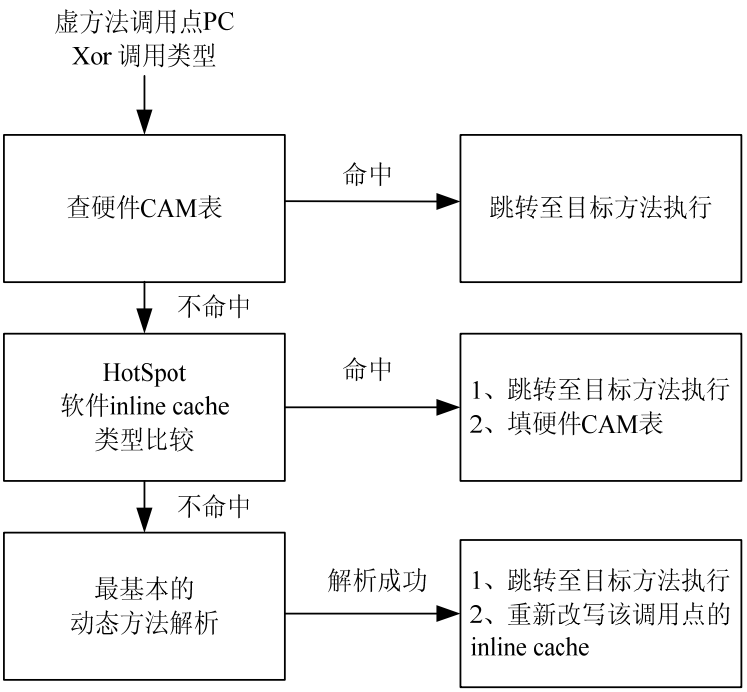

图 4.9 虚方法调用机制流程图

使用硬件 CAM 表软硬件协同设计的虚方法调用技术，有效地解决了原来 HotSpot 虚拟机处理同一个方法调用点频繁调用不同类型的目标方法效率低下的问题。这是因为我们的 CAM 表是一个全局的方法表，一共有 64 个表项，可以最多存储 64 个不同调用点和类型，而原有的 inline cache 技术只是每一个调用点存储一个调用类型。所以当一个调用点有多个不同类型调用时，都可以在 CAM 表命中。图 4.10 是最基本的动态解析、普通内联高速缓存、增加了 CAM 表的虚方法解析机制之间的比较。

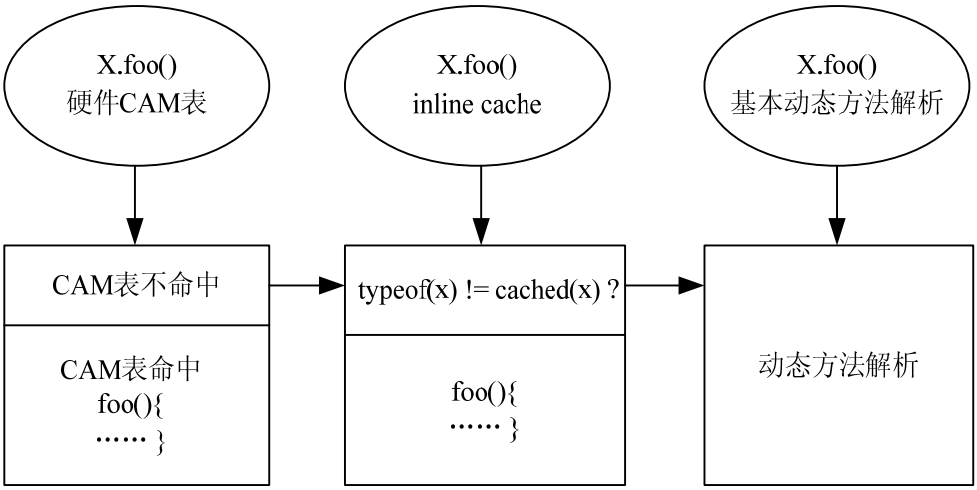

图 4.10 三种动态解析技术比较

另外，由于 CAM 表容量的原因，不可能把所有虚方法调用的情况都保存下来，除了因为硬件设计权衡而决定的 CAM 大小外，CAM 表的替换算法以及使用算法对间接跳转的优化效果有很大的影响。使用龙芯 3 号提供的 CAM 表机制如果表项命中，则能够快速跳转至目标方法，但一旦发生缺失，重填等开销会造成一定的性能损失，因此 CAM 表的命中率比较重要。当 CAM 表设计确定下来，填表的替换算法就是影响命中率的重要因素。本文对填写 CAM 表的替换算法做了一定研究，常用的替换算法主要有：LRU 算法、满则清空算法、预测式清空算法、FIFO、随机替换、轮转替换等等。考虑到实现的复杂度和算法的性能情况，我们使用了基于轮转的随机替换算法。

4.7 实验结果和分析

实验使用的硬件平台是龙芯 3 号处理器。龙芯 3 号处理器是 4 核的高性能通用处理器，我们在处理器中实现了用于虚方法调用的硬件 CAM 表，在加入 CAM 表之后，整个处理器的面积增加不足 5‰，功耗增加小于 1‰，开销可以忽略，符合硬件设计规范。

首先，使用 virtual_test 程序评估设计的性能，该程序代表了在同一调用点频繁调用不同类型的方法调用情况，具有典型性。程序的代码如图 4.11：

```
public class InlineCache
{

 static   Animal[] animal = new
Animal[8];
    static {
    animal[0] = new Animal();
    animal[1] = animal[0];
    animal[2] = new Dog();
    animal[3] = animal[2];
    animal[4] = new Cat();
    animal[5] = animal[4];        }

  public static void main(String argv[])
  {
    int i = 0;
    int ret;

    while (i++ < 1000000) {
      run(i%6);
        }
  }

  public static void run(int i)
  {
      int ret;
      ret = animal[i].run();
  }
}
```

```
class Animal
{
  public Animal() {
  }

  public int run() {
          int a = 12;
          return a;
    System.out.println("animal is running.");
  }
}

class Dog extends Animal
{
  public Dog() {
    super();
  }

  public int run() {
          int b = 0;
          int c = 23;
          int d = b+c;
          return d;
    System.out.println("dog is running.");
  }
}

class Cat extends Animal
{
  public Cat() {
    super();
  }

  public int run() {
          int a = 345;
          int b = a*12;
          return b;
          System.out.println("cat is running.");
  }
}
```

图 4.11　virtual_test 程序

从程序中可以看出，方法调用 animal[i].run()是虚方法调用，具体执行的目标方法依

赖于 animal 的具体类型而定，而主循环在执行时每次的调用类型都是不同的。使用该程序，改进前后 HotSpot 虚拟机的虚方法类型命中率以及程序性能的对比如表 4.2 所示：

表 4.2 改进前后 virtual test 测试结果

Virtual test	改进前	改进后
Inline cache 命中率	13.3%	76.4%
程序运行时间（秒）	36.098	30.250

从表 4.2 可以看出，改进后虚方法调用类型的命中率从 13.3%上升到了 76.4%，由于命中率的提高意味着可以直接从 CAM 中得到目标方法的地址，避免了动态方法解析以及访存操作，程序的性能提高了 16.2%。由于该程序的行为是在同一方法调用点频繁变换虚方法调用的类型，因此使用我们软硬件协同设计的虚方法调用机制使 HotSpot 虚拟机在处理该方法调用时有了大幅度的性能提升。

其次，本文使用 SPECjvm98 进行测试。经过测试，改进前后的 HotSpot 虚拟机运行 SPECjvm98 的性能比较如图 4.12 所示：

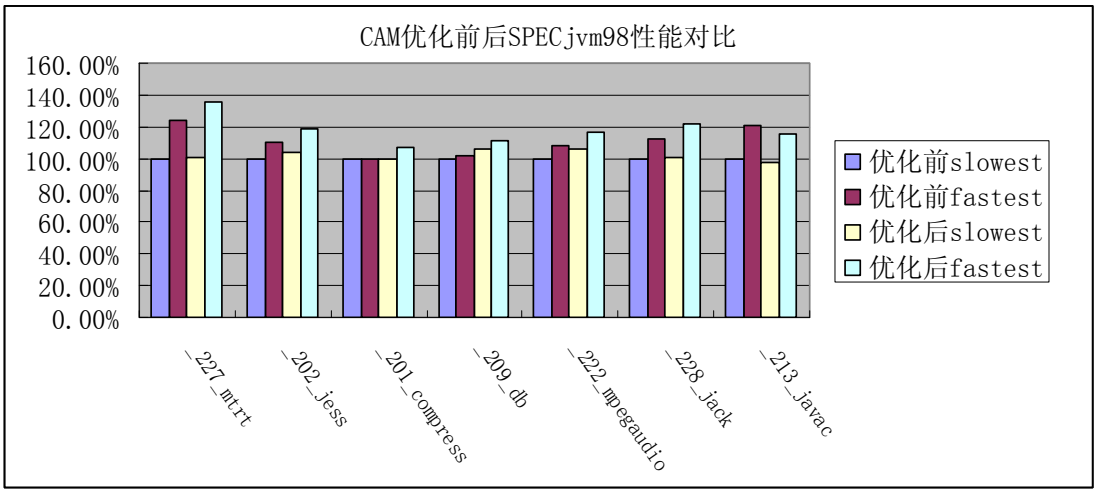

图 4.12 改进前后 SPECjvm2008 测试结果

从上图中可以看出，优化后，SPECjvm98 各程序的性能基本都有不同程度的提高，效果最明显的_209_db 测试程序性能提升 9%，该程序中虚方法在同一调用点调用不同类型目标方法的频率较高，本文方法能够有效地解决这一问题。其他大部分程序都有 4%-8%的性能提升。另外，从图中看到，测试程序_213_javac 的性能有 3%的下降，这是由于在这个测试程序中，同一调用点虚方法调用类型的重复度不高，改进后的机制在首次内联高速缓存命中后需要有填写 CAM 表的动作，这样就引入了性能开销，造成了一定程度的性能下降。总体来说，对于 SPECjvm98 测试程序，改进后 HotSpot 虚拟机的性能平均提升 6.4%，达到了我们设计之初的目的。

4.8 本章小结

虚方法调用时动态方法解析带来的性能损失，一直以来都是造成 Java 语言性能低下的重要原因。本文优化了传统的内联高速缓存技术，提出并实现了一种基于硬件 CAM 查找表的软硬件协同混合虚方法调用机制。实验表明，在不引入额外开销的前提下，有效地解决了同一调用点频繁调用多种不同类型方法的问题。调用点处虚方法的命中率从 13.3%上升到了 76.4%，程序的性能提升 16.2%，SPECjvm98 性能平均提升 6.4%。

第五章　　运行时环境的访存分析和优化

5.1 引言

运行时环境由方法区、堆、Java 栈、PC 寄存器、本地方法栈组成，是 Java 虚拟机运行的存储区，是 Java 虚拟机的重要组成部分。Java 虚拟机中的访存操作在虚拟机与运行时环境交互时非常重要。首先，即时编译器动态地生成编译后的二进制代码，编译后代码是编译器生成的数据，该部分数据首先存放在生成代码缓冲区，然后在安装代码时，将该部分代码从生成代码缓冲区拷贝至可执行代码数据区，这时的访存操作使用普通库函数提供的内存拷贝，效率较低。另外，Java 虚拟机是栈式结构的，编译过程中会出现大量的保存栈现场的操作，需要将栈中内容保存入存储器中，而这时的按字访问的访存操作已经远远不能满足上下文切换的性能要求。其次，为了保证 Java 程序运行的正确性，必须保证程序在运行过程中的访存安全。对于普通的访存操作，使用类型检查和字段偏移的验证可以消除非法访存；对于程序中的数组访问，则需要运行时的界限检查确定数组访问是否越界。

在 Java 语言规范中，检查数组是否越界尤其困难。首先， Java 语言数组的长度是动态决定的，数组分配之初，不能够确定数组的真正长度，因此数组越界检查只能在程序运行时进行，增加了程序运行时的开销；其次，在 Java 语言中数组是作为对象出现的，这些对象在方法调用时使用索引传递，或者同其他的对象一起存储，因此，在数组的调用点和访问点之间没有明显的联系；最后，由于 Java 语言比其他语言在异常的精确性上要求更加严格，因此数组越界检查的实现也不同。

鉴于 Java 语言的上述特性，Java 程序中检查运行时数组访问是否越界，需要在数组访问之前，加入判断数组索引是否合法的指令，在程序运行时首先执行该检查指令。数组索引的合法值是 0 至数组长度减 1 之间，如果索引值在该范围之内，那么进行正常的数组访问，否则引发数组越界例外。而引入这种检查机制的开销是比较高的，这主要源于两方面的因素：一是执行数组越界检查需要一条 load 数组长度的访存指令和一条比较指令，而每次检查都需要执行这两条指令无疑增加了程序的开销；二是 Java 语言规范要求精确异常，精确异常需要当异常发生时，在异常点保持程序的状态。Java 虚拟机一旦发现数组访问的索引值不合法，就会引发异常，由于精确异常的要求，越界检查指令附近的代码也就无法按原有的优化方法进行优化，这样就限制了像代码移动或者循环展开等进一步优化的机会，导致程序的性能下降。

在 Java 程序中，特别在科学计算类程序中，数组访问所占比重很大，因此处理数组访问越界检查的传统方法就会导致很大的开销。例如，矩阵乘算法的数组越界检查时间占程序执行时间的 50%，另外像动画类的数据计算密集型应用，受数组越界访问检查的

影响也很大。如此大的开销必然会造成程序性能低下。为了在保证 Java 程序安全运行的情况下提高程序的性能，减小数组越界检查的开销，需要优化现有的机制，提出一种更加高效的数组越界检查方法。

鉴于上述两方面需求，本文设计并实现了 128 位访存和有界访存指令，提出了一种软硬件结合的访存优化方法，有效地提高了 Java 虚拟机中内存拷贝的效率，降低了数组越界检查的开销，优化了 Java 程序的整体性能。

本章下面的五部分详细介绍了上述内容。第二部分列举了降低数组越界检查开销的主要技术；第三部分描述了新型访存指令的硬件设计；第四部分介绍了使用新型访存指令处理数组越界检查优化和上下文切换的算法设计；第五部分对这两方面进行了实验评估和分析；第六部分总结本章工作。

5.2 相关工作

数组越界检查给程序带来的开销非常大，特别是在计算型程序中。但是程序运行时发生数组访问越界的情况却很少，大部分越界检查是冗余的。有些数组越界检查在程序运行过程中永远不会引发异常，这种情况下越界检查的存在没有作用，这类越界检查被称为完全冗余；另外还有一类越界检查指令，它们不是完全冗余的，在某些情况下会引发异常，因此不能完全消除，但是通过移动或合并检查指令可以减少越界检查的数量。例如，循环内的越界检查指令可以移动到循环之前执行，可以大大减少运行时越界检查执行的次数。这类越界检查被称为部分冗余。

从这一点出发，学术界对数组访问越界检查展开了研究，旨在消除冗余的越界检查，提高数组访问的效率。其中部分的研究是基于不要求精确异常的语言，不是基于 Java 语言。这些研究主要分为两大类：第一，使用静态的数据流分析确定越界检查是否是冗余的[76,77,78,79,80]；第二，在编译期间使用数据流和符号信息减少程序中的越界检查[81,82,83,84,85]。

第一类技术是使用数据流信息证明越界检查在运行时是冗余的，可以消除该越界检查。在这类技术中，主要是使用循环、数组界限等信息来证明在一个循环中，访问冲突会不会发生，以确定检查指令是否需要。这种方法的难点在于访问冲突是否发生在编译期间有时很难确定。使用 JIT 编译器能够解决这个问题，但是开销比较大。

第二类技术目标是在不完全冗余情况下减少越界检查的数量。即使某个越界检查不能够完全消除，也可以减少检查的开销。为达到目标，主要用两种方法：（1）将越界检查移动到循环之外；（2）将多个越界检查合并成一个越界检查。但是这类技术在 Java 程序中却很难运用，因为 Java 规范要求精确异常，异常发生后程序的执行状态需要保证，而移动或者合并越界检查后就破坏了程序的行为。

基于 Java 语言数组越界检查的研究也有很多。Bodik et al.提出了一种按需要执行数组越界检查消除的算法[86]，这个算法只消除那些执行特别频繁的数组越界检查；Qian et

al.利用过程间分析，对一个方法中的关键路径建立加权图[87]，并利用该图来进行冗余检查的查找；另外还有一些方法利用注解 Java 语言的字节码来减少检查的数量[88,89]，这种方法的优点是不必修改 JIT 翻译器，算法上较为简单，但是类文件会更大并且需要在 JVM 中对注解进行验证。

在 Java HotSpot 虚拟机中也有过数组越界检查优化的研究。Java HotSpot 虚拟机执行引擎生成代码分为三个阶段，首先由字节码生成 HIR 表示，然后将 HIR 表示转换成 LIR 表示，最后由 LIR 表示生成可执行二进制代码。这些中间代码表示都是以基本块为单位的，其中 HIR 的指令格式是 SSA 格式，对于每条指令只有一个赋值点，该形式能够更方便地进行代码优化，因此，在这一层虚拟机对代码进行很多全局优化，包括死代码消除、空指针检查消除等；LIR 表示是类似于机器代码的指令表示，但也是平台无关的，在该层进行寄存器分配；最后由 LIR 生成可执行代码。对于数据越界检查的优化发生在 HIR 全局优化之后，LIR 层生成之前，因为全局优化有利于数组越界检查的消除。增加数组越界检查优化机制的 HotSpot 执行流程如图 5.1 所示。

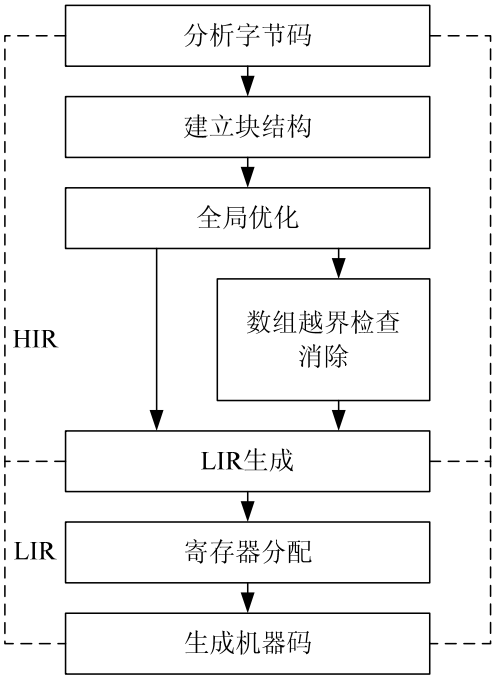

图 5.1 数组越界检查优化后的 HotSpot 执行流程

上面的各种优化方法中，较简单的算法不能够消除部分冗余的越界检查，只能消除完全冗余的检查；而较复杂的算法由于需要依赖于程序运行过程中的即时编译器，因此开销很大，不适用于我们对程序高效的要求。

5.3 新型访存的硬件设计

如前文所述，由于访存操作在 Java 虚拟机与运行时环境交互时具有非常重要的作用，而效率较低的上下文切换和数组越界检查机制严重影响了 Java 虚拟机的性能。针对这两个方面，我们实现了新型的硬件访存操作，用于优化 Java 虚拟机的访存机制，提高 Java 虚拟机的性能。

5.3.1 越界检查指令的硬件设计

为了优化 Java 程序中数组越界检查机制，减小数组越界检查的开销，在 MIPS 体系结构中，我们实现了一种特殊的访存指令，这种访存指令在访存操作的同时附带检查地址是否越界的操作，被称为有界访存指令。

在处理器原有的 MIPS 访存指令基础上，有界访存指令增加了检查所访问地址是否越界的操作，硬件实现如图 5.2 所示。包括物理寄存器堆、译码器和运算部件。物理寄存器堆包括上界地址寄存器，用于存储作为上界的有效地址；以及下界地址寄存器，用于存储作为下界的有效地址。其中上界地址寄存器、下界地址寄存器可以是物理寄存器中任一通用寄存器。译码器在 CPU 原有译码器的基础上增加对新型有界访存指令译码的功能。运算部件包含两个判断模块，第一判断模块用于在访存指令中，根据上界地址寄存器中存储的上界地址和/或下界地址寄存器中存储的下界地址，判断指令操作数地址的有效性；第二判断模块使用独立的比较指令判断访存操作数的有效性。

访存指令译码后，寄存器中的地址界限作为指令的一个寄存操作数，当操作内容数据根据 base 方式形成有效地址后，该有效地址首先跟寄存器中的地址界限比较，如果符合指令的语义条件，即如果寄存器为上界地址寄存器，该有效地址小于或者等于寄存器中的地址界限，或者如果寄存器为下界地址寄存器，该有效地址大于或者等于寄存器中的地址界限，那么完成正常的访存操作；否则引发地址错例外。

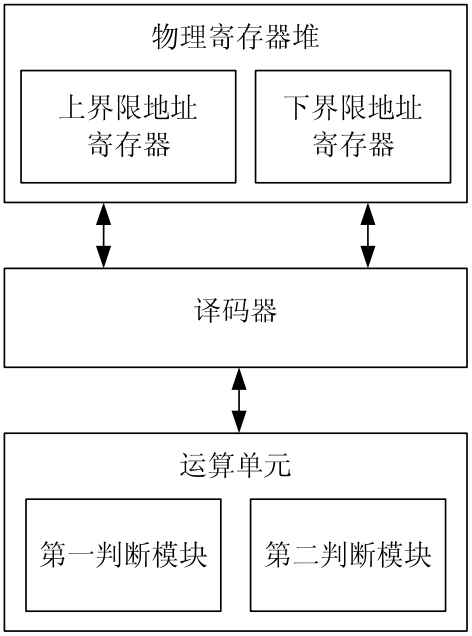

图 5.2 有界访存指令硬件实现

例如带上越界检查的字节访存指令，指令格式是 gsLBLE rt, base, bound，这条指令是运算部件中第一比较模块实现的。执行时先从 GPR[base]的内容中得到有效地址，如果有效地址不小于等于 GPR[bound]中的内容，则发出地址错例外；否则根据这个有效地址从内存中取出 8 位字节数据，将这个数据进行符号位扩展后存到 GPR[rt]。另外，单独的地址比较指令由第二比较模块实现，例如比较指令 gsLE rs, rt，两个寄存器中存放的都是地址值，该指令比较通用寄存器 GPR [rs]和通用寄存器 GPR [rt]中的值，如果 GPR [rs]中的值小于等于 GPR [rt]中的值，则顺序执行下一条指令；否则引发地址错例外。

我们一共实现了 24 条取数和存数指令，包含 16 条定点指令，8 条浮点指令，寻址方式都是 base 方式。这些指令分别为带上边界条件的存取字节、带下边界条件的存取字节、带上边界条件的存取半字、带下边界条件的存取半字、带上边界条件的存取字、带下边界条件的存取字、带上边界条件的存取双字、带下边界条件的存取双字、带上边界条件的存取单精度浮点数、带下边界条件的存取单精度浮点数、带上边界条件的存取双精度浮点数、带下边界条件的存取双精度浮点数。如表 5.1 所示。

另外还实现了 2 条地址比较指令，用于单独对地址大于或者小于界限的比较。如表 5.2 所示。

5.3.2 128 位访存指令的硬件设计

为了解决 Java 虚拟机上下文切换效率低下的问题，本文在 MIPS 体系结构下，实现了 128 位的访存操作。在处理器中将原来 64 位的数据通路扩展为 128 位，并设计了 4 条相应的访存指令，指令格式和功能如表 5.3 所示。这些指令分别用于存取整点和浮点

操作数，该指令每次存取数据都是以 128 位为单位，一次访存操作可以完成 4 条传统以字为单位访存的工作。这样不仅提高了访存的带宽，而且避免了访存操作过多而带来的处理器队列阻塞，减少了程序中指令的停顿。提高了处理器中访存操作的整体效率。

5.4 软硬件协同设计的 Java 虚拟机访存机制

如上一小节描述，本文设计并实现了新型访存指令。使用新型访存指令需要修改 Java 虚拟机中的访存机制。我们以 Java HotSpot 虚拟机为例，使用有界访存指令，优化虚拟机中的数组越界检查机制；使用 128 位访存指令，优化虚拟机中上下文切换机制。

5.4.1 数组越界检查软硬件优化

在 Java 字节码中，一共有 8 条字节码用于处理数组访存操作，分别是 iaload、laload、faload、daload、aaload、baload、caload 和 saload。以 iaload 为例，该字节码用于从数组中装载 int 类型值，在 Java HotSpot 虚拟机中对该字节码的实现是首先检查数组索引值，如果索引值合法，那么继续访存；否则引发异常。在 MIPS 体系结构下，该字节码的 MIPS 本地代码实现如图 5.3 所示。

```
// check index
Label ok;
__ lw(AT, array, arrayOopDesc::length_offset_in_bytes());
__ sltu(AT, index, AT);
__ bne(AT, ZERO, ok);
__ delayed()->nop();

//throw_ArrayIndexOutOfBoundsException assume abberrant index in A2
if (A2!=index) __ move(A2, index);
__ jmp(Interpreter::_throw_ArrayIndexOutOfBoundsException_entry);
__ delayed()->nop();
__ bind(ok);

__ shl(FSR, 2);
__ add(FSR, SSR, FSR);

__ lw(FSR, FSR, arrayOopDesc::base_offset_in_bytes(T_INT));
```

图 5.3　优化前 iaload 数组访问实现

在该实现中，使用一条访存指令和一条比较指令来进行索引值的越界检查，如果索引值合法，再进行普通的访存操作。这样每次数组访问都需要两条访存指令和一条比较指令，开销很大，由于 Java 程序的数组访问很频繁，这必然会影响程序的性能。

引入有界访存指令之后，我们在 HotSpot 虚拟机中使用有界访存指令实现数组访存

操作，改进后 iaload 字节码的实现如图 5.4 所示。

```
__ gslwle(FSR, AT, T1);
```

图 5.4 优化后 iaload 数组访问实现

由图 5.4 可见，使用有界访存指令优化后，只用一条本地指令就可以实现一次数组访存操作，比优化前的实现节省了一条访存操作和一条比较操作。

5.4.2 上下文切换的软硬件优化

使用新型的 128 位访存指令，一次访存操作可以存取 128 位宽度的数据，在 32 位情况下，同时可以对 4 个整点或者浮点寄存器进行操作。用该指令代替原来虚拟机中保存 Java 栈环境使用的普通按字访存指令，在 Java 虚拟机保存栈环境时，可以大大减少访存的次数，提高上下文切换的效率。

对于编译后代码的访存操作，Java 虚拟机使用内存拷贝 memcpy 处理。memcpy 内部的实现根据处理数据宽度的不同可以按字节、半字、字和双字分别进行，相应的 MIPS 访存指令是 lb、lh、lw 和 ld。在扩展了访存宽度之后，我们将 memcpy 使用 128 位访存指令 gslq 或者 gssq 指令实现。这样，原来实现中的双字访存替换成了四字访存，访存操作是原来的一半，减小了访存开销。内存拷贝 memcpy 改进前后的内部实现如图 5.5。

另外，需要注意的是 MIPS 访存指令是对齐访问的，使用 128 位访存的同时，还需要使用普通访存指令处理操作数不按 16 字节对齐的情况，以免出现访问地址错例外。

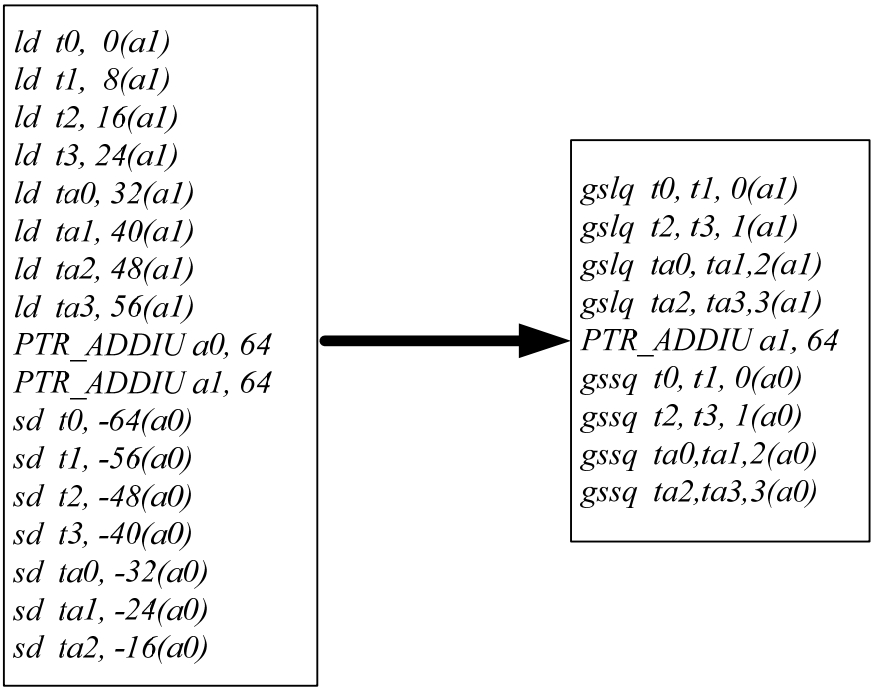

图 5.5 memcpy 改进前后的内部实现

5.5 实验结果和分析

本研究的目的在于使用软硬件协同的方法，优化 Java 虚拟机中的访存操作，优化 Java 程序中数组越界检查机制和上下文切换机制。我们设计实现了有界访存和 128 位访存指令，使用新型访存指令重新实现 Java HotSpot 虚拟机中的数组访问和内存拷贝，加速 Java 虚拟机与运行时环境的交互。

我们所有的测试都在龙芯 3 号处理器硬件平台上进行。在龙芯 3 号处理器中我们实现了有界访存指令和 128 位访存指令。

对于数组越界检查的优化，我们选取 SPECjvm2008 和 SciMark2.0 两个测试套件。其中 SPECjvm2008 是 Java 虚拟机的基准测试，该测试中数组访问占的比例较少；SciMark 2.0 中的程序都是科学计算类程序，该测试中数组访问占的比例相对较大。

使用有界访存指令越界检查机制的加速比取决于程序的行为，如果程序中最频繁执行的方法中数组访问所占比例很低，那么加速比就会很小；如果程序核心循环中数组访问所占比例很高，那么程序就会获得很大的加速比，使用我们的方法才能更有效地提高程序的性能。

SciMark2.0 的测试结果如图 5.6 所示，横轴两条立柱分别代表改进前后程序的性能值，纵坐标是各程序的分值。

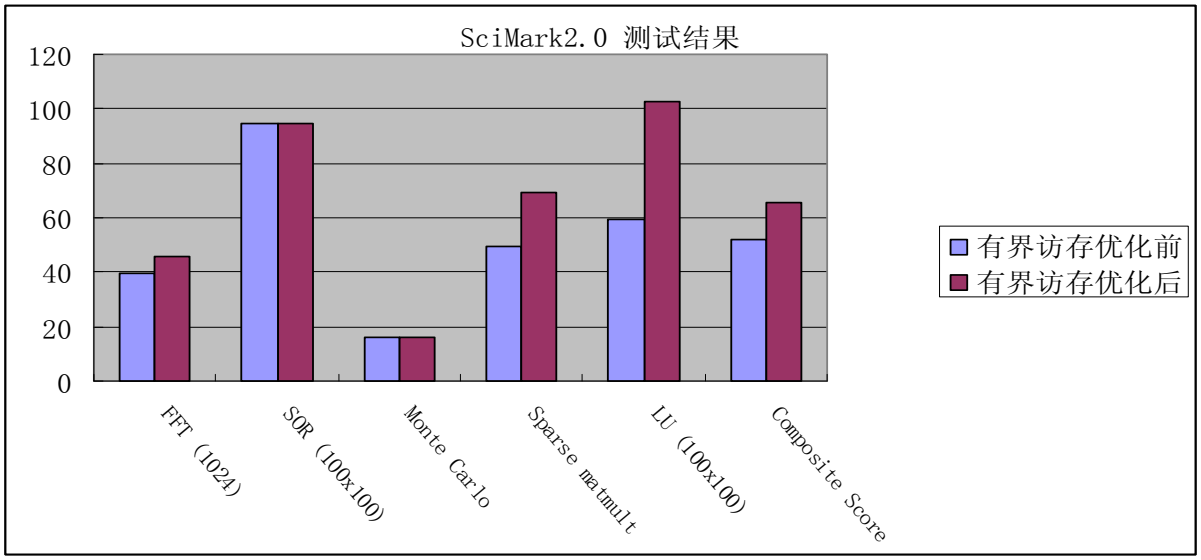

图 5.6 数组越界检查优化前后 SciMark2.0 性能对比

从图中我们可以看出，SciMark2.0 的 5 个程序（composite score 为综合分值）中有 3 个都获得了很大的加速比，其中性能提升最大的 LU 程序达到了 73%，其它 SMM 程序和 FFT 程序分别获得了 39%和 16%的性能提升，这是因为 SciMark2.0 这类科学计算程序中代码使用数组的比例很大，而且数组访问往往都存在于执行频度很高的方法中，所以该测试中大部分程序都获得了较高的加速比。SciMark2.0 程序在优化前后平均获得了 27%的性能提升。

图 5.7 是 SPECjvm2008 的测试结果，纵坐标是 SPECjvm2008 各程序的分值。

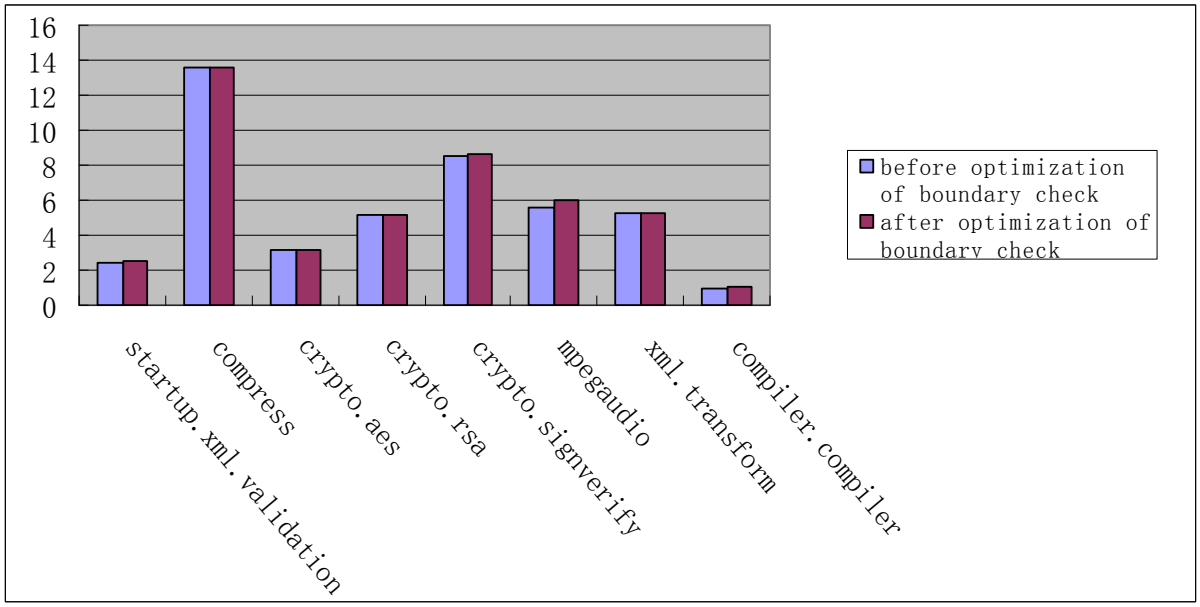

图 5.7 数组越界检查优化前后 SPECjvm2008 性能对比

从图中我们可以看出，在 SPECjvm2008 程序中，只有 mpegaudio 程序性能提升达到了 9%，这是因为 mpegaudio 程序使用数组进行了大量运算，例如离散余弦变换等；而其它程序由于数组使用的比例较小，因此性能提高不是很明显。综合来说，SPECjvm2008 程序在优化后获得了平均 1.5%的性能提升。

对于上下文切换优化，我们选取 SPECjvm2008 程序，测试通用程序运行情况下虚拟机的性能变化。测试结果如图 5.8 所示。

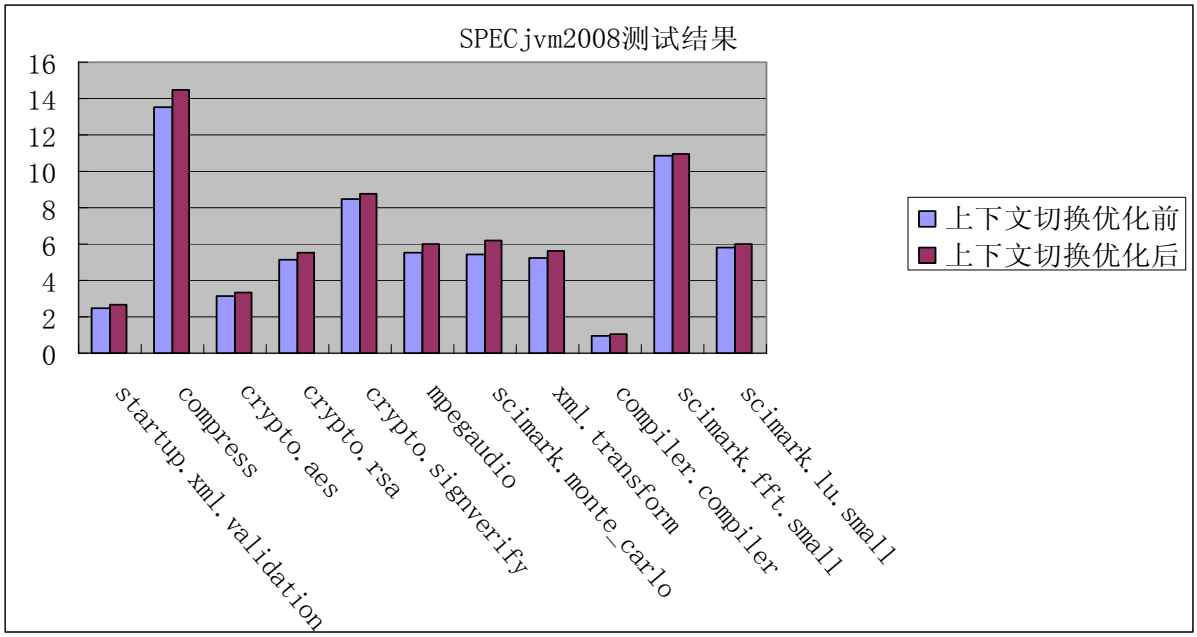

图 5.8　上下文切换优化前后 SPECjvm2008 性能对比

在 Java 虚拟机中，我们主要针对栈环境保存恢复以及生成代码的内存拷贝两方面进行的优化，使用 128 位访存指令代替原来的访存指令，旨在减少内存访问次数，削减因为访存而带来的指令停顿。从图中我们可以看出，SPECjvm2008 中各程序在优化前后表现的行为不太一致，mpegaudio 和 scimark.monte_carlo 两个程序在优化后性能提升较大，达到了 9%和 13%，这主要是因为这两个程序数据量很大，进行上下文切换的次数频繁，并且程序的重用性不高，需要大量的编译并拷贝的过程，因此经过 128 位访存优化之后，性能得到了显著提升。scimark.fft.small 和 scimark.lu.small 两个程序性能提高很小，这是因为 128 位访存优化内存拷贝，需要 16 字节对齐的内存访问，而这两个程序代码量小，很难能够出现生成代码 16 位对齐的情况，内存拷贝还是按字节访问，因此性能没有提升。综合来说，SPECjvm2008 程序在优化前后平均获得了 5.9%的性能提升。

5.6 本章小结

访存操作是 Java 虚拟机编译代码与运行时数据区交互的桥梁。Java 语言通过数组越界检查机制保证数组访问的正确性，而数组越界检查机制开销较大；另外 Java 虚拟机中因为上下文切换而产生的内存拷贝很多，普通内存拷贝的效率低下。这两方面严重影响了程序的性能。本文设计并实现了有界访存和 128 位访存指令，提出了一种软硬件结合的访存优化方法，有效地降低了数组越界检查的开销，提高了内存拷贝的效率，优化了 Java 程序的整体性能。使用有界访存指令，科学计算类测试 SciMark2.0 获得了 27%的性能提升，通用测试 SPECjvm2008 获得了 1.5%的性能提升；使用 128 位访存指令，SPECjvm2008 性能平均提升 5.9% 。

表 5.1　有界访存指令

指令格式	功能描述
gsLBLE	带上边界条件的取字节
gsLBGT	带下边界条件的取字节
gsLHLE	带上边界条件的取半字
gsLHGT	带下边界条件的取半字
gsLWLE	带上边界条件的取字
gsLWGT	带下边界条件的取字
gsLDLE	带上边界条件的取双字
gsLDGT	带下边界条件的取双字
gsSBLE	带上边界条件的存字节
gsSBGT	带下边界条件的存字节
gsSHLE	带上边界条件的存半字
gsSHGT	带下边界条件的存半字
gsSWLE	带上边界条件的存字
gsSWGT	带下边界条件的存字
gsSDLE	带上边界条件的存双字
gsSDGT	带下边界条件的存双字
gsLWLEC1	带上边界条件的取单精度浮点数
gsLWGTC1	带下边界条件的取单精度浮点数
gsLDLEC1	带上边界条件的取双精度浮点数
gsLDGTC1	带下边界条件的取双精度浮点数
gsSWLEC1	带上边界条件的存单精度浮点数
gsSWGTC1	带下边界条件的存单精度浮点数
gsSDLEC1	带上边界条件的存双精度浮点数
gsSDGTC1	带下边界条件的存双精度浮点数

表 5.2 地址比较指令

指令格式	功能描述
gsLE rs, rt	上边界比较
gsGT rs, rt	下边界比较

表 5.3 128 位访存指令

指令格式	功能描述
gsLQ	取 128 位整点数
gsLQC1	取 128 位浮点数
gsSQ	存 128 位整点数
gsSQC1	存 128 位浮点数

第六章　　执行引擎编译器系统的设计和优化

6.1 引言

执行引擎是 Java 虚拟机实现的核心，被装载的字节码由执行引擎编译成本地代码并执行。传统执行引擎的实现是解释器和即时编译器（Just-in-time compiler），其中解释器一次性解释字节码，这种执行引擎启动开销最小，它对待执行的字节码单条解释并执行；在即时编译器中，第一次被执行的字节码会被编译成本地机器代码，编译出的本地机器代码会被缓存，方法下次被调用的时候可以重用，这种方法可以利用代码的局部性，达到一次编译多次执行的效果，并且配合对编译后代码的优化，达到较高的性能，但缺点是启动开销较大，对于小规模单次执行的代码不适用。

Java 虚拟机即时编译器是以 Java 方法为单位进行编译的，当一个 Java 方法被载入之后，即时编译器首先将该方法编译成中间层表示，然后由中间层表示再转换成本地可执行代码。引入中间层表示的原因是中间层表示含有很多运行时信息，因此在中间层表示可以使用这些丰富的运行时信息进行优化。在中间层表示优化结束并进一步编译成可执行代码之后，执行引擎会在内存中新分配一块区域，然后把新生成的可执行代码拷贝至该区域中。当程序执行到该方法时，不需要重新编译这个方法，而是从存放生成代码的区域中取出代码并执行。

现代处理器内存管理系统引入 cache 后，大大缩小了内存和处理器之间速度的差距，提高了计算机系统的整体性能。方法的可执行代码生成时，处理器经过数据 cache 放入内存中，当执行到该代码段时，处理器需要从包含该代码段的内存区域通过指令 cache 取指令并执行。

在这个过程中，由于 cache 的存在，大大提高了处理器访存的速度。处理器从内存取指令时，如果该内存区域在数据 cache 中已经建立映射，那么就不需要从内存中取数据，而是直接从数据 cache 中读取数据，读数据的性能就会有大幅度的提高。写数据也是同样的情况。

但是，处理器的数据 cache 空间有限，其容量对数据的存取有很大的制约，读写内存数据时，如果不能在 cache 中命中，会发生 cache 失效，需要从主存中读写数据，这进一步增大了访存的开销。由于即时编译器以方法为单位编译 Java 字节码，编译好的所有方法的可执行代码所占空间很大，这么大的数据不能一直存在于数据 cache 中不被替换出去，因此下次执行时数据访问不能在 cache 中命中，访存的开销很大，制约了编译器的性能。

为了提高 Java 虚拟机执行引擎的性能，需要设计一种高效的编译代码执行代码的机制，让生成代码在运行时，能够保存在数据 cache 中不被替换，cache 锁技术是近年来出

现的根据程序局部性优化 cache 性能的技术，现在的 cache 锁技术大多用于嵌入式系统，并且作用于一级指令 cache，不能满足即时编译器对二级 cache 访问的需要。鉴于上述需求，本文在多核通用处理器系统中实现了二级 cache 锁机制，并配合软件算法，设计了一个软硬件结合的即时编译器系统。

本章第二部分介绍了 cache 锁机制的相关研究；第三部分使用一个例子对 cache 锁机制提高程序性能进行了分析；第四部分分析了即时编译器访存行为，引出了优化 cache 访存的必要性；第五部分描述了在通用处理器中二级 cache 锁机制的硬件设计；第六部分介绍使用 cache 锁机制设计的即时编译器；第七部分进行实验评估和分析；第八部分总结本章。

6.2 相关工作

Java 虚拟机执行引擎的性能在很大程度上取决于计算机存储系统的性能。cache 结构的出现缩小了处理器和内存访问速度的差距，很大程度上提高了处理器访存的效率。理想情况下，所有待访问的数据全部存放于 cache 中，访存发生时处理器直接从 cache 中读写数据，不访问主存储器，这种理想情况能够最大限度发挥处理器的能力。但是由于 cache 的容量有限，需要访问的数据都存放在 cache 中不现实，而如果访存操作不能在 cache 中命中，处理 cache 失效的开销将会很大，当失效频繁发生时，就会严重制约存储系统的性能。

为了提高存储系统的性能，近年来出现了很多存储优化的技术。广义上来说，这些技术可以分为两大类：第一，片上高速存储系统，与 cache 并行处理访存操作；第二，传统 cache 结构的优化。

第一类技术包含 SPM（可暂存存储器）或者循环 cache 等技术。片上 Scratchpad Memory （SPM）[90,91,92]是处理器中可以独立寻址的高速片上存储器，SPM 一般见于低端的嵌入式处理器，比如 Freescale 公司的 Mcore 和 68HC12 处理器、TI 公司的 TMS370Cx 以及 ARM 公司的 Cortex-M3 处理器。在通常的实现中，SPM 与处理器 cache 处于同一处理器层次，处理器的访问会同时查询 cache 和 SPM。由于 SPM 没有 cache 必须的 Tag 比较过程，其芯片面积和功耗相对 cache 有比较大的优势。循环 cache[93]是一个可以预取频繁执行循环的缓存，将频繁执行的循环预取之后，就能在执行时大大缩短执行时间。这两项技术都是用在程序的运行时行为可预知的系统或者实时系统中；第二类技术包括修改硬件 cache 结构、修改 cache 替换算法等，来提升存储系统的性能。Rudolph et al 提出了列缓存技术[94]；Wang et al 将每个 cache 块增加 evict-me 和 kill-me 两个标志位[95]，编译器收集程序运行时信息来决定这两位的值，从而 cache 块可以根据程序的行为的选择更合适的替换策略。这种硬件的方法虽然效果较好，但是需要大规模修改 cache 的结构，实现上有一定难度。

基于上述技术的优缺点，对 cache 特征的研究也越来越广泛，cache 锁技术就是其中

一个非常有代表性的技术。cache 锁技术最初出现在多任务的实时操作系统中，在多任务实时系统中，实时性和确定性是非常重要的特征，在这种系统中使用 cache 结构，一旦 cache 失效，就会增加系统的不确定性，无法估计系统最长响应时间（WCET），出现不可预料的后果。从这点出发，出现了指令 cache 锁技术，用于解决嵌入式多任务实时系统中确定性问题以及缩短系统中程序响应的最长路径。

在多任务实时系统中，cache 带来的不确定性主要表现在两方面：首先，对于最长响应时间（WCET），由于同一个任务内的指令可能在 cache 中相互替换，导致了每条指令执行时间不是固定的，这种干扰在任务内部发生，称为内部干扰；其次，在抢占式系统中，不同的任务之间的指令可能相互替换，增长了被抢占任务的执行时间，这种干扰在不同的任务之间出现，称为外部干扰。在这类系统中，cache 锁技术的出现大幅度缓解了不确定性，将频繁执行的代码段对应的指令 cache 块锁在 cache 中不被替换，这样就可以消除前面提到的内部和外部干扰。系统设计者可以方便地估计任务执行的时间。

cache 性能的高低取决于程序运行时的行为，为了达到较高的性能，cache 锁机制预取和锁住的指令需要基于一定的算法决定，基于不同的算法，cache 锁机制分为两类：静态锁机制和动态锁机制。在静态锁机制框架中，cache 的内容在系统启动时进行预取，并且在系统的整个运行生命周期一直保持在 cache 中不改变；在动态锁机制框架中，cache 的内容在系统运行期间可以发生改变，但也只是在特定的位置，比如当一个任务被其他任务抢占时。下面简要介绍这两种框架的基本技术。

6.2.1 静态 cache 锁机制

静态 cache 锁机制[96,97,98]在程序运行期间进行一次锁操作，不因为任务被抢占而解锁。系统启动的时候，程序中的循环操作被预取入 cache 中，当把循环预取完成之后，cache 将对应的 cache 块上锁，然后再预取下一个循环，预取的代码可以分别属于不同的任务，锁住的 cache 块可以是大段连续的 cache 块，或者是小段零散分布的 cache 块。当程序执行到该段代码时，可以直接从指令 cache 中取指执行，锁住的指令 cache 块不会被替换出 cache。这样就避免了任务内和任务间指令的干扰。

许多处理器都已经实现了指令锁 cache 的功能，像 Intel-960、x86 系列的很多处理器、Motorola MPC7400、79R4650 和 79RC64574 等等。不同处理器实现的 cache 锁机制不同，例如：可以对全部 cache 上锁、只对部分 cache 上锁或者对某一路 cache 上锁等等。

多任务实时系统的 cache 锁机制有两个重要的目标：系统的确定性和程序反应时间的优化。随机选择 cache 中上锁的内容可以实现系统的确定性，却不能兼顾任务的反应时间。为了尽量权衡两方面的情况，需要根据运行时的情况选择锁在 cache 中的内容。对于选择何种内容进行 cache 上锁，业界有很多的算法，主要根据两方面的内容：第一，程序执行的核心循环，核心循环的性能决定整个程序的性能，程序启动时将核心循环取入 cache 中并对相应的 cache 块上锁，可以很大程度上提高程序的性能；第二，对于程

序很少执行路径中的指令，这些指令可能位于 WCET 路径上，对于这些指令的选择不会缩短 WCET 和任务反应时间，因此这类指令尽量不应该锁在 cache 中。

静态 cache 锁框架虽然对程序的确定性有很大的改善，但是由于整个任务运行期间 cache 锁不会改变，因此在可抢占的实时系统中，如果锁在 cache 中的任务执行优先级改变，被其它的任务抢占，而 cache 块仍然被占用，这势必影响其它任务的性能。

6.2.2 动态 cahce 锁机制

动态 cache 锁机制[99,100]最大的特点在于它将一个任务划分成多个不同的部分，每个部分都可以进行独立的 cache 锁操作，这是动态 cache 锁机制和静态 cache 锁机制最大的区别。如果一个任务只是划分成一个部分，那么动态 cache 锁机制跟静态 cache 锁机制等价。动态 cache 锁机制有很多独有的特点：它可以提高多任务实时系统的最坏执行时间和平均响应时间；使用动态锁机制，可以方便地与体系结构中其他的部分进行交互并分析，例如分支预测等；另外，动态 cache 锁机制可以进行系统的混合设计以及软硬件的协同设计。

动态 cache 锁机制主要有两部分的工作，第一部分是使用剖析的方法，确定程序每条路径的执行频率。程序执行路径的确定遵循两个条件，首先是执行路径包含尽量多的基本块，其次执行路径尽量不要有交叉，数量尽量少，这样统计出基本块运行时的执行频率；第二部分是对程序进行区域划分。首先每个基本块作为一个区域，然后使用贪心算法，形成最终的区域，算法的每一步进行基本块的合并，合并的目标是尽量减少程序的最长执行时间。

动态方法比静态方法更加灵活，在保证确定性的情况下，对任务的执行时间有了更多优化。

上述两种框架在多任务实时系统中程序运行时都达到了较高的确定性和运行时性能，但这两种方法的侧重点有所不同。在静态框架中，所有的任务竞争 cache 空间，所有任务在同一时刻可以向 cache 中取数据，一旦数据取入 cache 中，就会将对应的 cache 块锁住，并且在运行时不被替换出 cache；在动态框架中，每一个任务都使用全部的 cache 空间，当一个任务开始执行或者在抢占恢复后，那么该任务内的指令就独占 cache 空间并锁住相应的 cache 块，在该任务运行期间，cache 的块不被替换。由于锁机制的使用，指令是否在 cache 中是可预知的，因此无论是静态框架还是动态框架，指令的运行时间都是确定的。

使用 cache 锁框架之后，解决了多任务实时系统 cache 引起的相关问题。由于指令的执行时间是确定的，因此最长执行时间也可以方便地估计出来。很多相关的技术都可以使用，例如可执行路径的估计，基于 cache 响应时间的调度算法分析等等。对于预取并锁入 cache 的指令，根据通用的算法[101]，该算法通过评估 cache 响应时间来决定各

任务内指令的预取和调度算法。

综合来说，cache 锁机制的静态框架和动态框架具有各自的特点，静态框架侧重于系统的确定性和可预测性，而动态框架能够更好地优化程序的性能，并且这两项技术都用于嵌入式实时系统的指令 cache 中。

6.3 cache lock 机制原理简介

本小节使用一个简单的例子介绍 cache 锁机制理想情况下带来系统性能的提高。图 6.1 分别是程序的加权控制流图（6.1(a)）、程序执行轨迹（6.1(d)）、内存中的分布（6.1(b)）以及 cache 中的映射（6.1(c)）。

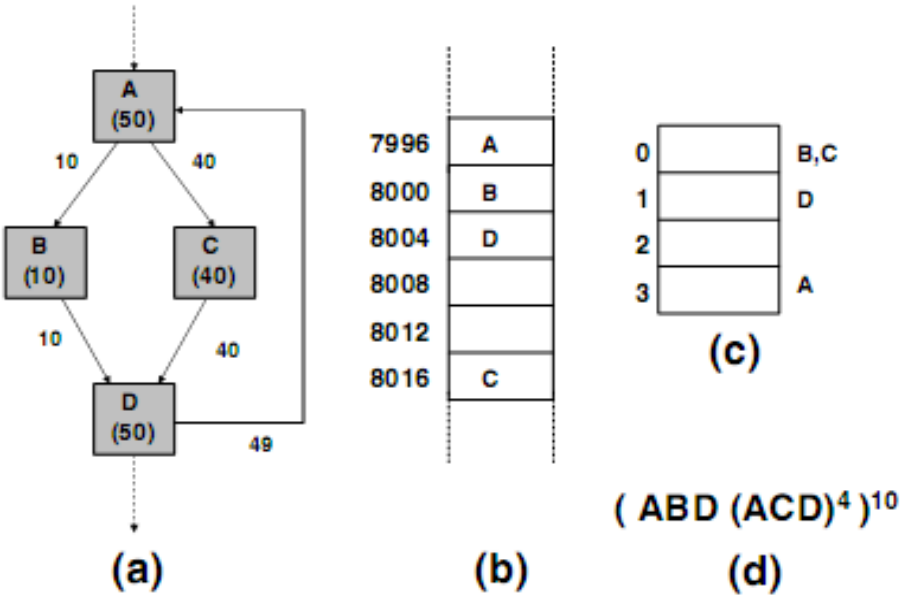

图 6.1 示例程序控制流图及内存 cache 分布

控制流图中的节点和边标明了程序运行时剖析得到的每部分的执行频率。根据程序的执行轨迹，节点 B 的一次执行后面跟着节点 C 的四次执行，而这个序列一共进行十次循环。为了简化起见，A、B、C、D 每个节点代表一条指令，每条指令 4 个字节。我们假设 cache 容量是 16 字节，内存映射策略是直接映射，每个 cache 块一个字。根据以下公式，内存地址映射到 cache 块中。

$$Set = (addr)\ mod\ \frac{Cache\text{-}Size}{Associativity * Words\text{-}Per\text{-}Line}$$

根据上述映射函数和程序在内存中的分布情况，节点 B 和 C 映射到同一个 cache 块中。该程序在运行期间，节点 B 和节点 C 不停地进行替换，导致了大量的 cache 失效。表 6.1 显示的是程序运行期间每个节点 cache 失效的次数以及 cache 失效的总次数，第二列表明这个小程序导致了一共 22 次 cache 失效，性能损失严重。

表 6.1　使用锁机制前后 cache 失效对比

节点	没有锁机制的 cache 失效次数	使用锁机制的 cache 失效次数
A	1	1
B	10	10
C	10	1
D	1	1
总和	22	13

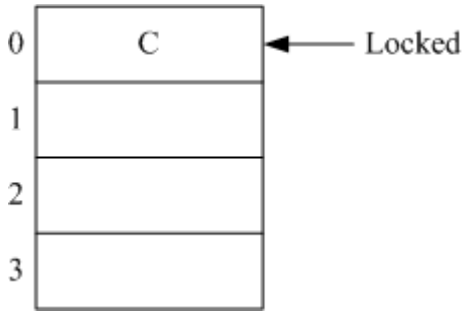

图 6.2　使用锁机制之后的 cache 分布

接下来，假设该指令 cache 具有锁 cache 机制，如果将节点 C 锁在 cache 块 0，那么在程序执行期间节点 C 就不会被节点 B 替换。在这种情况下，节点 C 只会出现一次强制失效而节点 B 的 cache 失效次数不变。表 6.1 的第三列显示对节点 C 使用锁 cache 后 cache 失效的数目，可以看出，在这个小程序的运行过程中，使用 cache 锁机制后，cache 失效的总数从 22 次降低到了 13 次。这个例子可以明显地表明 cache 锁机制可以大幅度降低 cache 的失效次数。

6.4 Java 虚拟机即时编译器访存行为分析

Java HotSpot 虚拟机执行引擎使用解释器和即时编译器融合的自适应优化技术。自适应优化器搜集那些运行时才有效的信息，试图以某种方式把字节码解释和编译结合起来，利用解释器开销小，即时编译器效率高的优点，以得到最优化的性能。自适应优化器开始的时候对所有的代码都是解释执行，同时监视代码的执行情况，大多数程序花费 80%~90%的时间执行 10%~20%的代码。因为可以监视程序的执行情况，所以虚拟机可

以识别哪些方法是程序的“热区”。当自适应优化器判断出某个特定的方法是热点方法后，它启动一个后台线程，把字节码编译成本地代码，并且尽可能优化这些本地代码。自适应优化技术使程序最终把原来占 80%~90%运行时间的代码变为更加优化的、静态链接的本地代码，提高了程序的性能。当热点方法不再是热点后，取消那些编译过的代码，重新解释执行那些字节码。

在执行引擎自适应解释的过程中，即时编译器性能的高低是能否编译并执行高效代码的关键。本小节分析即时编译器的工作过程，寻找可以优化的部分。

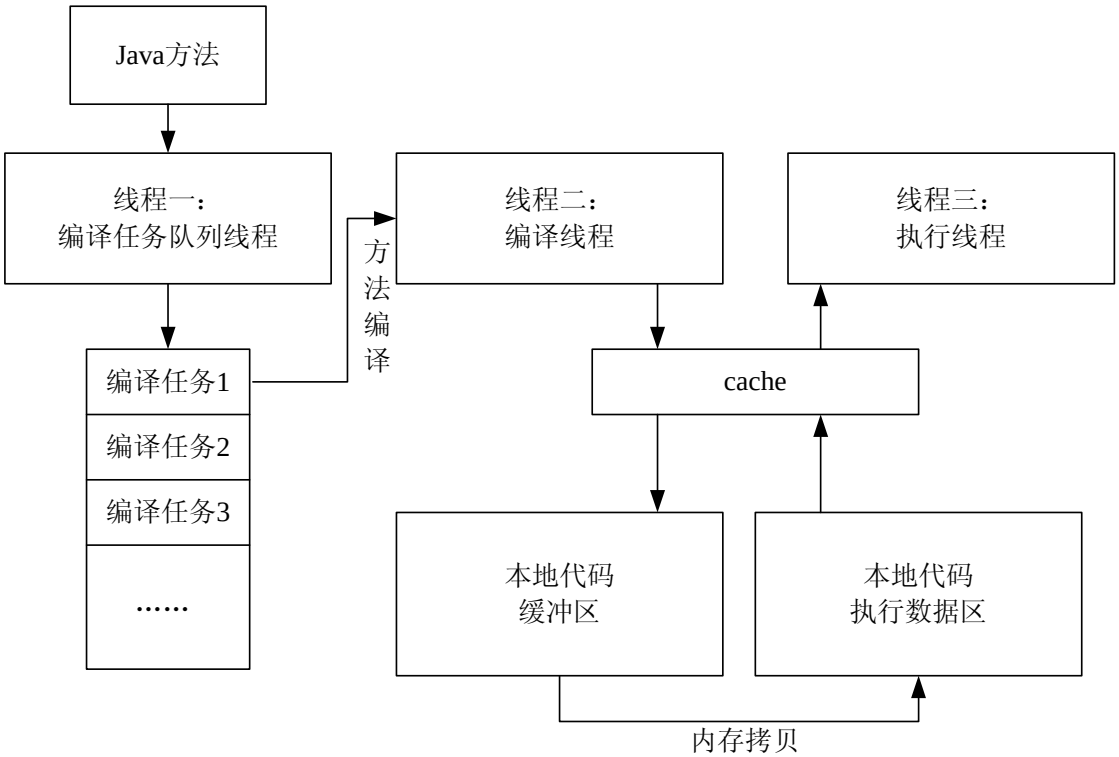

图 6.3 即时编译器工作过程

即时编译器工作过程如图 6.3 所示。Java 虚拟机初始化时，会创建即时编译线程。即时编译线程的主程序执行一个循环，该循环监视虚拟机中的编译任务队列，当编译任务队列不为空时，循环体从编译任务队列中取出一个待编译的任务，然后将其编译成本地代码。编译任务队列中存放待编译的任务，Java 虚拟机进行监视，如果 Java 方法被调用的次数超过一定阀值，或者某个往回跳转的次数超过一定阀值，则触发一个编译任务的产生，将该编译任务放入编译任务队列中。当编译任务队列不为空时，编译线程从队列中取出待编译方法，并将其编译为本地代码，然后将编译好的本地代码存放入虚拟机专门的数据缓冲区。总的来说，即时编译器使用双线程完成编译过程，一个线程负责监视方法执行频度，添加编译任务，另外一个线程负责真正编译方法，这种方式能够高效地完成编译任务，避免执行引擎的空等。

Java 方法被编译成本地代码之后，在执行前需要有一个代码安装的过程。虚拟机需要将编译出来的本地代码包装成一个统一的接口，然后将本地代码从存放编译好代码的数据区拷贝到执行代码区，当程序执行到该方法时，执行线程从执行代码区取指令并执行。

图 6.4 显示了 Java 方法经过编译器编译后，生成代码的大小，即时编译器以方法为单位进行编译，如果将一个方法经过编译后的生成代码称为方法生成代码，图中所显示的生成代码是 SPECjvm2008 测试中程序在执行过程中所有方法生成代码的总和，单位为兆字节。

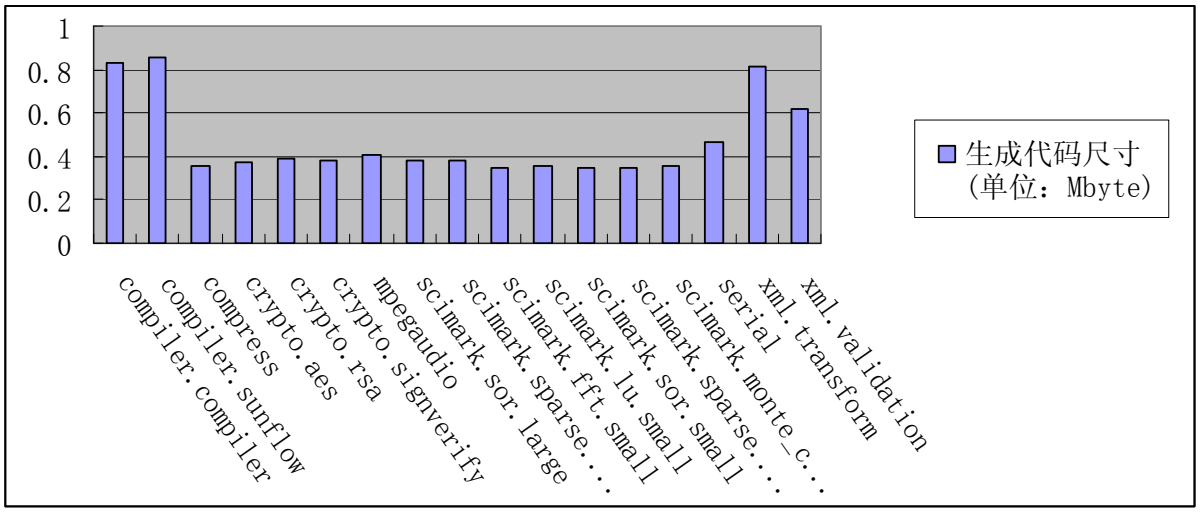

图 6.4　即时编译器生成代码尺寸统计

从上图可以看出，SPECjvm2008 主要测试程序方法生成代码尺寸的总和平均 0.6 兆字节左右，有几个程序接近 1 兆字节。理想的情况是所有生成的代码始终存在于数据 cache 中，这样处理器再次读写数据时不需要访问速度较慢的主存储器。但是理想情况不可能实现，因为：第一， cache 容量有限，由统计结果来看，生成代码的大小相对 cache 来说普遍较大；第二，Java 虚拟机运行过程中，操作系统以及其它程序频繁使用数据 cache，虚拟机本身能够使用的 cache 块相对有限；第三，Java 虚拟机是多线程的程序，在运行过程中同时有六至七个线程同时工作，这样虚拟机本身也占用了大量的数据 cache，可供生成代码使用的数据 cache 进一步减少。

由上述几点分析可知，方法生成代码在执行时很有可能已经被替换出二级 cache 块，这样处理器只能从主存中取指令并执行。即时编译器经过多年的研究和实践，现在编译的过程已经有多种自适应算法并且效率较高。在现代处理器系统中，内存的性能相对于处理器的处理速度来说是瓶颈，生成的代码在执行的过程中，大量的访存操作势必限制编译器的性能。

为了让生成代码在执行时在二级 cache 中不被替换，本文设计了一种软硬件结合的

即时编译方法，通过对二级 cache 增加锁的方法，对 Java 程序的方法生成代码段在二级 cache 中上锁，使该 cache 块不被替换出去，减小了即时编译器内存访问的开销，提高了 Java 虚拟机执行引擎的性能。

6.5 cache lock 机制硬件设计

我们基于龙芯 3 号处理器设计 cache 锁机制。龙芯 3 号使用了三种独立的 cache：一级指令 cache：共 64KB，cache 行大小 32 字节，采用四路组相联的结构；一级数据 cache：共 64KB，cache 行大小 32 字节，采用四路组相联的结构，采用写回策略；二级混合 cache：片上 cache，通过 128 位 AXI 总线和处理器核通讯，共有 4 个二级 cache 模块。每个二级 cache 模块全局编址，cache 行大小 32 字节，1M 容量，采用四路组相联的结构，采用写回策略。

龙芯 3 号中包括了 4 个片上二级 cache 模块。每个二级 cache 模块的容量为 1MB，共 4MB。每个 cache 行大小为 32 字节。二级 Cache 模块的主要特征包括：采用 128 位 AXI 接口，四路组相连，8 项 cache 访问队列，关键字优先，接收读失效请求到返回数据最快 8 拍，通过目录支持 cache 一致性协议，可用于片上多核结构（也可直接和单处理器 IP 对接），软 IP 级可配置二级 cache 模块的大小（512KB/1MB），采用四路组相联结构，运行时可动态关闭，支持 ECC 校验，支持 DMA 一致性读写和预取读，支持 16 种二级 Cache 散列方式。表 6.2 给出了龙芯 3 号处理器 cache 结构的一些参数。

表 6.2 龙芯 3 号 cache 参数

参数	指令 Cache	数据 Cache	二级 Cache
Cache 大小	64KB	64KB	1MB（共 4MB）
相联度	4 路组相联	4 路组相联	4 路组相联
替换策略	随机法	随机法	随机法（可锁定）
块大小(line size)	32 字节	32 字节	32 字节
索引(Index)	虚地址 13:5 位	虚地址 13:5 位	物理地址 17:5 位
标志(Tag)	物理地址 47:12 位	物理地址 47:12 位	物理地址 47:12 位
写策略	不可写	写回法	写回法
读策略	非阻塞（2 个同时）	非阻塞(24 个同时)	非阻塞（8 个同时）
读顺序	关键字优先	关键字优先	关键字优先

写顺序	不可写	顺序式	
校验手段	奇偶校验	ECC 校验	ECC 校验

由于即时编译器的生成代码是以数据的形式存放在内存中的，并且执行时需要从相应的内存位置进行访问，因此生成代码的读取与二级 cache 性能关系很大，我们在龙芯 3 号处理器的二级 cache 增加锁机制。

在二级 cache 中， cache 锁机制最多可以锁住 4 个二级 cache 区域，对应四个锁窗口，使用四组锁窗口寄存器进行控制，每组窗口寄存器由窗口有效位、窗口锁地址和窗口掩码组成。我们实现的是动态锁机制，可以对二级 cache 模块内部的四组锁窗口寄存器进行动态配置，但必须保证四路二级 cache 中一定有一路没有被锁住。每组窗口的大小可以根据掩码进行调整，但不能超过整个二级 cache 大小的 3/4。

落在被锁区域中的二级 cache 块会被锁住，因而不会被替换出二级 cache（除非四路二级 cache 中都是被锁住的块）。此外当二级 Cache 收到 DMA 写请求时，如果被写的区域在二级 cache 中命中且被锁住，那么 DMA 写将直接写入到二级 cache 而不是内存。

在二级 cache 上实现锁机制代价非常小，只需要修改 cache 替换算法，不会成为 cache 设计的关键路径，也不会改变原有的存储层次。增加 cache 锁机制的龙芯 3 号存储结构如图 6.5。

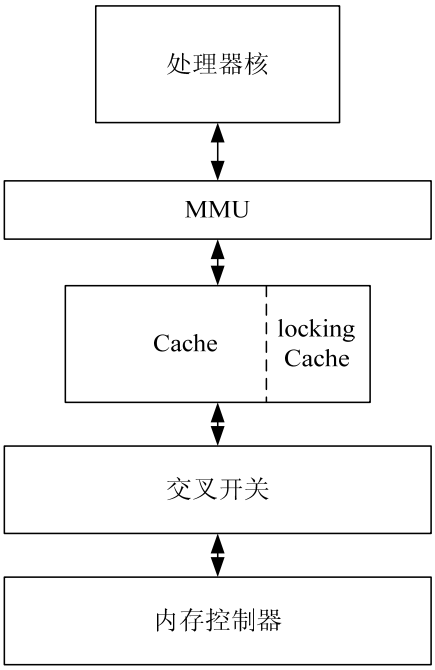

图 6.5 增加二级 cache 锁机制后处理器存储结构

6.6 软硬件协同设计 Java 虚拟机编译器系统

如上文所述，即时编译器执行代码时需要大量访问内存，而影响性能的最大因素是 cache 中数据的重用性。编译器将 Java 方法编译成本地代码后，首先保存在内存中的可执行代码区，同时在二级 cache 中保存了这部分可执行代码，当代码执行的时候，处理器从代码执行区访问数据，此时如果二级 cache 中的该段代码没有被替换，那么处理器可以直接从二级 cache 中取数据，不需要访问内存，这种情况下访存的效率较高；如果该段代码已经被替换出了二级 cache，那么处理器需要再次访问主存储器，访存性能较低。

使用我们实现的二级 cache 锁机制，可以解决程序访存局部性问题。在即时编译器中，需要设计一种算法，使用二级 cache 锁机制，降低 Java 程序生成代码编译执行时的访存开销，提高即时编译器的性能。最近的相关研究表明，cache 锁机制分为两大类：动态锁机制和静态锁机制，动态锁机制灵活多变，可以根据程序的行为实时选择 cache 块上锁；而静态锁机制虽然不如动态锁机制灵活，但是对于访存局部性很高的程序，可以达到更优的性能。选择何种算法需要根据即时编译器生成代码的行为而定。

由于即时编译器以方法为单位编译代码，我们选择 SPECjvm2008 中的程序 mpegaudio，通过动态分析，选择该程序中执行频率的前 20 位的方法，统计这些方法的生成代码尺寸，结果如图 6.6 所示：

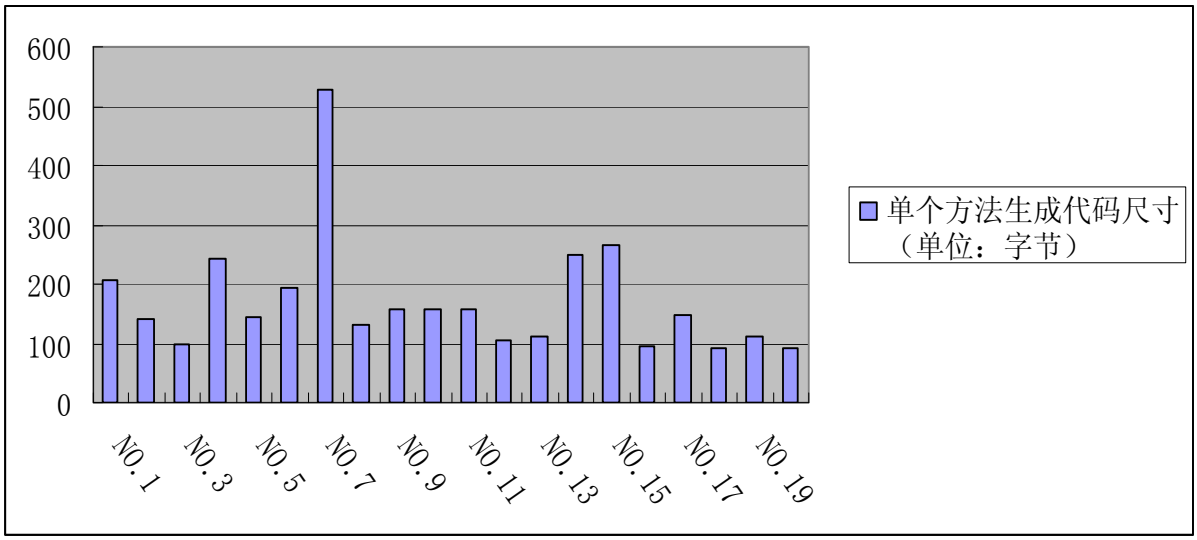

图 6.6 mpegaudio 程序中单个方法生成代码尺寸

从图中可以看出，在 mpegaudio 程序中，频繁使用的单个方法编译后生成代码尺寸平均 300 字节左右，如果使用动态的方法，需要增加运行时统计 Java 方法执行频率的剖析器，通过剖析寻找热点方法，将热点方法生成代码所占内存区域映射的 cache 块上锁。但是单个方法的生成代码相对较小，如果频繁对 cache 块上锁或者解锁，程序运行时的

开销会很大，另外运行时的剖析器的开销也会影响编译器的性能。鉴于上述考虑，动态 cache 锁机制在即时编译器中并不适用。

对于静态 cache 锁机制而言，为了评估它是否可用，我们对 mpegaudio 程序执行频率前 20 位的方法生成代码存储在内存中的首地址进行统计，如图 6.7 所示（以排名 NO.1 的方法首地址为基准，纵轴代表各方法地址与基准地址的差值，单位为 MByte）：

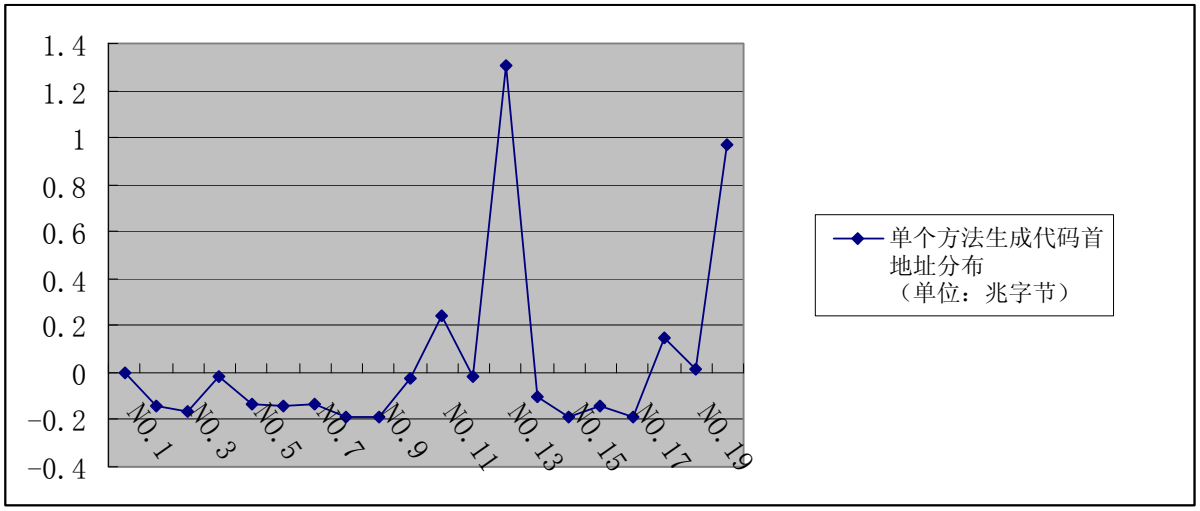

图 6.7　mpegaudio 中生成代码地址分布

执行频率前 20 位的方法，代码执行的时间占总执行时间的 80%以上。从图中可见，其生成代码在内存中的存储地址空间在 1 兆字节之内。从这点我们可以判断，即时编译器在编译方法时，生成代码的局部性很好——频繁执行的代码存储于内存中相对较小的空间。从程序的行为分析，如果将这 1 兆字节地址空间对应的 cache 块上锁，使这些方法始终存在于 cache 中不被替换，应该能够大幅度提高程序的性能。因此，我们选用静态 cache 锁的方法。

我们实现的 cache 锁机制的硬件平台龙芯 3 号处理器有 4MByte 的二级 cache，在编译器编译程序的初始阶段，从编译器中得到存储生成代码的内存地址，以该段地址为首地址，在二级 cache 中锁定 1MByte 的 cache 块，由于编译生成代码的局部性，大部分生成代码都会落到该地址窗口中，因此生成代码对应的 cache 块不会被替换，执行线程直接从 cache 中取数据，可以大大提高效率。直至程序运行结束，解锁这个地址窗口的 cache 块。使用 cache 锁机制设计的即时编译器执行流程如图 6.8。

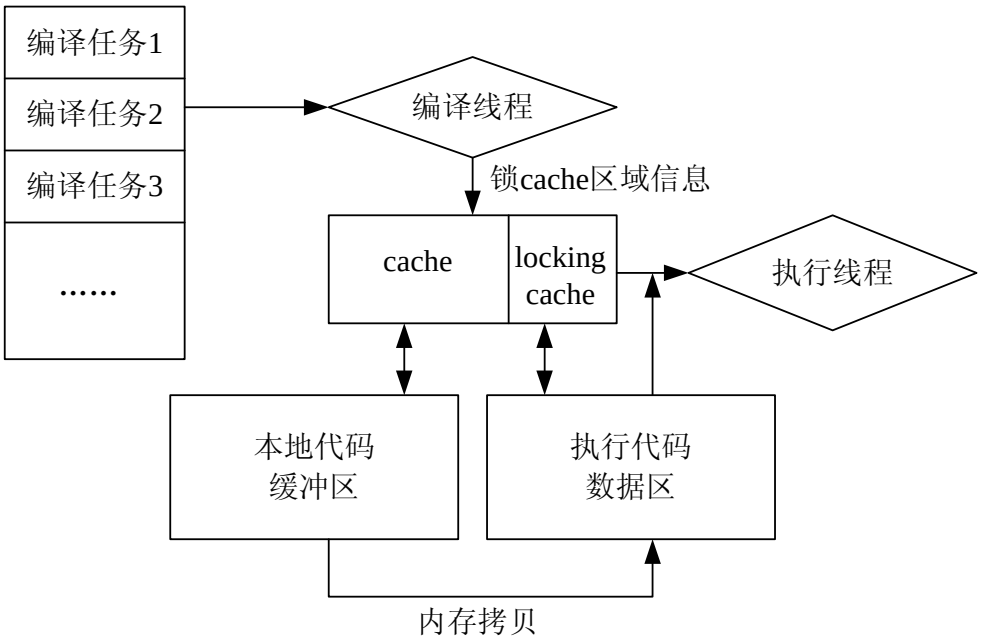

图 6.8 使用 cache 锁机制的即时编译器执行流程

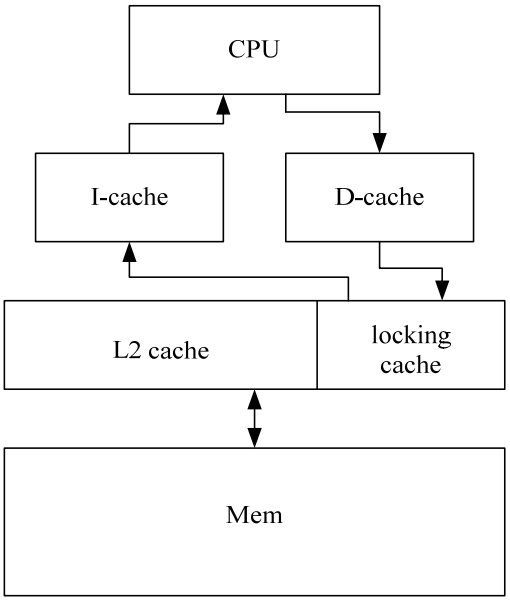

图 6.9 使用 cache 锁机制的处理器数据流

在这种新型设计中，热点方法生成的本地代码锁在数据 cache 中，执行线程从执行代码数据区取数据时，可以大大减少 cache 失效率，处理器从二级 cache 经一级指令 cache 取指执行，避免了对主存储器进行存取的次数。整个系统的执行流程如图 6.9。

6.7 实验结果和分析

我们基于龙芯 3 号处理器设计并实现了硬件数据 cache 锁机制，并使用该机制设计了一种高效的 Java 虚拟机即时编译执行方法。

评估 cache 锁方法的一个重要指标是程序执行时 cache 失效次数的变化。由于即时编译器生成代码后写内存存储代码，执行代码时对内存读操作取指令，如果二级 cache 失效，那么代码执行时处理器需要直接访问内存，所以对内存直接的读操作可以反映出代码执行时二级 cache 失效的情况。为此我们统计 SPECjvm2008 测试执行过程中处理器直接读取内存的次数，结果如图 6.10。

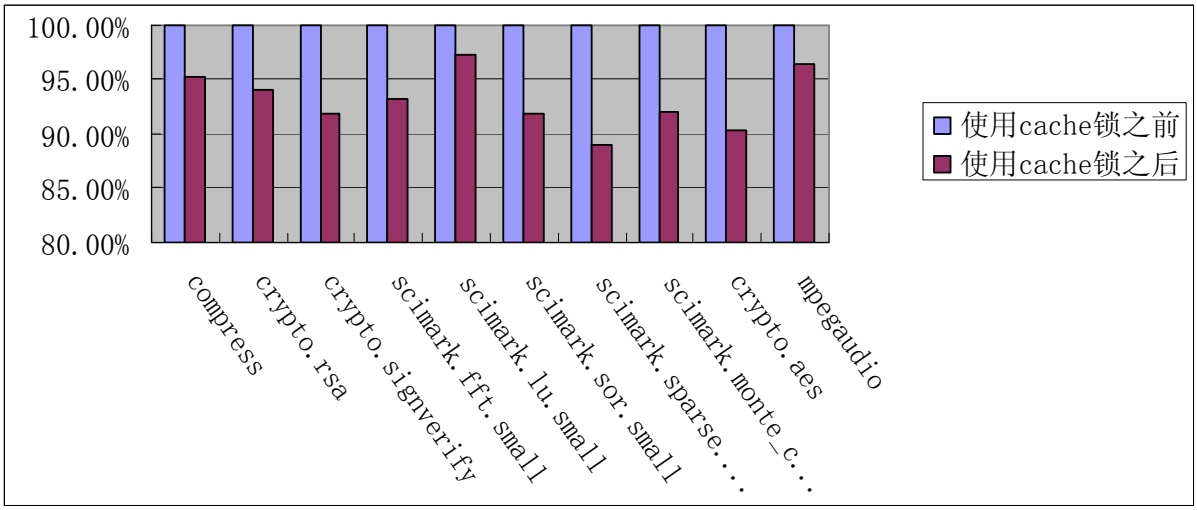

图 6.10　使用 cache 锁前后处理器直接读取内存次数比较

从图中可以看出，在使用 cache 锁机制之后，代码执行时处理器直接读内存的次数平均降低了 6.9%，因此相应的 cache 失效次数有了明显的改善。由于方法生成代码在二级 cache 中没有被替换，所以代码执行时处理器只需要从 cache 中就可以读取指令，不需要访问速度较慢的内存。

使用 SPECjvm2008 测试 cache 锁机制对程序性能的影响，结果如图 6.11。

伴随着程序执行时 cache 失效次数的降低，处理器取指令时访问主存储器的次数也大幅度减少，本地代码的执行速度随着取指访存性能的提高而增加，各测试程序的性能有了明显的提高。其中性能提高最多的 scimark.sparse.small 有 17.9%的性能提升，从图 6.10 也可以看出，scimark.sparse.small 程序 cache 失效率的降幅也是最大。SPECjvm2008 性能平均提高了 4.2% 。

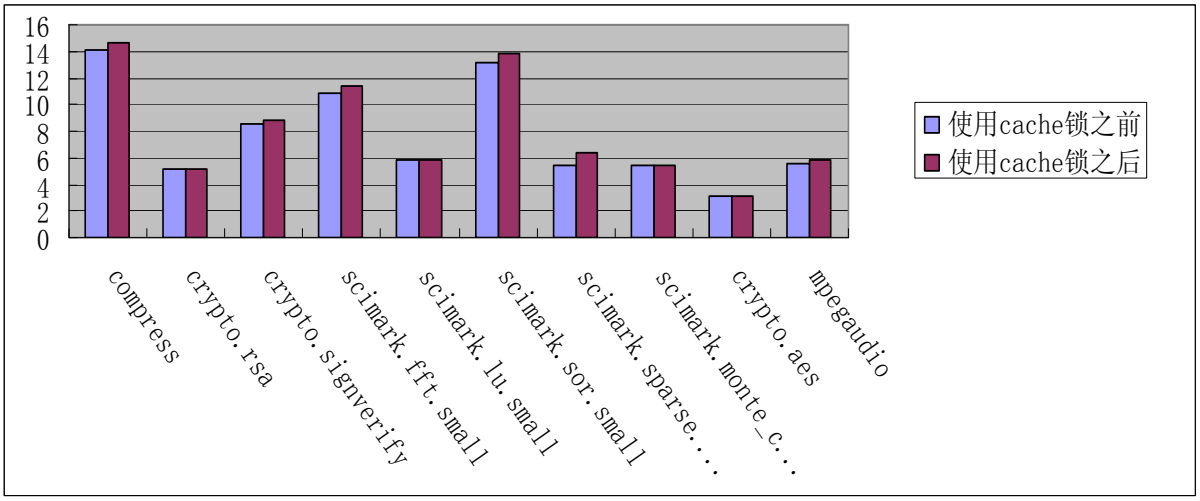

图 6.11 使用 cache 锁机制即时编译器性能测试

6.8 本章小结

本章对 Java 虚拟机即时编译器的编译行为进行了分析，并认为编译生成的大量可执行代码在 cache 中的频繁替换是即时编译器的性能瓶颈，因为当生成代码被替换出 cache 后，程序执行时处理器必须访问速度较慢的主存储器。从这一点出发，本文设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。通过该方法，生成代码执行时的 cache 失效次数降低了 6.9%，测试 SPECjvm2008 中程序最高获得了 17.9% 的性能提升，平均性能提升 4.2% 。

第七章　　结束语

7.1 引言

近年来，随着操作系统和计算机体系结构的快速发展，计算机基础平台出现了多样化的趋势，而不同平台间的应用程序不能无缝迁移成为了制约基础平台发展的主要问题，应用程序跨平台的支持也变得越来越重要。Java 技术是实现应用程序跨平台支持的重要途径，由于 Java 语言是平台无关的，Java 程序不需要移植就可以运行在多样化平台上，因此 Java 虚拟机对于以龙芯为代表的国产高性能处理器丰富软件环境具有重要的现实意义。

Java 程序的运行需要 Java 虚拟机的支持，但目前多数的 Java 虚拟机还存在效率较低的情况，目前基于纯软件和软硬件协同设计方法具有效率和硬件设计、验证复杂度等多方面的弊端，严重制约着 Java 技术的发展。为了研究上述问题，提高 Java 虚拟机的性能，促进应用软件的跨平台融合，本文在现有平台的基础上，设计和实现了一个简洁高效的 Java 虚拟机，并对该 Java 虚拟机的各组成部分进行了系统的优化。

本章节对上述设计进行了总结，并展望未来工作。

7.2 本文工作总结

本文的主要工作如下：

第一，以 Openjdk6 Java 编译环境为基础，设计实现了基于龙芯国产高性能处理器的 Java 虚拟机系统。Openjdk6 是一套完整的 Java 编译环境，它包括命令工具代码、本地方法库代码、虚拟机代码等，其中的 HotSpot 是最核心的 Java 虚拟机代码。本文以 Java HotSpot 虚拟机为基础，修改 HotSpot 虚拟机中的类装载子系统以及运行时环境等，使这些环境能够符合龙芯处理器的结构设计，从而能够在龙芯硬件平台上稳定运行。在此基础上，根据龙芯处理器的硬件结构，对 HotSpot 虚拟机做了初步优化。使用 SPECjvm2008 测试套件测试，在龙芯处理器硬件平台上，以 Openjdk6 为基础设计的 Java 虚拟机可以稳定运行并且性能达到了较高水平。该高性能的 Java 虚拟机系统是本文 Java 虚拟机研究和性能评估的基础平台。

第二，通过软硬件协同的方式，设计并实现了一种混合的虚方法调用机制。Java 作为一种面向对象的语言，为了方便实现多态的特性，有大量的虚方法调用。由于虚方法只有在运行时才能获得该方法的对象是什么，所以对一个虚方法的解析就比较复杂，只

能用运行时动态解析的方式来确定目标方法。这种解析机制只能在运行到调用点时才能获得调用的类型信息，从而用该类型信息检索特定类的方法并且调用该方法。这种动态解析一般被编译成间接跳转，而这种间接跳转很容易引起流水线的停顿，限制了 Java 虚拟机的性能。本文设计并实现了硬件 CAM 查找表，提出了一种软硬件协同设计的混合的虚方法调用机制。实验表明，采用本方法，virtual_test 测试程序虚方法调用点的类型比较命中率从 13.3%升高到了 76.4%，性能有了 16.2%的提升；SPECjvm98 性能平均提升 6.4% 。

第三，通过硬件设计，优化了 Java 虚拟机中的数组越界检查和上下文切换两部分访存机制。Java 程序中数组的长度是动态决定的，数组的分配之初，不能够确定数组的真正长度，为了保证 Java 程序运行的正确性，必须在运行时检查数组访问是否越界，这种附加的运行时开销很大；另外，Java 虚拟机在内存中移动生成代码和 Java 栈保存恢复时需要用到大量的访存操作，而传统的按字访问存储器效率较低，不能满足 Java 虚拟机上下文切换的需求。本文实现了有界访存和 128 位访存指令，提出了一种软硬件结合的访存优化方法，有效地降低了数组越界检查的开销，提高了内存拷贝的效率，优化了 Java 程序的整体性能。使用有界访存指令，科学计算类测试 SciMark2.0 获得了 27%的性能提升，通用测试 SPECjvm2008 获得了 1.5%的性能提升；使用 128 位访存指令，SPECjvm2008 性能平均提升 5.9%。

第四，设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。通过对 Java 虚拟机即时编译器的编译行为的分析，认为编译生成的大量可执行代码在 cache 中的频繁替换是即时编译器的性能瓶颈，因为当生成代码被替换出 cache 后，处理器取指执行时必须访问速度较慢的主存储器。从这一点出发，本文设计并实现了硬件 cache 锁机制，提出了一种软硬件协同设计的即时编译方法。通过该方法，生成代码执行时的 cache 失效次数降低了 6.9%，测试 SPECjvm2008 中程序最高获得了 17.9%的性能提升，平均性能提升 4.2%。

7.3 未来研究方向

Java 虚拟机是一个非常复杂的系统，本文的工作只是针对 Java 虚拟机的主要部分——类装载器、运行时环境和执行引擎展开。在今后的研究工作中，我们可以进行如下进一步的研究。

第一，我们设计的 Java 虚拟机执行引擎使用了较为高效的自适应方法，Java 字节码在被载入之后，首先是要被解释器解释执行，达到一定条件后该方法才会被编译执行。而解释器目前是单条字节码边解释边执行，没有任何的优化措施。这样虽然启动开销小，

对于代码重复性很少的程序，大部分的字节码都需要解释执行，性能相对较低。

Java 虚拟机中的编译器是以方法为单位的较为复杂的编译器，因此 Java 虚拟机中才引入了简单的解释器。如果我们可以借鉴二进制编译系统中以基本块为单位的轻量级的编译器，替代原有系统中低效的解释器，这样既没有引入太大的编译开销，又大大提高了性能。我们还可以在此基础上进一步研究多线程的编译和执行，以减少程序在编译过程中的空等时间，从而实现一种高效的执行引擎。

第二，本文的研究都是基于单核处理器进行的，随着处理器体系结构的发展，多核结构逐渐成为一种趋势， Java 虚拟机只在单核上执行无法发挥处理器的优势。Java 虚拟机在运行期间有多个线程同时工作，比如编译线程、执行线程、垃圾收集线程等等。我们可以利用处理器的多核结构，将这些线程分别绑定在不同处理器核上工作，提高多核结构的利用率。在此基础上，可以进一步研究一种根据处理器占用率，自适应在多个处理器核间动态切换线程的系统，这种多核多线程的 Java 虚拟机系统对于最大程度发挥处理器的性能有很重要的意义。

第三，本文的研究是基于客户版的 Java HotSpot 虚拟机开展的，服务器版的 Java HotSpot 虚拟机虽然编译启动时间长，但是对于长时间运行的服务器类任务来说，具有更深层次的优化。可以在服务器版的 Java 虚拟机上实现本文的工作，并结合硬件结构特征，进一步寻找优化方向。

参考文献

[1] J. E. Smith and R. Nair, *Virtual machines: versatile platforms for systems and processes*: Morgan Kaufmann Pub, 2005.

[2] J. Gosling, *The Java language specification*: Prentice Hall, 2000.

[3] W. W. Hu, F. X. Zhang, and Z. S. Li, *Microarchitecture of the Godson-2 processor*. Journal of Computer Science and Technology. 20(2): pp. 243-249, 2005.

[4] W. W. Hu, J. Wang, X. Gao, Y. Chen, Q. Liu, and G. Li, *Godson-3: A scalable multicore RISC processor with X86 emulation*. Micro, IEEE. 29(2): pp. 17-29, 2009.

[5] W. W. Hu, J. Y. Zhao, S. Q. Zhong, X. Yang, E. Guidetti, and C. Wu, *Implementing a 1GHz four-issue out-of-order execution microprocessor in a standard cell ASIC methodology*. Journal of Computer Science and Technology. 22(1): pp. 1-14, 2007.

[6] Ebcio lu K, Altman ER. DAISY: dynamic compilation for 100% architectural compatibility, *In: Proceedings of the 24th annual international symposium on Computer architecture*. Denver, Colorado, United States: ACM, pp. 26-37, 1997.

[7] P. Guide. *Intel(R) 64 and IA-32 Architectures Software Developer's Manual*. 2010.

[8] B. Venners, *Inside the Java virtual machine*: McGraw-Hill Professional, 1999.

[9] E. R. Altman, K. Ebcioglu, M. Gschwind, and S. Sathaye. Advances and future challenges in binary translation and optimization. *In: Proceedings of the IEEE*. 89(11): pp. 1710-1722, 2001.

[10] M. G. Burke, J.-D. Choi, S. Fink, D. Grove, M. Hind, V. Sarkar, M. J. Serrano, V. C. Sreedhar, H. Srinivasan, and J. Whaley. The Jalapeno dynamic optimizing compiler for Java, *In: Proceedings of the ACM 1999 conference on Java Grande*. San Francisco, California, United States: ACM, pp. 129-141, 1999.

[11] Cierniak M, Lueh GY, Stichnoth JM. Practicing JUDO: Java under dynamic optimizations, *In: Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation*. Vancouver, British Columbia, Canada: ACM, pp. 13-26, 2000.

[12] D. Detlefs and O. Agesen, *Inlining of virtual methods*. ECOOP'99 Object-Oriented Programming. pp. 668-668, 1999.

[13] K. Ishizaki, M. Kawahito, T. Yasue, M. Takeuchi, T. Ogasawara, T. Suganuma, T. Onodera, H. Komatsu, and T. Nakatani. Design, implementation, and evaluation of

optimizations in a just-in-time compiler, *In: Proceedings of the ACM 1999 conference on Java Grande.* San Francisco, California, United States: ACM, pp. 119-128, 1999.

[14] K. Ishizaki, M. Kawahito, T. Yasue, H. Komatsu, and T. Nakatani. A study of devirtualization techniques for a Java Just-In-Time compiler, *In: Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. Minneapolis, Minnesota, United States: ACM, pp. 294-310, 2000.

[15] M. Kawahito, H. Komatsu, and T. Nakatani, *Effective null pointer check elimination utilizing hardware trap.* ACM SIGPLAN Notices. 35(11): pp. 139-149, 2000.

[16] M. Paleczny, C. Vick, and C. Click. The java hotspot TM server compiler, *In: Proceedings of the 2001 Symposium on Java TM Virtual Machine Research and Technology Symposium - Volume 1*. Monterey, California: USENIX Association, pp. 1-1, 2001.

[17] T. Suganuma, T. Ogasawara, M. Takeuchi, T. Yasue, M. Kawahito, K. Ishizaki, H. Komatsu, and T. Nakatani, *Overview of the IBM Java just-in-time compiler.* IBM Systems Journal. 39(1): pp. 175-193, 2000.

[18] B.-S. Yang, S.-M. Moon, S. Park, J. Lee, S. Lee, J. Park, Y. C. Chung, S. Kim, K. Ebcioglu, and E. Altman. LaTTe: A Java VM Just-in-Time Compiler with Fast and Efficient Register Allocation, *In: Proceedings of the 1999 International Conference on Parallel Architectures and Compilation Techniques: IEEE Computer Society*, pp. 128, 1999.

[19] S. Meloan, *The Java HotSpot performance engine: An in-depth look.* Sun Java Developer Connection, 1999.

[20] D. Sweetman, *See MIPS run*: Morgan Kaufmann, 2007.

[21] J. Heinrich, *MIPS R4000 Microprocessor User's Manual*: MIPS Technologies, 1994.

[22] J. Auslander, M. Philipose, C. Chambers, S. J. Eggers, and B. N. Bershad. Fast, effective dynamic compilation, *In: Proceedings of the ACM SIGPLAN 1996 conference on Programming language design and implementation.* Philadelphia, Pennsylvania, United States: ACM, pp. 149-159, 1996.

[23] V. Bala, E. Duesterwald, and S. Banerjia. Dynamo: a transparent dynamic optimization system, *In: Proceedings of the ACM SIGPLAN 2000 conference on Programming language design and implementation*. Vancouver, British Columbia, Canada: ACM, pp. 1-12, 2000.

[24] Robert G. Burger , R. Kent Dybvig. An Infrastructure for Profile-Driven Dynamic

Recompilation, *In: Proceedings of the 1998 International Conference on Computer Languages*, pp. 240, 1998.

[25] G. Burger, *Efficient compilation and profile-driven dynamic recompilation in scheme*, Indiana University, Indianapolis, 1997.

[26] C. Consel. A general approach for run-time specialization and its application to C, *In: Proceedings of the 23rd ACM SIGPLAN-SIGACT symposium on Principles of programming languages.* St. Petersburg Beach, Florida, United States: ACM, pp. 145-156, 1996.

[27] B. Grant, M. Mock, M. Philipose, C. Chambers, and S. J. Eggers, *DyC: An expressive annotation-directed dynamic compiler for C.* Theoretical Computer Science. 248(1-2): pp. 147-199, 2000.

[28] B. Grant, M. Philipose, M. Mock, C. Chambers, and S. J. Eggers, *An evaluation of staged run-time optimizations in DyC.* ACM SIGPLAN Notices. 34(5): pp. 293-304, 1999.

[29] T. P. Kistler, *Continuous program optimization*: University of California, Irvine, 1999.

[30] M. Leone and P. Lee, *Dynamic specialization in the Fabius system.* ACM Computing Surveys (CSUR). 30(3es): pp. 23-es, 1998.

[31] R. Marlet, C. Consel, and P. Boinot, *Efficient incremental run-time specialization for free.* ACM SIGPLAN Notices. 34(5): pp. 281-292, 1999.

[32] M. Poletto, D. R. Engler, and M. F. Kaashoek. tcc: a system for fast, flexible, and high-level dynamic code generation, *In: Proceedings of the ACM SIGPLAN 1997 conference on Programming language design and implementation.* Las Vegas, Nevada, United States: ACM, pp. 109-121, 1997.

[33] B. Alpern, C. R. Attanasio, J. J. Barton, M. G. Burke, P. Cheng, J. D. Choi, A. Cocchi, S. J. Fink, D. Grove, and M. Hind, *The Jalapeno virtual machine.* IBM Systems Journal. 39(1): pp. 211-238, 2000.

[34] B. Alpern, C. R. Attanasio, A. Cocchi, D. Lieber, S. Smith, T. Ngo, J. J. Barton, S. F. Hummel, J. C. Sheperd, and M. Mergen. Implementing Jalapeno in Java, *In: Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications.* Denver, Colorado, United States: ACM, pp. 314-324, 1999.

[35] M. Arnold, S. Fink, D. Grove, M. Hind, and P. F. Sweeney. Adaptive optimization in the Jalapeno JVM, *In: Proceedings of the 15th ACM SIGPLAN conference on*

Object-oriented programming, systems, languages, and applications. Minneapolis, Minnesota, United States: ACM, pp. 47-65, 2000.

[36] M. Arnold, S. Fink, D. Grove, M. Hind, and P.F. Sweeney. Adaptive Optimizations in the Jalapeno JVM: The Controller's Analytical Model. *In: Proceedings of the ACM SIGPLAN Workshop on Feedback-Directed and Dynamic Optimization*, 2000.

[37] A. R. Adl-Tabatabai, M. Cierniak, G. Y. Lueh, V. M. Parikh, and J. M. Stichnoth, *Fast, effective code generation in a just-in-time Java compiler.* ACM SIGPLAN Notices. 33(5): pp. 280-290, 1998.

[38] Standard Performance Evaluation Corporation. *SPEC JVM98 Benchmarks.* Available at http://www.spec.org/osg/jvm98

[39] U. H lzle, *Adaptive optimization for SELF: Reconciling high performance with exploratory programming.* Sun Microsystems, Inc. Mountain View, CA, USA, 1995.

[40] U. H lzle and D. Ungar, *Reconciling responsiveness with performance in pure object-oriented languages.* ACM Transactions on Programming Languages and Systems (TOPLAS). 18(4): pp. 355-400, 1996.

[41] C. Chambers and D. Ungar, *Making pure object-oriented languages practical.* ACM SIGPLAN Notices. 26(11): pp. 1-15, 1991.

[42] L. P. Deutsch and A. M. Schiffman. Efficient implementation of the smalltalk-80 system, *In: Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of programming languages.* Salt Lake City, Utah, United States: ACM, pp. 297-302, 1984.

[43] Sun Microsystems. *The Java Hotspot Performance Engine Architecture*. White paper available at http://java.sun.com/products/hotspot/index.html, 2001.

[44] OpenJDK 2007. *Homepage of the Open-Source JDK Community.* Sun Microsystems, Inc. http://openjdk.java.net/

[45] U. Hölzle, D. Ungar. Optimizing dynamically-dispatched calls with run-time type feedback, *In: Proceedings of the ACM SIGPLAN 1994 conference on Programming language design and implementation.* Orlando, Florida, United States: ACM, pp. 326-336, 1994.

[46] S. J. Fink and F. Qian. Design, implementation and evaluation of adaptive recompilation with on-stack replacement, *In: Proceedings of the international symposium on Code generation and optimization: feedback-directed and runtime optimization.* San Francisco, California: IEEE Computer Society, pp. 241-252, 2003.

[47] R. Griesemer and S. Mitrovic, *A compiler for the Java HotSpot(tm) virtual machine.* The School of Niklaus Wirth: The Art of Simplicity. pp. 133-152.

[48] O. Agesen, D. Detlefs, A. Garthwaite, R. Knippel, Y. Ramakrishna, and D. White, *An efficient meta-lock for implementing ubiquitous synchronization.* Sun Microsystems, Inc. Mountain View, CA, USA, 1999.

[49] D. F. Bacon, R. Konuru, C. Murthy, and M. Serrano. Thin locks: featherweight synchronization for Java, *In: Proceedings of the ACM SIGPLAN 1998 conference on Programming language design and implementation.* Montreal, Quebec, Canada: ACM, pp. 258-268, 1998.

[50] K. Russell and D. Detlefs, *Eliminating synchronization-related atomic operations with biased locking and bulk rebiasing.* ACM SIGPLAN Notices. 41(10): pp. 263-272, 2006.

[51] D. Ungar. *Generation scavenging: A non-disruptive high performance storage reclamation algorithm.* ACM SIGSOFT Software Engineering Notes. 9(3): pp. 157-167, 1984.

[52] H.-J. Boehm, A. J. Demers, and S. Shenker. Mostly parallel garbage collection, *In: Proceedings of the ACM SIGPLAN 1991 conference on Programming language design and implementation.* Toronto, Ontario, Canada: ACM, pp. 157-164, 1991.

[53] T. Printezis and D. Detlefs. A generational mostly-concurrent garbage collector, *In: Proceedings of the 2nd international symposium on Memory management.* Minneapolis, Minnesota, United States: ACM, pp. 143-154, 2000.

[54] U. Hölzle, C. Chambers, and D. Ungar. Debugging optimized code with dynamic deoptimization, *In: Proceedings of the ACM SIGPLAN 1992 conference on Programming language design and implementation.* San Francisco, California, United States: ACM, pp. 32-43, 1992.

[55] V. Bala, E. Duesterwald, and S. Banerjia, *Transparent dynamic optimization: The design and implementation of Dynamo*, Citeseer 1999.

[56] Sanjeev Banerjia Vasanth Bala, Evelyn Duesterwald. *Transparent dynamic optimization.* Technical report, HP Laboratories Cambridge, 1999.

[57] H. McGhan and M. O'Connor, *Picojava: A direct execution engine for java bytecode.* Computer. 31(10): pp. 22-30, 1998.

[58] W. Puffitsch and M. Schoeberl. picoJava-II in an FPGA, *In: Proceedings of the 5th international workshop on Java technologies for real-time and embedded systems.*

Vienna, Austria: ACM, pp. 213-221, 2007.

[59] P. Scientific. *Java Processor PSC1000*. electronic industry. H 2, S 51 f, 1998.

[60] R. Radhakrishnan, R. Bhargava, and L. K. John. Improving Java performance using hardware translation, *In: Proceedings of the 15th international conference on Supercomputing.* Sorrento, Italy: ACM, pp. 427-439, 2001.

[61] M. Tremblay. MAJC(tm): An Architecture for the New Millennium, *In: Proceedings of Hot Chips 11*, pp. 275-288, 1999.

[62] W. W. Hu, F. X. Zhang, and Z. S. Li, *Microarchitecture of the Godson-2 processor*. Journal of Computer Science and Technology. 20(2): pp. 243-249, 2005.

[63] 胡伟武，张福新，李祖松. *龙芯 2 号处理器设计和性能分析*. 计算机研究与发展, 43:959－966, 2006.

[64] Standard Performance Evaluation Corporation. *SPEC JVM2008 Benchmarks.* Available at http://www.spec.org/osg/jvm2008

[65] K. Driesen, *Efficient Polymorphic Calls*: Springer Netherlands, 2001.

[66] K. Driesen, P. Lam, J. Miecznikowski, F. Qian, and D. Rayside. On the predictability of Java byte codes (abstract) (poster session), *In: Addendum to the 2000 proceedings of the conference on Object-oriented programming, systems, languages, and applications (Addendum).* Minneapolis, Minnesota, United States: ACM, pp. 127-128, 2000.

[67] D. F. Bacon and P. F. Sweeney, *Fast static analysis of C++ virtual function calls.* ACM SIGPLAN Notices. 31(10): pp. 324-341, 1996.

[68] D. M. Ungar, *The design and evaluation of a high performance Smalltalk system.* 1986.

[69] D. Ungar and D. Patterson, *What Price Smalltalk.* Computer;(United States). 20(1), 1987.

[70] http://en.wikipedia.org/wiki/Inline_caching

[71] J. Dean, D. Grove, and C. Chambers. Optimization of Object-Oriented Programs Using Static Class Hierarchy Analysis, *In: ECOOP'95 Object-Oriented Programming, 9th European Conference*, Åarhus, Denmark, August 7–11, 1995, vol. 952, Lecture Notes in Computer Science, M. Tokoro and R. Pareschi, Eds.: Springer Berlin / Heidelberg, pp. 77-101, 1995.

[72] U. Pagh Schultz, J. Lawall, C. Consel, and G. Muller, *Towards automatic specialization of Java programs.* ECOOP'99 Object-Oriented Programming. pp. 668-669, 1999.

[73] V. Sundaresan, L. Hendren, C. Razafimahefa, R. Vallée-Rai, e-Rai, P. Lam, E. Gagnon,

and C. Godin. Practical virtual method call resolution for Java, *In: Proceedings of the 15th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications.* Minneapolis, Minnesota, United States: ACM, pp. 264-280, 2000.

[74] G. Aigner and U. Hölzle, *Eliminating virtual function calls in C++ programs.* ECOOP'96 Object-Oriented Programming. pp. 142, 1996.

[75] U. Hölzle, C. Chambers, and D. Ungar. Optimizing dynamically-typed object-oriented languages with polymorphic inline caches, *In: ECOOP'91 European Conference on Object-Oriented Programming,* vol. 512, Lecture Notes in Computer Science, P. America, Ed.: Springer Berlin / Heidelberg, pp. 21-38, 1991.

[76] G. McGary, *Bounds Checking in C & C++ Using Bounded Pointers*, 2000

[77] G. C. Necula, S. McPeak, and W. Weimer. CCured: type-safe retrofitting of legacy code, *In: Proceedings of the 29th ACM SIGPLAN-SIGACT symposium on Principles of programming languages.* Portland, Oregon: ACM, pp. 128-139, 2002.

[78] T. M. Austin, S. E. Breach, and G. S. Sohi. Efficient detection of all pointer and array access errors, *In: Proceedings of the ACM SIGPLAN 1994 conference on Programming language design and implementation.* Orlando, Florida, United States: ACM, pp. 290-301, 1994.

[79] T. Jim, G. Morrisett, D. Grossman, M. Hicks, J. Cheney, and Y. Wang. Cyclone: A Safe Dialect of C, *In: USENIX Annual Technical Conference,* Monterey, CA, pp. 275–288, 2002.

[80] M. Harren and G. C. Necula, *Lightweight wrappers for interfacing with binary code in ccured.* Software Security-Theories and Systems. pp. 209-225, 2004.

[81] J. M. Asuru, *Optimization of array subscript range checks.* ACM Letters on Programming Languages and Systems (LOPLAS). 1(2): pp. 109-118, 1992.

[82] P. Zhou, W. Liu, L. Fei, S. Lu, F. Qin, Y. Zhou, S. Midkiff, and J. Torrellas. AccMon: Automatically Detecting Memory-Related Bugs via Program Counter-Based Invariants, *In: Proceedings of the 37th annual IEEE/ACM International Symposium on Microarchitecture.* Portland, Oregon: IEEE Computer Society, pp. 269-280, 2004.

[83] J. Newsome and D. Song. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. *In: Proceedings of NDSS'05,* San Diego, California, USA, 2005

[84] J. R. Crandall and F. T. Chong. Minos: Control Data Attack Prevention Orthogonal to Memory Model, *In: Proceedings of the 37th annual IEEE/ACM International*

Symposium on Microarchitecture. Portland, Oregon: IEEE Computer Society, pp. 221-232, 2004.

[85] G. E. Suh, J. W. Lee, D. Zhang, and S. Devadas. Secure program execution via dynamic information flow tracking, *In: Proceedings of the 11th international conference on Architectural support for programming languages and operating systems.* Boston, MA, USA: ACM, pp. 85-96, 2004.

[86] R. Bodik, R. Gupta, and V. Sarkar, *ABCD: eliminating array bounds checks on demand.* ACM SIGPLAN Notices. 35(5): pp. 321-333, 2000.

[87] F. Qian, L. Hendren, and C. Verbrugge. A Comprehensive Approach to Array Bounds Check Elimination for Java, *In: Compiler Construction, vol. 2304, Lecture Notes in Computer Science,* R. Horspool, Ed.: Springer Berlin / Heidelberg, pp. 117-135, 2002.

[88] J. Hummel, A. Azevedo, D. Kolson, and A. Nicolau, *Annotating the Java bytecodes in support of optimization.* Concurrency: Practice and Experience. 9(11): pp. 1003-1016, 1997.

[90] O. Avissar, R. Barua, and D. Stewart, *An optimal memory allocation scheme for scratch-pad-based embedded systems.* ACM Transactions on Embedded Computing Systems (TECS). 1(1): pp. 6-26, 2002.

[91] R. Banakar, S. Steinke, B.-S. Lee, M. Balakrishnan, and P. Marwedel. Scratchpad memory: design alternative for cache on-chip memory in embedded systems, *In: Proceedings of the tenth international symposium on Hardware/software codesign.* Estes Park, Colorado: ACM, pp. 73-78, 2002.

[92] P. R. Panda, N. D. Dutt, and A. Nicolau, *On-chip vs. off-chip memory: the data partitioning problem in embedded processor-based systems.* ACM Transactions on Design Automation of Electronic Systems (TODAES). 5(3): pp. 682-704, 2000.

[93] A. Gordon-Ross, S. Cotterell, and F. Vahid, *Exploiting fixed programs in embedded systems: A loop cache example.* IEEE Computer Architecture Letters, 2002.

[94] D. Chiou, P. Jain, L. Rudolph, and S. Devadas. Application-specific memory management for embedded systems using software-controlled caches, *In: Proceedings of the 37th Annual Design Automation Conference.* Los Angeles, California, United States: ACM, pp. 416-419, 2000.

[95] J. B. Sartor, V. Subramaniam, K. S. McKinley, and Z. Wang. Cooperative caching with keep-me and evict-me, *In: Interaction between Compilers and Computer Architectures, 2005.* INTERACT-9. 9th Annual Workshop on, 2005.

[96] M. Campoy, A. P. Ivars, and J. V. Busquets-Mataix. Static use of locking caches in multitask preemptive real-time systems. *In: Proceedings of IEEE/IEE Real-Time Embedded Systems Workshop (Satellite of the IEEE Real-Time Systems Symposium),* 2001.

[97] I. Puaut and D. Decotigny. Low-complexity algorithms for static cache locking in multitasking hard real-time systems, *In: Real-Time Systems Symposium, 2002. RTSS 2002.* 23rd IEEE, 2002.

[98] A. M. Campoy, A. Perles, F. Rodriguez, and J. V. Busquets-Mataix. Static use of locking caches vs. dynamic use of locking caches for real-time systems, *In: Electrical and Computer Engineering, 2003.* IEEE CCECE 2003. Canadian Conference on, 2003.

[99] X. Vera, B. Lisper, and X. Jingling. Data caches in multitasking hard real-time systems, *In: Real-Time Systems Symposium, 2003. RTSS 2003.* 24th IEEE, 2003.

[100] A. Arnaud and I. Puaut. Dynamic instruction cache locking in hard real-time systems. *In: Proceedings of the 14th International Conference on Real-Time and Network Systems (RNTS),* Poitiers, France, 2006.

[101] M. Campoy, P. Jiménez, P. Ivars, B. Mataix. Using Genetic Algorithms in Content Selection for Locking-Caches. *In: Proceedings of the IASTED International Symposia Applied Informatics, Acta Press.* Innsbruck, Austria. pp. 271–276, 2001.

致谢

在本论文完成之际，我要衷心感谢我的导师胡伟武研究员。是胡老师让我有机会进入到龙芯这个大家庭中来，让我能够参与到龙芯处理器设计这个振奋人心的事业中。在研究上，胡老师在百忙之中给了我大量指导，使我明确了博士期间的研究方向；在工作中，胡老师忘我的工作作风令人敬佩；在生活中，胡老师强烈的民族自尊心和爱国热情影响着我们，让我们时刻不忘要做一个对国家有用的人。胡老师为龙芯事业呕心沥血，他渊博的学识、执着进取的敬业精神、踏实严谨勇于创新的精神深深地感染着我，让我受益匪浅；胡老师做人、做事、做学问的教诲更是令我永远铭记在心。

感谢龙芯公司的谷总、晋老师、张总和辉总在工作和生活上对我的关心和照顾，感谢他们在我研究和论文期间对我的鼓励和支持。

感谢王剑、章隆兵、沈海华、高翔、陈云霁等老师，他们在我几年的研究生生涯中给予了巨大的帮助，无论在工作学习还是研究上都给了我很多的指导，让我能够在工作和学习中一天天进步。感谢陆京老师、刘凤琴老师、杨杰老师，他们在我日常生活上的莫大帮助，让我没有后顾之忧，使我能够专心投身于紧张的工作和研究中。

感谢 Java 工作小组的同学们，他们直接参与了论文的研究工作，对本文的工作很大的帮助和贡献。他们工作认真主动、踏实刻苦，正是有了他们的大力支持，我才有更多的精力投入到研究和论文中。杨勇强、敖琪、林传文，谢谢你们。感谢陈天石对论文撰写的指导，在论文的完成之际，陈天石还对论文进行了细心的检查和修改。

感谢刘奇对我的热心支持和大力帮助，刘奇是我生活中的朋友，更是我学习和工作中的老师，是他手把手的将我领进了系统软件的大门，让我有着明确的方向一路走来；感谢郝守青、汪文祥、王焕东、王茹、苏孟豪、蔡万伟、孟小甫、孙明明，我们一起工作一起进步一起成长，从他们身上，我学到了许多；感谢李文刚、钱天柱、曹磊、沈晓宁、徐德法，我们是同事，也是朋友，正是有了他们的支持，我才有信心和勇气攻克一个又一个难关。

感谢我的女友媛媛，她在生活中给予了我无微不至的关心，有了她在我背后的照顾，才使我能够专注于学业。在我工作和生活最迷茫的时候，是她在背后默默地支持我，让我能够勇敢地奋斗下去。

最后特别感谢我的父母，多年来他们含辛茹苦，为我提供了良好的学习条件和宽松的学习环境。没有他们的支持鼓励，就没有我的今天。我要用我今后的努力和拼搏来报答他们的养育之恩。

谨以此文献给所有关心、帮助和支持我的人。

作者简历

姓名：蔡嵩松　　性别：男　　出生日期：1986.1.23　　籍贯：山东潍坊

2008.9 — 至今	中国科学院计算技术研究所	计算机系统结构	博士研究生
2005.9 — 2008.7	首都师范大学	计算机应用技术	硕士研究生
2001.9 — 2005.7	中国科学技术大学	少年班	本科

【攻读博士学位期间发表的论文】

[1] Songsong Cai, Yongqiang Yang, Chuanwen Lin, Qi Liu. “JVM virtual method invoking optimization based on CAM table”. In Proceedings of Networking, Architecture, and Storage, 2011.

[2] 蔡嵩松，刘奇，沈海华，章隆兵，跨平台系统级虚拟机的访存优化，计算机研究与发展（增刊）。（已录用）

[3] 敖琪，蔡嵩松，王剑，基于硬件 cache 锁机制的 Java 虚拟机即时编译器优化，计算机研究与发展（增刊）。（已录用）

[4] 蔡嵩松，刘奇，王剑，刘金刚。基于龙芯处理器的二进制翻译器优化。计算机工程，第 35 卷第 7 期，2009。

[5] Weiwu Hu, Qi Liu, Jian Wang, Songsong Cai, Menghao Su and Xiaoyu Li. “Efficient Binary Translation System with Low Hardware Cost”. In Proceedings of the 27th IEEE International Conference of Computer Design (ICCD), 2009.

[6] 胡伟武，蔡嵩松，李国杰。一种 RISC 处理器装置及其有界访存方法。发明专利，申请号：200810057216.6，已受理。

[7] 蔡嵩松，苏孟豪。RISC 处理器及其浮点寄存器的非对齐访存方法。发明专利，申请号：200810055928.4，已受理。

[8] 蔡嵩松，李晓钰。精简指令集计算机处理器装置及其数据处理方法。发明专利，申请号：200810056665.9，已受理。

【攻读博士学位期间参加的科研项目】

[1] 2006－2010 国家 973 计划项目，延长硅摩尔定律微处理器芯片的新原理、新结构、新方法研究（2005CB321600）

[2] 2008－2012 国家自然基金重点项目，高性能片上存储系统（60736012）

[3] 2008－2010 国家 863 计划重点项目，四核龙芯通用 CPU 研制(2008AA110900)

【攻读博士学位期间的获奖情况】

2009 年度腾讯科技卓越奖学金优秀奖